

SPPU DS Exam Prompt

05/15/2026 00:16:56

Your prompt is already very strong for generating high-scoring SPPU DS...

Nov_Dec_2024 (1).pdf Nov_Dec_2023 (1).pdf May_Jun_2025 (2).pdf...

SPPU BE IT Distributed Systems (DS) PYQ Analysis Most Repeated → Less...

i want exact question which is appeared in exam higher to lower reated till 1...

UNIT 3 (Q1, Q2) Exact Questions Appeared in Exam Higher Repeated → Lower...

UNIT 3 (Q1, Q2) "What is the purpose of Message Passing Interface? Illustrate...

Q. What is the purpose of Message Passing Interface? Illustrate the architectur...

[9 Marks] "What is MPI? Describe the point-to-point communication in MPI wit...

Q. What is MPI? Describe the point-to-point communication in MPI with suitabl...

2 Overlay Network — [8M / 9M] [9 Marks] "Discuss the purpose of overlay...

Q. Discuss the purpose of Overlay Network. Describe in brief any three types o...

[9 Marks] "What is Network virtualization? What are the types of overlay...

Q. What is Network Virtualization? What are the types of Overlay Networks?...

3 Clock Synchronization — [8M / 9M] [9 Marks] "Why is clock synchronization...

Q. Why is clock synchronization using a global time impossible in a distributed...

[8 Marks] "Discuss the problem of clock synchronization in distributed operati...

Q. Discuss the problem of clock synchronization in distributed operating...

4 Bully Algorithm — [9M] [9 Marks] "Explain the goal of an Election algorithm...

Q. Explain the goal of an Election Algorithm. Illustrate the Bully Algorithm usin...

5 XDR / Marshalling / Unmarshalling — [8M] [8 Marks] "Define External Data...

Q. Define External Data Representation, Marshalling and Unmarshalling....

6 Mutual Exclusion — [9M] [9 Marks] "What are the two important properties...

Q. What are the two important properties of token-based approach? Explain...

7 NTP — [9M] [9 Marks] "What is NTP? With the help of a diagram, describe...

Q. What is NTP? With the help of a diagram, describe how NTP works. (9 Marks...

8 Lamport Logical Clock — [8M] [8 Marks] "What are the advantages of logica...

Q. What are the advantages of logical clock over physical clock? Describe...

UNIT 4 (Q3, Q4) 🔥 4 TIMES REPEATED 1 Checkpointing — [9M] [9 Marks]...

Q. What is checkpointing in a distributed system? Explain the working of...

2 Replica Management / Replication — [9M] [9 Marks] “What are the key issu...

Q. What are the key issues in Replica Management? Explain the following with...

[9 Marks] “Why replication is important and how it relates with Scalability? Ho...

Q. Why replication is important and how it relates with Scalability? How are...

3 Causal Consistency — [9M] [9 Marks] “What are two primary reasons for...

Q. What are two primary reasons for replication? Explain the causal consistenc...

5 Push vs Pull Protocol — [9M] [9 Marks] “Discuss and compare Push versus...

Q. Discuss and compare Push versus Pull Protocols propagation design issue f...

6 Two Phase Commit Protocol — [9M] [9 Marks] “What is the distribution...

Q. What is the distributed commit problem? Discuss how this problem is solve...

UNIT 5 (Q5, Q6) 🔥 4 TIMES REPEATED 1 DNS vs X.500 — [8M / 9M] [8 Marks]...

Q. What is a Directory Service? What is the difference between DNS and X.500?...

2 Apache Web Server — [9M] [9 Marks] “What are web services? Describe wit...

Q. What are Web Services? Describe with a suitable diagram the general...

3 Cooperative Caching — [8M / 9M] [8 Marks] “Explain in brief, the two places...

Q. Explain in brief the two places of client-side web caching. Explain cooperativ...

4 BitTorrent — [8M] [8 Marks] “Describe the design of a peer-to-peer file...

Q. Describe the design of a peer-to-peer file sharing application designed to...

UNIT 6 (Q7, Q8) 🔥 5 TIMES REPEATED 1 Mach & CHORUS — [9M] [9 Marks]...

Q. Provide an overview of Mach and CHORUS microkernels. How are memory...

3 SOA (Service Oriented Architecture) Exact Questions Nov/Dec 2023 — [9...

Q. What is Service Oriented Architecture (SOA)? Explain the various SOA...

4 Prometheus Exact Questions Nov/Dec 2023 — [9 Marks] “Explain in brief, th...

Q. Explain in brief the key features of Prometheus including data model, query...

5 Wearable Devices + PVM + JINI Exact Questions Nov/Dec 2024 — [9 Marks]...

Q. Explain the following in brief: i) Wearable Devices ii) PVM iii) JINI (9 Marks -...

6 Nagios + Zabbix Comparison May/Jun 2024 — [9 Marks] “Compare the...

Q. Compare the following tools for Distributed System Monitoring: Prometheu...

7 Wearable Computing Problems May/Jun 2024 — [9 Marks] “How wearable...

Q. How wearable devices work in distributed systems? Discuss the problems...

7 Wearable Computing Problems May/Jun 2024 — [9 Marks] “How wearable...

Q. How wearable devices work in distributed systems? Discuss the problems...



Your prompt is already very strong for generating high-scoring SPPU DS answers. I improved and optimized it further so ChatGPT gives:

- more exam-oriented answers,
- cleaner structure,
- better diagrams,
- easier explanations,
- and more predictable topper-style output.

Here is the improved final version:

Final Optimized Prompt for SPPU BE IT Distributed Systems (DS)

Generate a **SPPU University BE IT Distributed Systems (DS) endsem exam-ready answer** that helps a student score **maximum marks in theory exams**. `Pasted text.txt`

Main Goal

The answer should help a student:

- easily understand Distributed Systems concepts,
- remember answers quickly,
- reproduce answers effectively in SPPU university exams,
- and write high-scoring theory answers even with weak technical knowledge.

Very Important Teaching Style

Teach the concept like explaining to a **10-year-old beginner student** who has no prior knowledge of Distributed Systems.

Assume the student:

- is weak in theory,
- is learning the topic for the first time,
- and forgets technical terms easily.

Whenever a difficult technical word is used:

- immediately explain it in very simple English.

Use:

- simple English,
- short sentences,
- logical explanation flow,
- relatable examples,
- and easy-to-remember explanations.

The student should feel:

"Now I can easily write this answer in SPPU exam."

Important Writing Rules

Write answers in proper **SPPU university topper answer-sheet style**.

Use:

- proper headings,
- subheadings,
- numbering,
- bullet points,
- tables,
- neat formatting,
- highlighted keywords,
- and step-by-step explanations.

Avoid:

- complicated English,
- unnecessary theory,
- long confusing paragraphs.

Mandatory Answer Structure

1 Simple Introduction (2-4 lines)

Explain topic in very simple beginner-friendly language.

2 Definition

Write:

- short,

- technical,
 - university-style definition.
-

3 Need of Concept

Explain:

- why this concept is needed,
 - what problem it solves,
 - where it is used in distributed systems.
-

4 Core Concept Explanation

Explain complete concept step-by-step.

Break difficult ideas into:

- small points,
- simple logic,
- easy understanding.

Explain every component separately.

5 Architecture / Components

If applicable:

- explain architecture,

- nodes,
 - communication,
 - processes,
 - synchronization,
 - coordination,
 - message passing,
 - servers/clients,
 - distributed components.
-

6 Working / Algorithm / Process

Explain:

- objective,
- steps,
- working,
- final result.

Use:

- numbering,
- simple language,
- logical flow.

For algorithms:

- explain each step clearly.
-

7 Diagram (VERY IMPORTANT)

Always include neat exam-style ASCII/text diagrams.

Examples:

- architecture diagram,
- client-server model,
- communication flow,
- synchronization flow,
- distributed node diagram,
- replication model,
- message passing flow.

Diagram should be:

- simple,
- easy to draw in exam,
- properly labeled.

After diagram:

- explain each component briefly.

8 Real-Life Analogy (MANDATORY)

For every DS concept:

give at least one very simple real-life analogy.

Examples:

- WhatsApp messaging,
- ATM network,

- Google Drive sync,
- online banking,
- railway reservation,
- multiplayer gaming,
- cloud storage,
- food delivery apps.

After analogy explain:

"How this analogy relates to the Distributed Systems concept."

9 Real-World Industry Example

Include practical examples from:

- Google
- Amazon
- Netflix
- WhatsApp
- YouTube
- Cloud Computing,
- Banking Systems,
- Distributed Databases,
- E-commerce,
- Social Media.

Explain clearly how the concept is used there.

Important DS Topics to Explain Clearly

Whenever applicable, explain:

- architecture,
- communication,
- synchronization,
- coordination,
- processes,
- distributed algorithms,
- protocols,
- message passing,
- fault tolerance,
- consistency,
- replication,
- concurrency,
- scalability,
- security,
- distributed coordination,
- load balancing,
- distributed databases.

Must Include These Sections Whenever Relevant

- Features
- Characteristics

- Advantages
- Disadvantages
- Applications
- Limitations
- Types
- Components
- Architecture
- Working Principle
- Algorithm Steps
- Security Issues
- Fault Tolerance
- Synchronization
- Scalability
- Challenges
- Performance Issues

These sections are highly repeated in SPPU papers.

Comparison Table Requirement

Whenever related concepts exist,
add short easy comparison tables.

Examples:

- Client-Server vs Peer-to-Peer
- RPC vs RMI
- Centralized vs Distributed System

- Thread vs Process
- Logical Clock vs Vector Clock

Table should be:

- simple,
 - scoring-oriented,
 - easy to memorize.
-

Memory Tricks / Mnemonics

Add:

- mnemonics,
- keyword tricks,
- revision shortcuts,
- quick memory tricks.

Example:

- CAP → Consistency, Availability, Partition Tolerance
 - FIFO → First In First Out
-

Important Exam Keywords

Highlight important university keywords such as:

- scalability,
- transparency,

- concurrency,
- synchronization,
- fault tolerance,
- replication,
- consistency,
- distributed coordination,
- message passing,
- load balancing.

These improve theory marks.

Conclusion

Write short conclusion:

- 2-3 lines,
 - scoring-oriented,
 - summarizing importance of topic.
-

Marks Format

For 5 Marks

- Short but complete answer
- Important points only
- Small diagram

- Applications + advantages

For 8 Marks

- Medium detailed answer
- Diagram + examples + working
- Advantages/disadvantages
- Real-life analogy

For 10 Marks

- Full detailed university answer
 - Architecture + diagrams + algorithm + examples
 - Comparison tables
 - Mnemonics
 - Real-world applications
 - Conclusion
-

Final Output Style

The final answer must feel like:

- topper handwritten notes,
- easy last-minute revision notes,
- exam-ready theory content,
- simple enough for weak students,
- but technical enough to score maximum marks.

Subject

Distributed Systems (DS) - BE IT - SPPU

Question

[Paste your Distributed Systems question here]

Marks

[5 / 6 / 8 / 10]

5/14/2026, 1:44:29 PM



Nov_Dec_2024 (1).pdf



Nov_Dec_2023 (1).pdf



May_Jun_2025 (2).pdf



May_Jun_2024 (1).pdf



May_Jun_2023 (2).pdf

i am a final yera enginerring student studied in sppu brach IT

here i attched pyq can u extract most repeater to less repeated question in unit 3 means Q.1,2 and uit 4 means Q.3,4, unit 5 means Q.5,6 and unit 6 means Q.7,8

5/14/2026, 1:52:19 PM



SPPU BE IT Distributed Systems (DS) PYQ Analysis

Most Repeated → Less Repeated Questions

(Units based on your mapping)

- Unit 3 → Q1, Q2
- Unit 4 → Q3, Q4
- Unit 5 → Q5, Q6
- Unit 6 → Q7, Q8

Analyzed Papers:

- May/Jun 2023 `May_Jun_2023 (2).pdf`
- May/Jun 2024 `May_Jun_2024 (1).pdf`
- Nov/Dec 2023 `Nov_Dec_2023 (1).pdf`
- Nov/Dec 2024 `Nov_Dec_2024 (1).pdf`
- May/Jun 2025 `May_Jun_2025 (2).pdf`

UNIT 3 (Q1, Q2)

MOST REPEATED TOPICS

VERY HIGH REPEATED (Most Important)

Topic	Repeated
MPI (Message Passing Interface)	3 Times
Overlay Network	3 Times
Clock Synchronization	3 Times
Berkeley Algorithm	2 Times
Bully Election Algorithm	2 Times
XDR / Marshalling / Unmarshalling	2 Times

MEDIUM REPEATED

Topic	Repeated
Mutual Exclusion Algorithms	2 Times
Lamport Logical Clock	2 Times
NTP	1 Time
Token Ring Algorithm	1 Time

UNIT 3 IMPORTANT QUESTIONS PRIORITY

1 MPI (VERY IMPORTANT)

Asked in:

- Nov/Dec 2024 [Nov_Dec_2024 \(1\).pdf](#)
- Nov/Dec 2023 [Nov_Dec_2023 \(1\).pdf](#)
- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)

Study:

- MPI architecture
- send/receive primitives
- MPI communication types
- point-to-point communication

2 Overlay Network

Asked in:

- Nov/Dec 2024 [Nov_Dec_2024 \(1\) .pdf](#)
- May/Jun 2024 [May_Jun_2024 \(1\) .pdf](#)
- May/Jun 2023 [May_Jun_2023 \(2\) .pdf](#)

Study:

- Definition
 - Types
 - Advantages
 - Skype overlay architecture
-

3 Clock Synchronization

Asked in:

- Nov/Dec 2024 [Nov_Dec_2024 \(1\) .pdf](#)
- Nov/Dec 2023 [Nov_Dec_2023 \(1\) .pdf](#)
- May/Jun 2025 [May_Jun_2025 \(2\) .pdf](#)

Study:

- Berkeley Algorithm
 - NTP
 - Clock synchronization problems
 - Logical vs physical clocks
-

4 Election Algorithms

Asked in:

- May/Jun 2025 (Bully Algorithm) `May_Jun_2025 (2).pdf`
- May/Jun 2023 (Bully Algorithm) `May_Jun_2023 (2).pdf`

Study:

- Goal
 - Working
 - Steps
 - Diagram
-

Prediction for Exam (Unit 3)

Most probable:

1. MPI
 2. Overlay Network
 3. Clock Synchronization
 4. Bully Algorithm
 5. Mutual Exclusion
-

UNIT 4 (Q3, Q4)

MOST REPEATED TOPICS

 **VERY HIGH REPEATED**

Topic	Repeated
Checkpointing & Coordinated Checkpointing	4 Times
Causal Consistency Model	3 Times
Replica Management / Replication	4 Times
Primary Based Protocol	3 Times

MEDIUM REPEATED

Topic	Repeated
Fault Tolerance	2 Times
Two Phase Commit Protocol	1 Time
RPC Failure Handling	1 Time
Push vs Pull Protocol	1 Time

UNIT 4 IMPORTANT QUESTIONS PRIORITY

1 Checkpointing (SUPER IMPORTANT)

Asked in:

- Nov/Dec 2024 [Nov_Dec_2024 \(1\).pdf](#)
- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)
- May/Jun 2024 [May_Jun_2024 \(1\).pdf](#)

- May/Jun 2023 [May_Jun_2023 \(2\) .pdf](#)

Study:

- Definition
 - Need
 - Coordinated checkpointing
 - Recovery mechanism
 - Diagram
-

2 Replication / Replica Management

Asked in:

- Nov/Dec 2023 [Nov_Dec_2023 \(1\) .pdf](#)
- May/Jun 2025 [May_Jun_2025 \(2\) .pdf](#)
- May/Jun 2024 [May_Jun_2024 \(1\) .pdf](#)
- May/Jun 2023 [May_Jun_2023 \(2\) .pdf](#)

Study:

- Permanent replicas
 - Server initiated replicas
 - Client initiated replicas
 - Scalability relation
-

3 Causal Consistency

Asked in:

- May/Jun 2025 [May_Jun_2025 \(2\) .pdf](#)
- May/Jun 2024 [May_Jun_2024 \(1\) .pdf](#)
- May/Jun 2023 [May_Jun_2023 \(2\) .pdf](#)

Study:

- Data-centric consistency
 - Causal consistency
 - Examples
-

4 Primary Based Protocol

Asked in:

- Nov/Dec 2023 [Nov_Dec_2023 \(1\) .pdf](#)
 - May/Jun 2025 [May_Jun_2025 \(2\) .pdf](#)
 - May/Jun 2023 [May_Jun_2023 \(2\) .pdf](#)
-

Prediction for Exam (Unit 4)

Most probable:

1. Checkpointing
2. Replica Management
3. Causal Consistency
4. Primary Based Protocol
5. Fault Tolerance

UNIT 5 (Q5, Q6)

MOST REPEATED TOPICS

VERY HIGH REPEATED

Topic	Repeated
DNS vs X.500 + Directory Service	4 Times
Apache Web Server	4 Times
Distributed File System Design Issues	2 Times
Cooperative Caching	2 Times

MEDIUM REPEATED

Topic	Repeated
Sun NFS Architecture	2 Times
BitTorrent	2 Times
QoS in Multimedia Systems	2 Times
Web Server Clusters	1 Time
Hadoop Distributed File System	1 Time

UNIT 5 IMPORTANT QUESTIONS PRIORITY

1 DNS vs X.500 (SUPER IMPORTANT)

Asked in:

- Nov/Dec 2024 `Nov_Dec_2024 (1).pdf`
- May/Jun 2025 `May_Jun_2025 (2).pdf`
- May/Jun 2024 `May_Jun_2024 (1).pdf`
- May/Jun 2023 `May_Jun_2023 (2).pdf`

Study:

- Directory service
 - DNS
 - X.500 architecture
 - Difference table
-

2 Apache Web Server

Asked in:

- Nov/Dec 2024 `Nov_Dec_2024 (1).pdf`
- May/Jun 2025 `May_Jun_2025 (2).pdf`
- May/Jun 2024 `May_Jun_2024 (1).pdf`
- May/Jun 2023 `May_Jun_2023 (2).pdf`

Study:

- Web services
 - Apache architecture
 - Components
 - Diagram
-

3 Cooperative Caching

Asked in:

- Nov/Dec 2024 [Nov_Dec_2024 \(1\).pdf](#)
 - Nov/Dec 2023 [Nov_Dec_2023 \(1\).pdf](#)
-

4 BitTorrent

Asked in:

- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)
 - May/Jun 2023 [May_Jun_2023 \(2\).pdf](#)
-

Prediction for Exam (Unit 5)

Most probable:

1. DNS vs X.500
2. Apache Web Server
3. Cooperative Caching

- 4. Distributed File Systems
- 5. BitTorrent

UNIT 6 (Q7, Q8)

MOST REPEATED TOPICS

VERY HIGH REPEATED

Topic	Repeated
Mach & CHORUS Microkernels	5 Times
Zabbix	4 Times
Wearable Devices + PVM + JINI	3 Times
SOA	3 Times
Nagios	3 Times
Prometheus	3 Times

UNIT 6 IMPORTANT QUESTIONS PRIORITY

Mach & CHORUS (MOST IMPORTANT OF UNIT 6)

Asked in ALL PAPERS:

- Nov/Dec 2024 Nov_Dec_2024 (1).pdf
- Nov/Dec 2023 Nov_Dec_2023 (1).pdf
- May/Jun 2025 May_Jun_2025 (2).pdf
- May/Jun 2024 May_Jun_2024 (1).pdf
- May/Jun 2023 May_Jun_2023 (2).pdf

SUPER IMPORTANT.

2 Zabbix

Asked in:

- Nov/Dec 2024 Nov_Dec_2024 (1).pdf
 - Nov/Dec 2023 Nov_Dec_2023 (1).pdf
 - May/Jun 2025 May_Jun_2025 (2).pdf
 - May/Jun 2023 May_Jun_2023 (2).pdf
-

3 SOA

Asked in:

- Nov/Dec 2023 Nov_Dec_2023 (1).pdf
 - May/Jun 2024 May_Jun_2024 (1).pdf
 - May/Jun 2023 May_Jun_2023 (2).pdf
-

4 Prometheus

Asked in:

- Nov/Dec 2023 `Nov_Dec_2023 (1).pdf`
 - May/Jun 2025 `May_Jun_2025 (2).pdf`
 - May/Jun 2024 `May_Jun_2024 (1).pdf`
-

FINAL HIGH PRIORITY LIST (MOST IMPORTANT OVERALL)

TOP 15 MOST REPEATED QUESTIONS

1. Mach & CHORUS
2. Checkpointing
3. DNS vs X.500
4. Apache Web Server
5. Replica Management
6. Zabbix
7. MPI
8. Overlay Network
9. Clock Synchronization
10. Causal Consistency
11. SOA
12. Prometheus
13. Primary Based Protocol
14. Cooperative Caching
15. BitTorrent

Smart Exam Strategy

If very less time:

Study these first:

- Checkpointing
- DNS vs X.500
- Apache Web Server
- Mach & CHORUS
- MPI
- Overlay Network
- Replica Management
- Zabbix

These alone can cover huge marks probability.

5/14/2026, 1:52:20 PM



i want exact question which is appeared in exam higher to lower reated till 1 time

5/14/2026, 1:53:35 PM



UNIT 3 (Q1, Q2)

Exact Questions Appeared in Exam

Higher Repeated → Lower Repeated



3 TIMES REPEATED

1 MPI (Message Passing Interface)

Exact Questions

- "What is the purpose of Message Passing Interface? Illustrate the architectural model for MPI using send and receive primitives."

Asked in:

- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)
- Nov/Dec 2024 [Nov_Dec_2024 \(1\).pdf](#)

- "What is MPI? Describe the point-to-point communication in MPI with suitable diagram."

Asked in:

- Nov/Dec 2023 [Nov_Dec_2023 \(1\).pdf](#)

2 Overlay Network

Exact Questions

- "Discuss the purpose of overlay network. Describe in brief, any three types of Overlay."
Asked in:
 - Nov/Dec 2024 [Nov_Dec_2024 \(1\).pdf](#)
 - "Describe the overlay network and its benefits? Explain it with the help of Skype overlay architecture."
Asked in:
 - May/Jun 2024 [May_Jun_2024 \(1\).pdf](#)
 - "What is Network virtualization? What are the types of overlay networks? What are the various advantages of overlay networks?"
Asked in:
 - May/Jun 2023 [May_Jun_2023 \(2\).pdf](#)
-

3 Clock Synchronization

Exact Questions

- "Why is clock synchronization using a global time impossible in a distributed system? Illustrate the active time server based centralized clock synchronization algorithm."
Asked in:
 - Nov/Dec 2024 [Nov_Dec_2024 \(1\).pdf](#)
- "Why is computer clock synchronization necessary? Describe the design requirements for a system to synchronize the clocks in a distributed system."
Asked in:
 - Nov/Dec 2023 [Nov_Dec_2023 \(1\).pdf](#)

- “Discuss the problem of clock synchronization in distributed operating systems. Illustrate the Berkeley’s clock synchronization algorithm with neat diagram and the drawback of Network Time Protocol (NTP).”

Asked in:

- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)
-

2 TIMES REPEATED

4 Bully Election Algorithm

Exact Questions

- “Explain the goal of an Election algorithm. Illustrate the bully algorithm using appropriate diagrams.”

Asked in:

- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)

- “Why do we need an election algorithm? Using appropriate diagrams describe the bully algorithm step-by-step.”

Asked in:

- May/Jun 2023 [May_Jun_2023 \(2\).pdf](#)
-

5 XDR / Marshalling / Unmarshalling

Exact Questions

- “Define External Data Representation, Marshalling and Unmarshalling. Discuss the three alternative approaches to XDR and marshalling.”

Asked in:

- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)
- “Explain external data representation (XDR), marshalling and unmarshalling. Why is XDR required? Discuss in brief the three alternative approaches to external data representation.”

Asked in:

- May/Jun 2024 [May_Jun_2024 \(1\).pdf](#)
-

6 Mutual Exclusion

Exact Questions

- “What are the two important properties of token-based approach? Explain token-ring algorithm to achieve mutual exclusion in a distributed system.”

Asked in:

- Nov/Dec 2024 [Nov_Dec_2024 \(1\).pdf](#)
- “What is the need of mutual exclusion? Explain the permission-based centralized distributed mutual exclusion algorithm.”

Asked in:

- May/Jun 2024 [May_Jun_2024 \(1\).pdf](#)
-

7 Logical Clock / Lamport Clock

Exact Questions

- “What are the advantages of logical clock over physical clock? ... Provide timestamps for each labeled event.”

Asked in:

- May/Jun 2024 [May_Jun_2024 \(1\).pdf](#)
- “Describe Lamport’s clock algorithm and give an example of how to order events in a distributed system. What are the algorithm’s limitations?”

Asked in:

- May/Jun 2023 [May_Jun_2023 \(2\).pdf](#)

1 TIME REPEATED

8 NTP

- “What is NTP? With the help of a diagram, describe how NTP works.”

Asked in:

- Nov/Dec 2023 [Nov_Dec_2023 \(1\).pdf](#)

UNIT 4 (Q3, Q4)

Exact Questions Appeared in Exam

4 TIMES REPEATED

Checkpointing

Exact Questions

- “What is checkpointing in a distributed system? Explain the working of Coordinated checkpointing recovery mechanism.”

Asked in:

- Nov/Dec 2024 [Nov_Dec_2024 \(1\).pdf](#)
 - May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)
 - May/Jun 2024 [May_Jun_2024 \(1\).pdf](#)
 - May/Jun 2023 [May_Jun_2023 \(2\).pdf](#)
-

2 Replica Management / Replication

Exact Questions

- “What are the key issues in Replica Management? Explain the following with respect to content replication and placement with suitable diagram:

i) Permanent Replicas

ii) Server-Initiated Replicas

iii) Client-Initiated Replicas”

Asked in:

- Nov/Dec 2023 [Nov_Dec_2023 \(1\).pdf](#)
- May/Jun 2023 [May_Jun_2023 \(2\).pdf](#)

- “Discuss the two important reasons for wanting to replicate data and how does replication relate to scalability.”

Asked in:

- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)

- “Why replication is important and how it relates with Scalability? How are replicas kept consistent?”

Asked in:

- May/Jun 2024 [May_Jun_2024 \(1\).pdf](#)
-

3 TIMES REPEATED

3 Causal Consistency

Exact Questions

- “Explain the causal consistency model with suitable example using distributed shared database.”

Asked in:

- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)
 - May/Jun 2024 [May_Jun_2024 \(1\).pdf](#)
 - May/Jun 2023 [May_Jun_2023 \(2\).pdf](#)
-

4 Primary Based Protocol

Exact Questions

- “What is a primary-based protocol in a consistency protocol? Explain the working of primary-backup protocol with suitable diagram.”

Asked in:

- Nov/Dec 2023 [Nov_Dec_2023 \(1\).pdf](#)
- “What is a primary-based protocol in a consistency protocol? Explain the working of replicated write protocols with active replication.”

Asked in:

- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)

- May/Jun 2023 [May_Jun_2023 \(2\).pdf](#)
-

UNIT 5 (Q5, Q6)

Exact Questions Appeared in Exam

4 TIMES REPEATED

1 DNS vs X.500

Exact Questions

- “What is a Directory Service? What is the difference between DNS and x500? Describe in detail the components of X.500 service architecture.”

Asked in:

- Nov/Dec 2024 [Nov_Dec_2024 \(1\).pdf](#)
- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)
- May/Jun 2023 [May_Jun_2023 \(2\).pdf](#)

- “What Is a Directory Service? What is the difference between DNS and x.500? Describe in detail the components of X.500 service architecture.”

Asked in:

- May/Jun 2024 [May_Jun_2024 \(1\).pdf](#)
-

2 Apache Web Server

Exact Questions

- “What are web services? Describe with a suitable diagram the general organization of the Apache web server.”

Asked in:

- Nov/Dec 2024 [Nov_Dec_2024 \(1\).pdf](#)
 - May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)
 - May/Jun 2024 [May_Jun_2024 \(1\).pdf](#)
 - May/Jun 2023 [May_Jun_2023 \(2\).pdf](#)
-

2 TIMES REPEATED

3 Cooperative Caching

Exact Questions

- “Explain in brief, the two places of client-side web caching? Explain cooperative caching with suitable diagram.”

Asked in:

- Nov/Dec 2024 [Nov_Dec_2024 \(1\).pdf](#)
 - Nov/Dec 2023 [Nov_Dec_2023 \(1\).pdf](#)
-

4 BitTorrent

Exact Questions

- “Describe the design of a peer-to-peer file sharing application designed to support very large multimedia files. How does the BitTorrent protocol operate?”

Asked in:

- May/Jun 2025 [May_Jun_2025 \(2\) .pdf](#)
- “Describe the design of a peer-to-peer download system designed to support very large multimedia files. How does the BitTorrent protocol operate?”

Asked in:

- May/Jun 2023 [May_Jun_2023 \(2\) .pdf](#)

UNIT 6 (Q7, Q8)

Exact Questions Appeared in Exam

 **5 TIMES REPEATED**

1 Mach & CHORUS

Exact Questions

- “Provide an overview of Mach and CHORUS microkernels. How are memory management techniques used to avoid physical copying of data in Mach and CHORUS?”

Asked in:

- Nov/Dec 2024 [Nov_Dec_2024 \(1\) .pdf](#)
- Nov/Dec 2023 [Nov_Dec_2023 \(1\) .pdf](#)
- “Explain in brief following Microkernels:
i) CHORUS

ii) Mach”

Asked in:

- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)
- “Explain in brief following Microkernels
i) Mach
ii) CHORUS”

Asked in:

- May/Jun 2024 [May_Jun_2024 \(1\).pdf](#)
- May/Jun 2023 [May_Jun_2023 \(2\).pdf](#)

4 TIMES REPEATED

Zabbix

Exact Questions

- “Explain in brief, the key features of Zabbix including auto-discovery, triggers, or dashboards.”

Asked in:

- Nov/Dec 2024 [Nov_Dec_2024 \(1\).pdf](#)

- “Explain in brief, the key features of Zabbix.”

Asked in:

- Nov/Dec 2023 [Nov_Dec_2023 \(1\).pdf](#)

- “Explain in brief the following Distributed System monitoring tools:

i) Zabbix

ii) Nagios”

Asked in:

- May/Jun 2023 [May_Jun_2023 \(2\).pdf](#)

- “Explain in brief, the following Distributed System monitoring tools:

i) Nagios

ii) Zabbix”

Asked in:

- May/Jun 2025 [May_Jun_2025 \(2\).pdf](#)

5/14/2026, 1:53:37 PM



UNIT 3 (Q1, Q2)

"What is the purpose of Message Passing Interface? Illustrate the architectural model for MPI using send and receive primitives."

Asked in:

May/Jun 2025

Nov/Dec 2024

[9 Marks]

5/14/2026, 6:46:19 PM



Q. What is the purpose of Message Passing Interface? Illustrate the architectural model for MPI using send and receive primitives.

(9 Marks - SPPU BE IT DS)



1 Simple Introduction

In a Distributed System, many computers work together to solve one big problem. These computers need a way to communicate with each other.

MPI (Message Passing Interface) is used for communication between different processes running on multiple computers.

It helps systems exchange messages quickly and efficiently.

2 Definition

MPI (Message Passing Interface)

MPI is a **standard communication library** used in distributed systems and parallel computing for exchanging messages between processes using **send** and **receive** operations.

3 Purpose of MPI

The main purpose of MPI is to allow multiple processes/computers to communicate and work together.

MPI is used for:

- Process communication
 - Data sharing
 - Parallel processing
 - High-speed computation
 - Distributed computing
-

4 Why MPI is Needed

Without MPI:

- Processes cannot coordinate properly.

- Data sharing becomes difficult.
- Distributed tasks become slow.

MPI solves these problems by:

- providing communication rules,
- message transfer,
- synchronization,
- and coordination between processes.

5 Real-Life Analogy

WhatsApp Group Example

Imagine:

- many students are working on one project.
- One student sends files/messages to another student using WhatsApp.

Similarly:

- in MPI,
- one process sends data,
- another process receives data.

Relation with MPI

Real Life

MPI

Student

Process

WhatsApp Message

MPI Message



Real Life

MPI

Send Button

MPI_Send

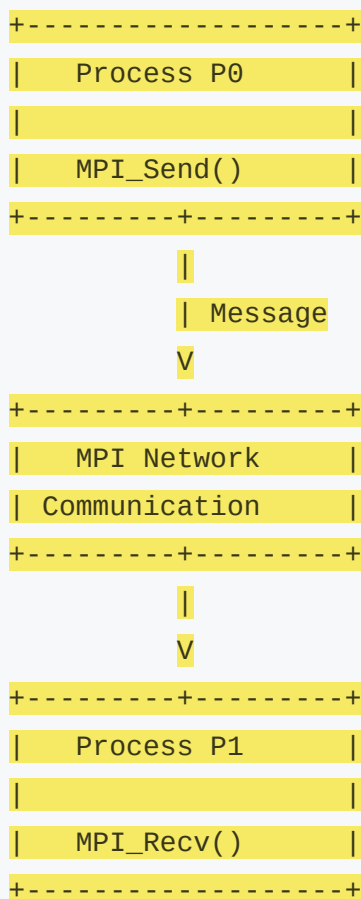
Receive Message

MPI_Recv

6

Architectural Model of MPI

Simple MPI Architecture Diagram



7

Explanation of Diagram

Process P0

- Sender process
- Uses `MPI_Send()`
- Sends data/message

MPI Communication Network

- Transfers message between processes

Process P1

- Receiver process
- Uses `MPI_Recv()`
- Receives message

8 Send and Receive Primitives in MPI

MPI mainly uses two primitives:

Primitive	Purpose
<code>MPI_Send()</code>	Sends message
<code>MPI_Recv()</code>	Receives message

9 Syntax of MPI_Send()


```
MPI_Send(data, count, datatype, destination, tag, communicator)
```

Meaning

Parameter	Meaning
data	Data to send
count	Number of data items
datatype	Type of data
destination	Receiver process
tag	Message identifier
communicator	Group of processes

10 Syntax of MPI_Recv()

```
MPI_Recv(data, count, datatype, source, tag, communicator, status)
```

1 1 Working of MPI (Step-by-Step)

Step 1

Multiple processes are created.

Step 2

Sender process prepares data.

Step 3

Sender uses `MPI_Send()`.

Step 4

Message travels through communication network.

Step 5

Receiver process waits using `MPI_Recv()`.

Step 6

Receiver gets data and continues execution.

1 2 Features of MPI

- Fast communication
- Portable
- Scalable
- Supports parallel processing
- Standard communication model
- Efficient message passing

13 Advantages of MPI

- Improves performance
- Supports distributed systems
- Enables parallel computing
- Efficient resource utilization
- Reliable communication

14 Disadvantages of MPI

- Programming is complex
- Debugging is difficult
- Requires synchronization
- Large systems become difficult to manage

15 Real-World Applications

MPI is used in:

- Scientific computing
- Weather forecasting
- AI and Machine Learning
- Space research
- Supercomputers

16 Industry Example

Google Data Centers

Google uses distributed computing where many servers communicate together for:

- searching data,
- indexing webpages,
- large computations.

MPI-like communication helps processes exchange data efficiently.

17 Important Exam Keywords

Write these keywords in exam for better marks:

- Message Passing
 - Parallel Computing
 - Distributed Communication
 - MPI_Send
 - MPI_Recv
 - Synchronization
 - Interprocess Communication
 - Scalability
-

18 Memory Trick

“SRM”

- **S** → Send
- **R** → Receive
- **M** → Message

Remember:

MPI = Send + Receive + Message Passing

19 Conclusion

MPI is an important communication standard in distributed systems.

It allows processes to exchange messages using send and receive operations efficiently.

MPI improves scalability, coordination, and performance in distributed and parallel computing systems.

5/14/2026, 6:46:21 PM



[9 Marks]

"What is MPI? Describe the point-to-point communication in MPI with suitable diagram."

5/14/2026, 6:47:35 PM



Q. What is MPI? Describe the point-to-point communication in MPI with suitable diagram.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, many computers or processes work together to complete one large task.

To work properly, they must exchange data and messages with each other.

MPI helps processes communicate efficiently using message passing.

2 Definition of MPI

MPI (Message Passing Interface)

MPI is a **standard communication library** used in distributed systems and parallel computing to exchange messages between multiple processes.

It allows processes running on different computers to communicate using send and receive operations.

3 Why MPI is Needed

MPI is needed because:

- Distributed processes must share data
- Processes need coordination
- Parallel tasks require communication
- Systems must work together efficiently

Without MPI:

- communication becomes difficult,
 - synchronization problems occur,
 - and distributed execution becomes slow.
-

4 What is Point-to-Point Communication?

Point-to-point communication means:

One process sends a message directly to another process.

Only:

- one sender,
- and one receiver are involved.

MPI mainly uses:

- `MPI_Send()`
- `MPI_Recv()`

for point-to-point communication.

5 Real-Life Analogy

Mobile Phone Call Example

Imagine:

- one person calls another person directly.

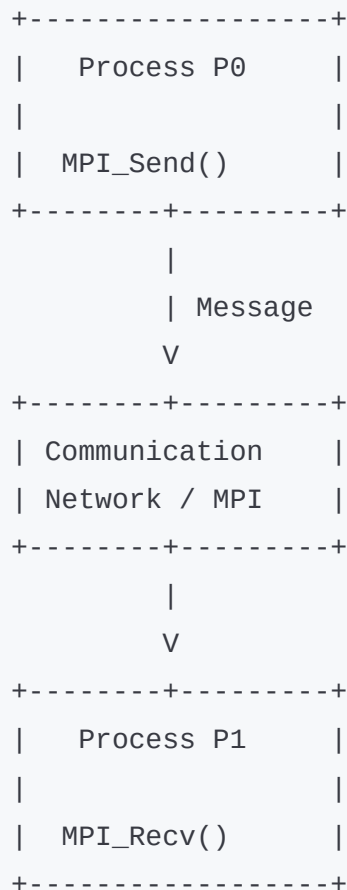
There is:

- one sender,
- one receiver,
- and direct communication.

Relation with MPI

Real Life	MPI
Caller	Sender Process
Receiver	Receiving Process
Phone Call	Message
Call Transfer	MPI Communication

6 Suitable Diagram of Point-to-Point Communication



7 Explanation of Diagram

Process P0

- Sender process
- Sends data using `MPI_Send()`

Communication Network

- Transfers message between processes

Process P1

- Receiver process
- Receives data using `MPI_Recv()`

8 MPI_Send() Function

Syntax

```
MPI_Send(buffer, count, datatype, destination, tag, communicator)
```

Purpose

Used to send data from one process to another.

9 MPI_Recv() Function

Syntax

```
MPI_Recv(buffer, count, datatype, source, tag, communicator, status)
```

Purpose

Used to receive data from another process.

10 Working of Point-to-Point Communication

Step 1

Processes are created in distributed system.

Step 2

Sender process prepares message/data.

Step 3

Sender uses `MPI_Send()`.

Step 4

Message passes through MPI network.

Step 5

Receiver waits using `MPI_Recv()`.

Step 6

Receiver gets message and processes it.

11 Types of Point-to-Point Communication

Type	Meaning
Blocking Communication	Sender/receiver waits until operation completes
Non-Blocking Communication	Process continues execution without waiting

12 Features of MPI

- High-speed communication
 - Portable
 - Scalable
 - Supports distributed systems
 - Efficient process coordination
-

13 Advantages of MPI

- Fast communication
- Better performance
- Supports parallel computing
- Efficient resource usage

- Reliable data transfer
-

14 Disadvantages of MPI

- Programming complexity
 - Difficult debugging
 - Synchronization overhead
 - Large systems are harder to manage
-

15 Real-World Applications

MPI is used in:

- Supercomputers
 - Weather forecasting
 - Scientific simulations
 - AI processing
 - Space research
-

16 Industry Example

Amazon Cloud Computing

Amazon cloud servers work together using distributed communication.

Large tasks are divided among many servers, and processes exchange messages similar to MPI communication.

17 Important Exam Keywords

Write these keywords in exam:

- Message Passing
 - MPI_Send
 - MPI_Recv
 - Point-to-Point Communication
 - Distributed Computing
 - Parallel Processing
 - Interprocess Communication
 - Synchronization
-

18 Memory Trick

“SR”

- **S** → Send
- **R** → Receive

Remember:

Point-to-Point = One Send + One Receive

19 Conclusion

MPI is a standard communication mechanism used in distributed systems for message passing between processes.

Point-to-point communication allows direct data exchange between sender and receiver processes using send and receive primitives efficiently.

5/14/2026, 6:47:36 PM



2 Overlay Network — [8M / 9M]

[9 Marks]

“Discuss the purpose of overlay network. Describe in brief, any three types of Overlay.”

Asked in:

Nov/Dec 2024

[9 Marks]

“Describe the overlay network and its benefits? Explain it with the help of Skype overlay architecture.”

Asked in:

May/Jun 2024

5/14/2026, 8:20:27 PM



Q. Discuss the purpose of Overlay Network. Describe in brief any three types of Overlay.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, computers are connected through networks to communicate and share data.

Sometimes, a special virtual network is created on top of the existing Internet network to improve communication and services.

This virtual network is called an **Overlay Network**.

2 Definition

Overlay Network

An Overlay Network is a **virtual network built on top of another physical network** where nodes are connected using logical links instead of direct physical connections.

3 Purpose of Overlay Network

Overlay networks are used to improve:

- communication,
 - scalability,
 - routing,
 - file sharing,
 - and distributed services.
-

4 Why Overlay Network is Needed

Normal Internet networks may have:

- slow routing,
- poor scalability,

- difficult resource sharing.

Overlay networks solve these problems by:

- creating logical connections,
- improving data transfer,
- enabling distributed applications.

5 Real-Life Analogy

Google Maps Shortcut Example

Imagine:

- roads already exist physically,
- but Google Maps creates a smart virtual route for faster travel.

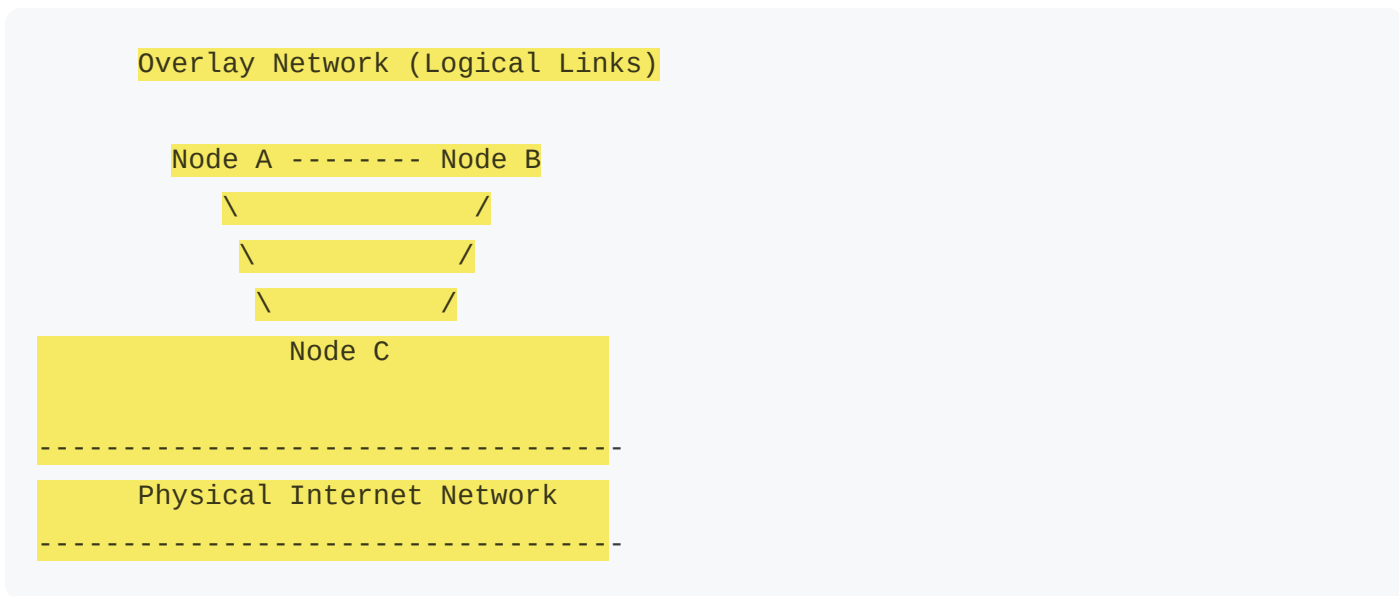
Similarly:

- Internet is the physical network,
- Overlay network creates smart logical paths over it.

Relation with Overlay Network

Real Life	Overlay Network
Roads	Physical Network
Google Maps Route	Overlay Network
Smart Path	Logical Connection

6 Basic Overlay Network Diagram



7 Explanation of Diagram

Physical Network

Actual Internet cables and routers.

Overlay Network

Virtual logical connections created between nodes.

Nodes

Computers participating in distributed communication.

8 Types of Overlay Networks

i) Peer-to-Peer (P2P) Overlay

In this:

- all nodes act equally,
- each node can send and receive data.

Example

BitTorrent

Features

- No central server
 - Direct sharing
 - Scalable
-

ii) Structured Overlay

Uses predefined structure for routing and searching data.

Example

Distributed Hash Table (DHT)

Features

- Organized network
- Fast searching
- Efficient routing

iii) Unstructured Overlay

Nodes connect randomly without fixed structure.

Example

Gnutella

Features

- Easy to build
- Flexible
- Search may be slower

9 Advantages of Overlay Network

- Better scalability
- Efficient routing
- Improved communication
- Supports distributed applications
- Easy resource sharing

10 Disadvantages of Overlay Network

- Additional overhead
- Complex management

- Security issues
 - Extra routing cost
-

1 1 Applications of Overlay Network

Used in:

- Video streaming
 - Online gaming
 - Peer-to-peer systems
 - Cloud computing
 - VoIP applications
-

1 2 Real-World Industry Example

Skype

Skype uses overlay networks for:

- voice calls,
- video calls,
- peer-to-peer communication.

It creates logical communication paths between users over the Internet.

13 Important Exam Keywords

Write these keywords in exam:

- Logical Links
 - Virtual Network
 - Physical Network
 - Peer-to-Peer
 - Structured Overlay
 - Unstructured Overlay
 - Scalability
 - Routing
-

14 Memory Trick

"PSU"

- **P** → Peer-to-Peer
- **S** → Structured
- **U** → Unstructured

Remember:

Types of Overlay = PSU

15 Conclusion

Overlay Network is a virtual network built on top of an existing network for better communication and distributed services.

It improves scalability, routing, and resource sharing in distributed systems.

Q. Describe the Overlay Network and its Benefits. Explain it with the help of Skype Overlay Architecture.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

Applications like Skype need fast communication between millions of users worldwide.

To achieve this, Skype uses an Overlay Network where computers communicate through logical connections over the Internet.

2 Definition

Overlay Network

An Overlay Network is a virtual communication network built on top of an existing physical network using logical connections.

3 Benefits of Overlay Network

Overlay networks provide:

- Better scalability
- Faster communication
- Efficient routing
- Resource sharing
- Fault tolerance
- Flexible communication

4 Why Skype Uses Overlay Network

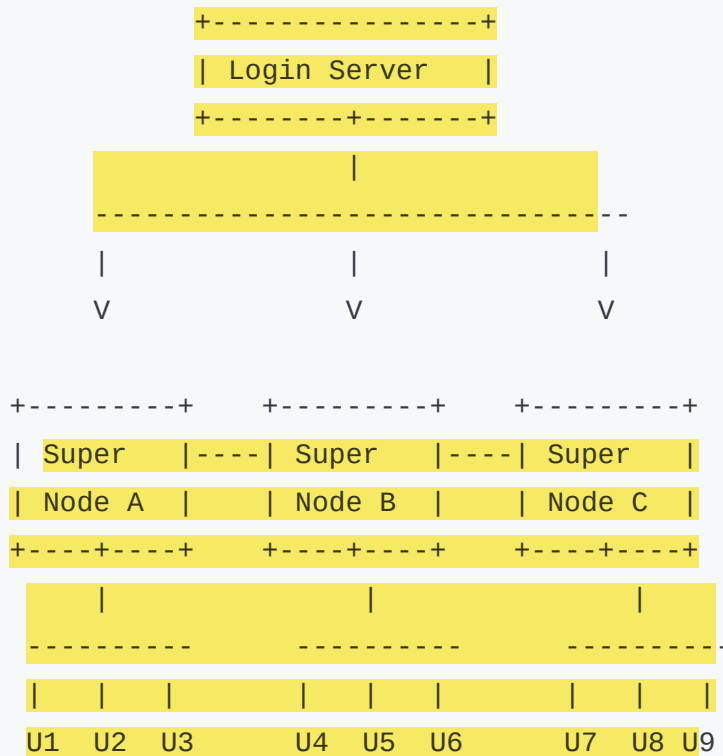
Skype requires:

- fast voice/video calls,
- peer-to-peer communication,
- reliable connectivity.

Overlay networks help Skype:

- connect users directly,
- reduce server load,
- improve communication quality.

5 Skype Overlay Architecture Diagram



(Ordinary Nodes / Skype Users)

6 Explanation of Skype Architecture

Login Server

- Authenticates users during login.

Super Nodes

- Powerful computers with high bandwidth.
- Help route communication.

Ordinary Nodes

- Normal Skype users.

Overlay Links

- Logical communication paths between nodes.
-

Working of Skype Overlay Network

Step 1

User logs into Skype server.

Step 2

Skype connects user to nearest Super Node.

Step 3

Super Nodes help locate other users.

Step 4

Voice/data packets are transferred through overlay paths.

Step 5

Users communicate directly.

8 Features of Skype Overlay Architecture

- Peer-to-peer communication
 - Distributed routing
 - Scalable network
 - Efficient communication
 - Reduced server dependency
-

9 Advantages of Skype Overlay Network

- Supports millions of users
 - Fast communication
 - Reduced central server load
 - Better scalability
 - Fault tolerance
-

10 Disadvantages

- Security concerns

- Complex management
 - Super node dependency
 - Network overhead
-

1 1 Real-Life Analogy

Railway Junction Example

Super Nodes are like railway junctions:

- they help passengers (data) reach destination efficiently.

Ordinary users connect through these junctions.

Relation to Skype

Railway System	Skype
Junction	Super Node
Passenger	Data Packet
Train Route	Overlay Path

1 2 Important Exam Keywords

- Overlay Network
- Logical Links
- Super Nodes
- Peer-to-Peer

- Distributed Communication
 - Routing
 - Scalability
 - Skype Architecture
-

13 Memory Trick

"SLS"

- **S** → Super Nodes
- **L** → Logical Links
- **S** → Skype

Remember:

Skype Overlay works using Super Nodes and Logical Links.

14 Conclusion

Overlay networks help distributed applications communicate efficiently over existing Internet infrastructure.

Skype uses overlay architecture with super nodes and logical links to provide scalable and reliable communication services.

5/14/2026, 8:20:30 PM



[9 Marks]

"What is Network virtualization? What are the types of overlay networks? What are the various advantages of overlay networks?"

5/14/2026, 8:21:30 PM



Q. What is Network Virtualization? What are the types of Overlay Networks? What are the various advantages of Overlay Networks?

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, many computers communicate through networks.

Sometimes, instead of using only physical hardware connections, a virtual network is created to improve communication, flexibility, and scalability.

This concept is called **Network Virtualization**.

Overlay networks are virtual networks built on top of existing physical networks.

2 Definition of Network Virtualization

Network Virtualization

Network Virtualization is a technique in which physical network resources are converted into multiple virtual networks using software.

It allows different virtual networks to run on the same physical infrastructure.

3 Definition of Overlay Network

Overlay Network

An Overlay Network is a virtual network built on top of another network using logical connections between nodes.

4 Why Network Virtualization is Needed

It is needed because:

- physical networks are costly,
- difficult to manage,
- less flexible,
- and hard to scale.

Network virtualization helps in:

- better resource utilization,
- flexible communication,
- easier management,
- and scalability.

5 Real-Life Analogy

Apartment Building Example

Imagine:

- one large building is divided into many apartments.

Each apartment works independently even though the building is same.

Similarly:

- one physical network can support many virtual networks.

Relation with Network Virtualization

Real Life	Network Virtualization
Building	Physical Network
Apartments	Virtual Networks
Separate Families	Different Users/Organizations

6 Basic Diagram of Network Virtualization



7 Explanation of Diagram

Physical Network

Actual routers, switches, cables, and hardware.

Virtual Networks

Software-created logical networks running independently.

8

Types of Overlay Networks

i) Peer-to-Peer (P2P) Overlay Network

All nodes work equally and directly share resources.

Example

BitTorrent

Features

- No central server
 - Direct communication
 - Distributed sharing
-

ii) Structured Overlay Network

Uses organized structure and routing methods.

Example

Distributed Hash Table (DHT)

Features

- Fast searching
 - Efficient routing
 - Structured organization
-

iii) Unstructured Overlay Network

Nodes are connected randomly.

Example

Gnutella

Features

- Easy to create
 - Flexible
 - Slower searching
-

9 Advantages of Overlay Networks

1. Scalability

Supports large number of users and nodes.

2. Better Routing

Provides efficient data transfer paths.

3. Resource Sharing

Easy sharing of files and services.

4. Flexibility

Virtual networks can be modified easily.

5. Fault Tolerance

Network continues working even if some nodes fail.

6. Reduced Cost

Uses existing physical infrastructure.

7. Improved Performance

Optimizes communication between nodes.

10 Disadvantages of Overlay Networks

- Additional overhead
- Security concerns
- Complex management

- Extra routing layers
-

11 Applications of Overlay Networks

Used in:

- Peer-to-peer systems
 - Cloud computing
 - Video streaming
 - Online gaming
 - VoIP systems
 - Distributed systems
-

12 Real-World Industry Example

Netflix

Netflix uses distributed overlay-like networks and CDNs to deliver videos quickly to users worldwide.

Logical routing helps reduce delay and improve streaming performance.

13 Difference Between Physical and Overlay Network

Physical Network

Uses physical hardware

Fixed infrastructure

Difficult to modify

Hardware dependent

Overlay Network

Uses logical links

Flexible structure

Easy to modify

Software controlled

14 Important Exam Keywords

Write these keywords in exam:

- Network Virtualization
- Overlay Network
- Logical Links
- Peer-to-Peer
- Structured Overlay
- Unstructured Overlay
- Scalability
- Fault Tolerance

15 Memory Trick

“PSU”

- **P** → Peer-to-Peer
- **S** → Structured
- **U** → Unstructured

Remember:

Types of Overlay Networks = PSU

16 Conclusion

Network Virtualization allows multiple virtual networks to run on the same physical infrastructure efficiently.

Overlay networks improve scalability, routing, flexibility, and resource sharing in distributed systems.

5/14/2026, 8:21:32 PM



3 Clock Synchronization — [8M / 9M]
[9 Marks]

“Why is clock synchronization using a global time impossible in a distributed system? Illustrate the active time server based centralized clock synchronization algorithm.”

Asked in:

Nov/Dec 2024
[8 Marks]

“Why is computer clock synchronization necessary? Describe the design requirements for a system to synchronize the clocks in a distributed system.”

5/14/2026, 8:22:08 PM



Q. Why is clock synchronization using a global time impossible in a distributed system? Illustrate the active time server based centralized clock synchronization algorithm.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, many computers work together from different locations.

Each computer has its own clock, but all clocks may show slightly different times.

Clock synchronization is used to maintain approximately same time on all systems.

2 Definition

Clock Synchronization

Clock synchronization is the process of matching the time of all clocks in a distributed system as closely as possible.

3 Why Global Time Synchronization is Impossible

Perfect global time synchronization is impossible because:

1. Network Delay

Messages take different amounts of time to travel through the network.

Delay is not constant.

2. Clock Drift

Every computer clock runs at slightly different speed.

Some clocks become fast, some become slow.

3. No Single Universal Clock

Distributed systems do not share one common physical clock.

4. Unpredictable Communication

Network traffic and congestion affect message delivery time.

5. Hardware Differences

Different computers use different hardware clocks with varying accuracy.

Real-Life Analogy

Classroom Clock Example

Imagine:

- every classroom has its own wall clock.
- Some clocks are fast, some slow.

Even if teacher tries to set all clocks together:

- after some time, clocks again show different times.

Relation with Distributed System

Real Life	Distributed System
Classroom clocks	Computer clocks
Teacher adjusting clocks	Synchronization algorithm
Different timings	Clock drift

5 Need of Clock Synchronization

Clock synchronization is necessary for:

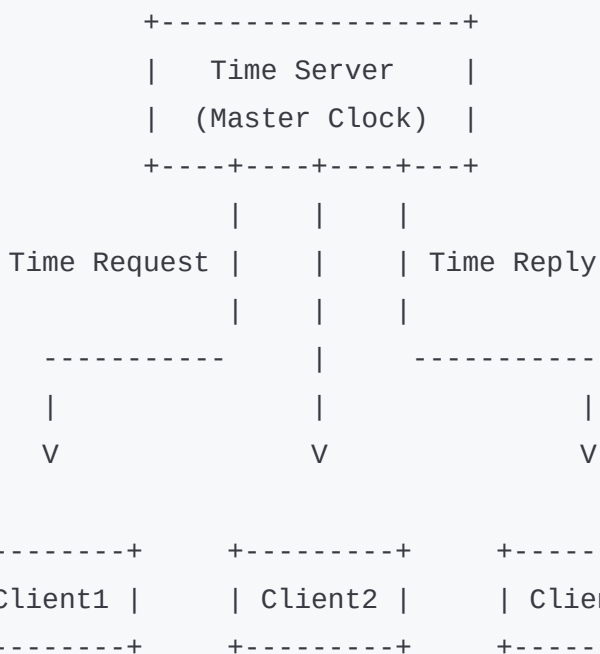
- Event ordering
- Distributed coordination
- Logging
- File updates
- Database consistency
- Transaction management

6 Active Time Server Based Centralized Clock Synchronization Algorithm

In this method:

- one machine acts as a **Time Server**,
- all other systems synchronize their clocks with it.

7 Diagram of Centralized Clock Synchronization



8 Explanation of Diagram

Time Server

- Maintains standard time
- Sends synchronized time to clients

Clients

- Request current time from server
 - Adjust their clocks accordingly
-

Working of Centralized Clock Synchronization

Step 1

One computer becomes Time Server.

Step 2

Clients send time request to server.

Step 3

Server replies with current time.

Step 4

Client calculates network delay.

Step 5

Client updates its local clock.

10 Advantages

- Simple implementation
 - Easy management
 - Better synchronization accuracy
 - Centralized control
-

1 1 Disadvantages

- Single point of failure
 - Server overload possible
 - Network delay affects accuracy
 - Scalability issue
-

1 2 Applications

Used in:

- Banking systems

- Distributed databases
 - Cloud computing
 - Airline reservation systems
 - Distributed file systems
-

13 Industry Example

Google Distributed Data Centers

Google servers across the world require synchronized clocks for:

- database updates,
- logging,
- distributed transactions.

Clock synchronization helps maintain proper event order.

14 Important Exam Keywords

Write these keywords in exam:

- Clock Drift
- Network Delay
- Global Time
- Time Server
- Synchronization
- Distributed Coordination

- Master Clock
 - Event Ordering
-

15 Memory Trick

“DDN”

- **D** → Delay
- **D** → Drift
- **N** → No universal clock

Remember:

Global synchronization impossible because of DDN.

16 Conclusion

Perfect global clock synchronization is impossible in distributed systems because of network delays and clock drift.

The centralized clock synchronization algorithm uses a time server to synchronize all client clocks approximately.

Q. Why is computer clock synchronization necessary? Describe the design requirements for a system to synchronize the clocks in a distributed system.

(8 Marks - SPPU BE IT DS)

1 Simple Introduction

In distributed systems, different computers maintain different local clocks.

Because clocks may show different times, synchronization is required for proper coordination and communication.

2 Definition

Clock Synchronization

Clock synchronization is the process of keeping the clocks of all distributed computers approximately equal.

3 Why Clock Synchronization is Necessary

Clock synchronization is needed because:

1. Event Ordering

To identify which event happened first.

2. Coordination Between Systems

Processes must work in proper sequence.

3. Consistency Maintenance

Helps maintain correct data updates.

4. Distributed Transactions

Required in banking and databases.

5. Logging and Monitoring

Correct timestamps are necessary.

4 Real-Life Analogy

Railway Station Example

Imagine:

- all railway stations must use same time.
- Otherwise trains may collide or become delayed.

Similarly:

- distributed systems require synchronized clocks for proper coordination.
-

5 Design Requirements for Clock Synchronization

1. Accuracy

Clocks should remain very close to actual time.

2. Reliability

System should work even if some nodes fail.

3. Fault Tolerance

Synchronization should continue during failures.

4. Scalability

Should support large distributed systems.

5. Low Network Overhead

Should minimize extra communication.

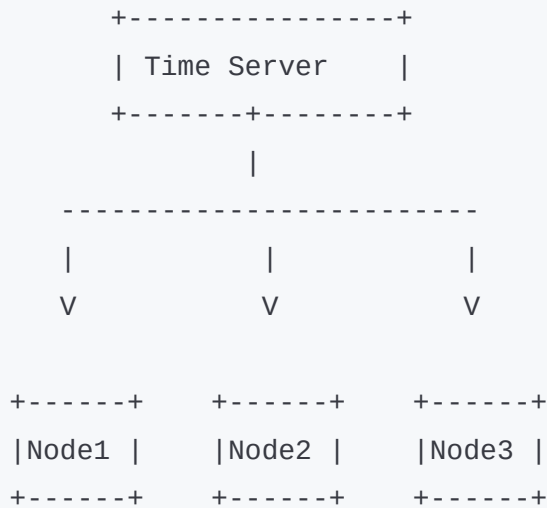
6. Synchronization Frequency

Clocks should synchronize regularly.

7. Handling Clock Drift

Must correct fast and slow clocks.

Diagram



7 Advantages of Clock Synchronization

- Proper coordination
- Correct event ordering
- Better consistency
- Reliable distributed processing

8 Limitations

- Network delay affects accuracy
- Complex implementation
- Additional communication overhead

9 Applications

- Banking systems
 - Cloud computing
 - Distributed databases
 - Online transactions
 - Airline reservation systems
-

10 Important Exam Keywords

- Clock Drift
 - Synchronization
 - Accuracy
 - Scalability
 - Fault Tolerance
 - Coordination
 - Event Ordering
-

11 Memory Trick

“ARFSL”

- **A** → Accuracy
- **R** → Reliability
- **F** → Fault tolerance

- **S** → Scalability
 - **L** → Low overhead
-

1 2 Conclusion

Clock synchronization is essential in distributed systems for proper coordination and event ordering.

A synchronization system must provide accuracy, reliability, scalability, and fault tolerance for efficient distributed operation.

5/14/2026, 8:22:10 PM



[8 Marks]

"Discuss the problem of clock synchronization in distributed operating systems. Illustrate the Berkeley's clock synchronization algorithm with neat diagram and the drawback of Network Time Protocol (NTP)."

5/14/2026, 8:22:34 PM



Q. Discuss the problem of clock synchronization in distributed operating systems. Illustrate the Berkeley's clock synchronization algorithm with neat diagram and the drawback of Network Time Protocol (NTP).

(8 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Operating Systems, many computers work together through a network.

Each computer has its own clock, but these clocks may show different times.

This creates synchronization problems in distributed systems.

Clock synchronization algorithms like Berkeley Algorithm help maintain approximately same time on all systems.

2 Definition

Clock Synchronization

Clock synchronization is the process of keeping clocks of all computers in a distributed system approximately equal.

3 Problems of Clock Synchronization in Distributed Systems

1. Clock Drift

Every computer clock runs at slightly different speed.

Some clocks become:

- fast,
 - slow,
 - or inaccurate over time.
-

2. Network Delay

Messages take different time to travel through network.

Delay is unpredictable.

3. No Global Physical Clock

There is no single universal clock shared by all computers.

4. Different Hardware Clocks

Different systems use different clock hardware with different accuracy.

5. Event Ordering Problem

Unsynchronized clocks create confusion in event sequence.

Real-Life Analogy

School Wall Clock Example

Imagine:

- every classroom has different wall clock timing.
- Some clocks are ahead, some behind.

Students become confused about lecture timings.

Similarly:

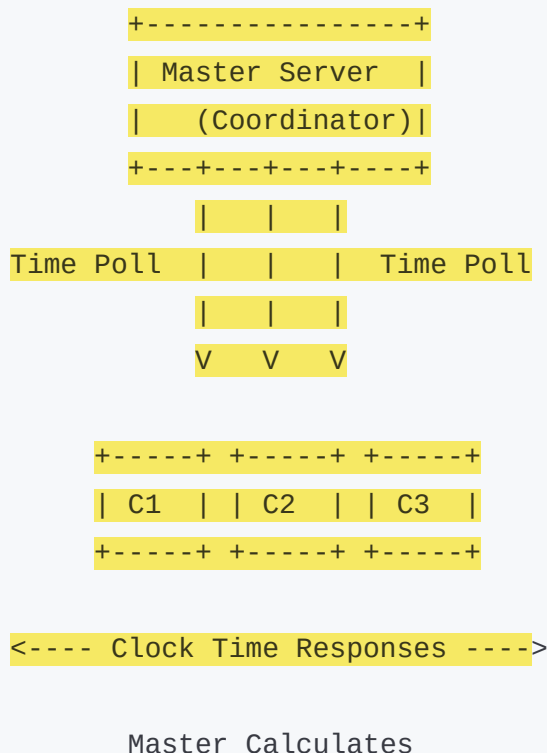
- distributed computers require synchronized clocks for proper coordination.

5 Berkeley Clock Synchronization Algorithm

Definition

Berkeley Algorithm is a centralized clock synchronization algorithm where one system acts as a master and synchronizes all other clocks.

6 Berkeley Algorithm Diagram



Average Time

<---- Time Adjustment ---->

7 Explanation of Diagram

Master Server

- Coordinator node
- Collects time from all systems

Client Systems (C1, C2, C3)

- Send local clock times to master

Average Time

Master calculates average system time.

Time Adjustment

Master sends correction values to clients.

8 Working of Berkeley Algorithm (Step-by-Step)

Step 1

One machine becomes master server.

Step 2

Master asks all clients for current time.

Step 3

Clients send their local clock times.

Step 4

Master calculates average time.

(Abnormal clocks are ignored.)

Step 5

Master sends adjustment values to all clients.

Step 6

Clients adjust their clocks.

9 Advantages of Berkeley Algorithm

- Good synchronization accuracy
 - No external UTC clock required
 - Simple centralized approach
 - Reduces clock drift
-

10 Disadvantages of Berkeley Algorithm

- Single point of failure
 - Master overload possible
 - Network delay affects accuracy
 - Poor scalability
-

11 Drawbacks of Network Time Protocol (NTP)

NTP (Network Time Protocol)

NTP is used to synchronize clocks over Internet.

Drawbacks of NTP

1. Network Delay Problem

Variable network delay affects synchronization accuracy.

2. Complex Configuration

NTP setup and management can be difficult.

3. Security Issues

NTP servers may face attacks or fake time updates.

4. Internet Dependency

Requires stable Internet connection.

5. Less Accurate in Heavy Traffic

Accuracy decreases during network congestion.

12 Applications of Clock Synchronization

Used in:

- Banking systems
 - Cloud computing
 - Distributed databases
 - Airline reservation systems
 - Online transaction systems
-

13 Industry Example

Amazon Cloud Servers

Amazon cloud data centers use synchronized clocks for:

- transaction processing,
 - distributed logging,
 - database consistency.
-

14 Important Exam Keywords

Write these keywords in exam:

- Clock Drift
- Network Delay

- Berkeley Algorithm
 - Master Server
 - Synchronization
 - Average Time
 - NTP
 - Distributed Coordination
-

15 Memory Trick

“DNA”

Problems of synchronization:

- **D** → Drift
 - **N** → Network Delay
 - **A** → Absence of Global Clock
-

16 Conclusion

Clock synchronization is essential in distributed operating systems for proper coordination and event ordering.

The Berkeley Algorithm synchronizes clocks using a centralized master server, while NTP provides Internet-based synchronization but suffers from delay and security limitations.

5/14/2026, 8:22:35 PM



4 Bully Algorithm — [9M]
[9 Marks]

"Explain the goal of an Election algorithm. Illustrate the bully algorithm using appropriate diagrams."

Asked in:

May/Jun 2025
[9 Marks]

"Why do we need an election algorithm? Using appropriate diagrams describe the bully algorithm step-by-step."

5/14/2026, 8:26:38 PM



Q. Explain the goal of an Election Algorithm. Illustrate the Bully Algorithm using appropriate diagrams.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, many computers work together.

Sometimes, one computer acts as a **Coordinator (Leader)** to manage tasks.

If the coordinator fails, the system must choose a new coordinator automatically.

This process is called an **Election Algorithm**.

2 Definition

Election Algorithm

An Election Algorithm is a method used in distributed systems to select a new coordinator process when the current coordinator fails.

3 Goal of Election Algorithm

The main goal is:

- to select one coordinator,
 - maintain system control,
 - avoid conflicts,
 - and continue distributed operations smoothly.
-

4 Why Election Algorithm is Needed

Election algorithm is needed because:

- coordinator may crash,
- system must continue working,
- processes need centralized coordination,
- fault tolerance is required.

5 Real-Life Analogy

Class Monitor Election Example

Imagine:

- class monitor is absent,
- students select a new monitor.

Usually:

- strongest or most suitable student becomes monitor.

Similarly:

- in Bully Algorithm,
- process with highest ID becomes coordinator.

6 Bully Algorithm

Definition

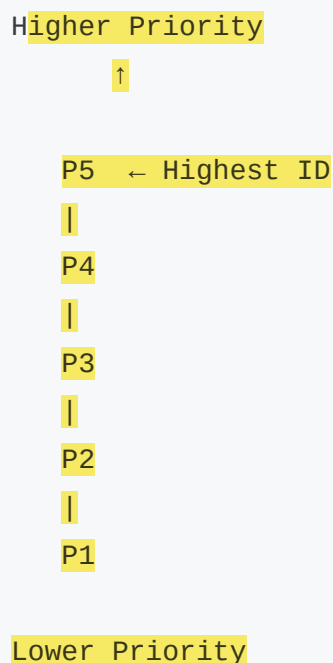
Bully Algorithm is an election algorithm in which the process having the **highest process ID** becomes the new coordinator.

It is called "Bully" because the higher-ID process dominates lower-ID processes.

7 Assumptions of Bully Algorithm

- Every process has unique ID.
- Higher ID = higher priority.
- Each process knows IDs of other processes.
- Processes can communicate with each other.

8 Diagram of Bully Algorithm



9 Explanation of Diagram

- P5 has highest ID.
- If coordinator fails,

- highest active process becomes new coordinator.
-

10 Working of Bully Algorithm (Step-by-Step)

Assume:

- P5 is coordinator.
 - P5 crashes.
 - P2 detects failure.
-

Step 1

P2 sends ELECTION message to higher processes:

- P3
 - P4
 - P5
-

Step 2

P3 and P4 reply with OK message.

Step 3

Now P3 and P4 independently start election.

Step 4

P4 sends ELECTION message to P5.

But P5 is failed, so no reply.

Step 5

P4 realizes no higher active process exists.

Step 6

P4 becomes new coordinator.

Step 7

P4 sends COORDINATOR message to all lower processes.

Step-by-Step Diagram

P2 ----ELECTION----> P3

P2 ----ELECTION----> P4

P2 ----ELECTION----> P5

P3 ----OK-----> P2

P4 ----OK-----> P2

P4 ----ELECTION----> P5

(No Reply)

P4 ----COORDINATOR----> P1, P2, P3

1 2 Features of Bully Algorithm

- Highest ID wins
- Distributed coordination
- Automatic leader selection
- Fault recovery support

1 3 Advantages

- Simple algorithm
- Fast coordinator selection
- Easy implementation
- Supports fault tolerance

14 Disadvantages

- Large number of messages
 - High network overhead
 - Poor scalability
 - Multiple elections possible
-

15 Applications

Used in:

- Distributed databases
 - Cluster systems
 - Cloud computing
 - Distributed operating systems
-

16 Real-World Industry Example

Google Distributed Servers

In distributed server systems:

- if one leader server fails,
- another server becomes coordinator automatically.

Leader election mechanisms similar to Bully Algorithm are used.

17 Difference Between Bully and Ring Algorithm

Bully Algorithm

Highest ID wins

Faster

More messages

Easy to implement

Ring Algorithm

Token circulates in ring

Slower

Fewer messages

Ring structure required

18 Important Exam Keywords

Write these keywords in exam:

- Coordinator
- Election Message
- OK Message
- Highest Process ID
- Fault Tolerance
- Distributed Coordination
- Leader Election

19 Memory Trick

“HOC”

- **H** → Highest ID
- **O** → OK Message
- **C** → Coordinator

Remember:

Bully Algorithm = Highest ID becomes Coordinator.

Conclusion

Election algorithms are used to select a new coordinator when the current coordinator fails. In Bully Algorithm, the active process with highest ID becomes the coordinator and manages distributed system operations efficiently.

5/14/2026, 8:26:40 PM



5 XDR / Marshalling / Unmarshalling — [8M]

[8 Marks]

“Define External Data Representation, Marshalling and Unmarshalling. Discuss the three alternative approaches to XDR and marshalling.”

Asked in:

May/Jun 2025

[8 Marks]

“Explain external data representation (XDR), marshalling and unmarshalling. Why is XDR required? Discuss in brief the three alternative approaches to external data representation.”

5/14/2026, 8:27:26 PM



Q. Define External Data Representation, Marshalling and Unmarshalling. Discuss the three alternative approaches to XDR and Marshalling.

(8 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, different computers communicate with each other.

But different computers may use:

- different operating systems,
- different data formats,
- different architectures.

So, data must be converted into a common format before transmission.

This is done using:

- XDR,
 - Marshalling,
 - and Unmarshalling.
-

2 Definition of XDR

External Data Representation (XDR)

XDR is a standard format used to represent data in a machine-independent form so that different computers can exchange data correctly.

3 Definition of Marshalling

Marshalling

Marshalling is the process of converting data or objects into a standard transferable format before sending over network.

4 Definition of Unmarshalling

Unmarshalling

Unmarshalling is the process of converting received standard data back into original machine-specific format.

5 Why XDR is Required

XDR is required because different systems may have:

- different byte order,
- different data formats,
- different architectures.

Without XDR:

- data may be interpreted incorrectly.
-

6 Real-Life Analogy

Language Translation Example

Imagine:

- one person speaks Marathi,

- another speaks English.

Translator converts language into common understandable form.

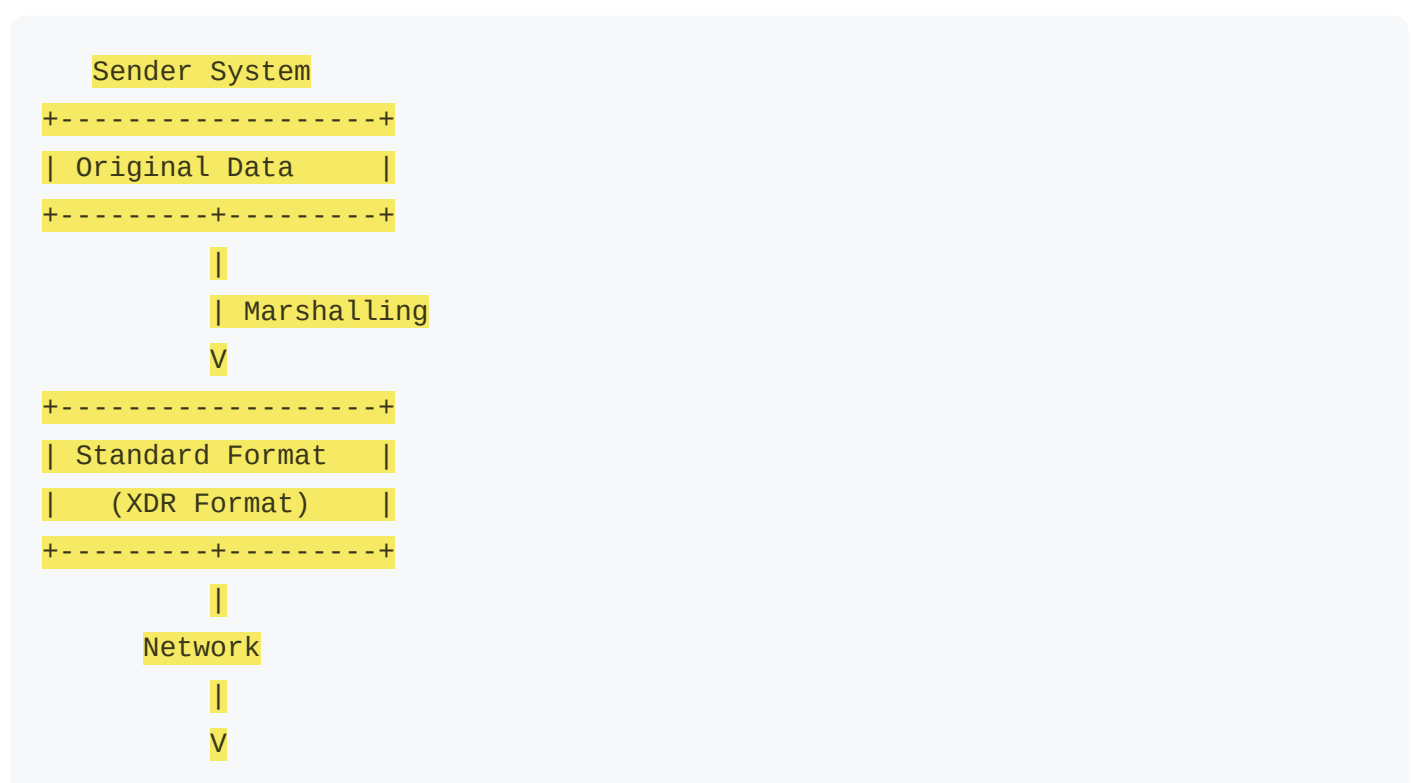
Similarly:

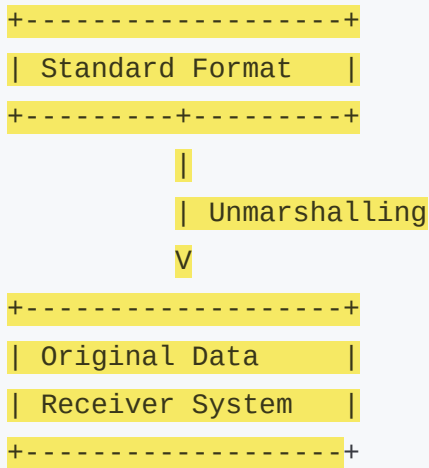
- XDR converts data into common machine-independent format.

Relation with XDR

Real Life	Distributed System
Translator	XDR
Language Conversion	Data Conversion
Common Language	Standard Format

7 Diagram of Marshalling and Unmarshalling





8 Explanation of Diagram

Marshalling

Converts original data into standard transferable format.

XDR Format

Machine-independent data representation.

Unmarshalling

Converts received standard data back into original form.

9 Working of Marshalling and Unmarshalling

Step 1

Sender prepares data.

Step 2

Marshalling converts data into XDR format.

Step 3

Data is sent over network.

Step 4

Receiver gets standardized data.

Step 5

Unmarshalling converts data back into original format.

10 Three Alternative Approaches to XDR / External Data Representation

1. CORBA Common Data Representation (CDR)

Used in CORBA distributed systems.

Features

- Standard binary format
 - Platform independent
 - Supports object communication
-

2. XML-Based Representation

Uses XML tags to represent data.

Features

- Human readable
- Platform independent
- Easy web integration

Disadvantage

- Larger message size
-

3. Java Object Serialization

Converts Java objects into byte stream.

Features

- Simple in Java systems
- Easy object transfer

Disadvantage

- Java dependent
-

1 1 Advantages of XDR

- Platform independent communication
 - Easy data exchange
 - Supports heterogeneous systems
 - Reliable communication
-

1 2 Disadvantages of XDR

- Additional processing overhead
 - Increased complexity
 - Conversion consumes time
-

1 3 Applications

Used in:

- RPC systems
 - Distributed databases
 - Client-server systems
 - Web services
 - Cloud computing
-

14 Real-World Industry Example

Amazon Web Services

Amazon cloud services exchange data between different systems using standardized formats similar to XDR for interoperability.

15 Difference Between Marshalling and Unmarshalling

Marshalling

Converts data to standard format

Done at sender side

Before transmission

Unmarshalling

Converts standard format to original data

Done at receiver side

After receiving

16 Important Exam Keywords

Write these keywords in exam:

- Machine Independent
- Data Conversion
- Standard Format
- Serialization
- Marshalling
- Unmarshalling
- Interoperability
- XDR

17 Memory Trick

“MUX”

- **M** → Marshalling
- **U** → Unmarshalling
- **X** → XDR

Remember:

Data → Marshalling → XDR → Unmarshalling

18 Conclusion

XDR, Marshalling, and Unmarshalling are important mechanisms for data communication in distributed systems.

They help different computers exchange data correctly using standard machine-independent representation formats.

5/14/2026, 8:27:29 PM



6 Mutual Exclusion — [9M]

[9 Marks]

“What are the two important properties of token-based approach? Explain token-ring algorithm to achieve mutual exclusion in a distributed system.”

Asked in:

Nov/Dec 2024

[9 Marks]

“What is the need of mutual exclusion? Explain the permission-based centralized distributed mutual exclusion algorithm.”

5/14/2026, 8:27:47 PM



Q. What are the two important properties of token-based approach? Explain token-ring algorithm to achieve mutual exclusion in a distributed system.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, many processes may try to use the same resource at the same time.

For example:

- printer,
- file,
- database,
- shared memory.

If multiple processes access it together, conflicts may occur.

Mutual exclusion ensures that only one process accesses the shared resource at a time.

2 Definition

Mutual Exclusion

Mutual exclusion is a technique used in distributed systems to allow only one process at a time to enter the critical section (shared resource area).

3 Token-Based Approach

In token-based mutual exclusion:

- a special message called **Token** is used.
 - Only the process holding the token can enter critical section.
-

4 Two Important Properties of Token-Based Approach

1. Uniqueness of Token

Only one token exists in the system.

This ensures:

- only one process enters critical section.

2. Token Possession Grants Access

Only the process holding token can use shared resource.

Without token:

- process must wait.

5 Real-Life Analogy

Classroom Mic Example

Imagine:

- only student holding microphone can speak.
- others must wait for mic.

Relation with Token Ring

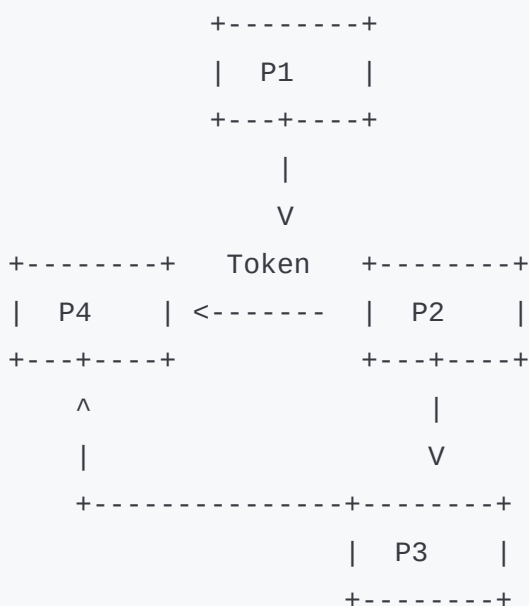
Real Life	Distributed System
Microphone	Token
Student	Process
Speaking	Critical Section

6 Token Ring Algorithm

Definition

Token Ring Algorithm is a distributed mutual exclusion algorithm in which processes are arranged in a logical ring and token circulates around the ring.

7 Token Ring Diagram



8 Explanation of Diagram

Processes

P1, P2, P3, P4 form logical ring.

Token

Special permission message.

Critical Section Access

Only token holder can access shared resource.

9 Working of Token Ring Algorithm

Step 1

Processes are connected in logical ring.

Step 2

Token continuously circulates in ring.

Step 3

When process receives token:

- if it needs resource,
 - it enters critical section.
-

Step 4

After completing work:

- process releases resource,
 - passes token to next process.
-

Step 5

Other processes wait until token arrives.

10 Advantages of Token Ring Algorithm

- No starvation
- Fair access
- Simple implementation
- No deadlock

11 Disadvantages

- Token loss problem
- Ring failure affects system
- Token management overhead
- Delay if token moves slowly

12 Applications

Used in:

- Distributed databases
- Network systems
- Shared printers
- Distributed operating systems

13 Industry Example

Google Distributed Resource Access

Distributed systems use controlled access mechanisms similar to token-based methods to avoid simultaneous resource conflicts.

14 Difference Between Token-Based and Permission-Based Approach

Token-Based	Permission-Based
Uses token	Uses permission messages
Only token holder enters CS	Requires permission from others
Less message overhead	More message overhead
Token loss possible	No token issue

15 Important Exam Keywords

Write these keywords in exam:

- Mutual Exclusion
- Critical Section
- Token
- Logical Ring
- Shared Resource
- Deadlock Free
- Fairness

16 Memory Trick

“UTA”

- **U** → Unique Token
 - **T** → Token Access
 - **A** → Access Shared Resource
-

17 Conclusion

Token Ring Algorithm achieves distributed mutual exclusion using a circulating token. Only the token holder can enter the critical section, ensuring safe and fair resource sharing.

Q. What is the need of mutual exclusion? Explain the permission-based centralized distributed mutual exclusion algorithm.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In distributed systems, many processes share common resources.

If multiple processes use same resource simultaneously:

- data inconsistency,

- corruption,
- and conflicts may occur.

Mutual exclusion prevents this problem.

2 Definition

Mutual Exclusion

Mutual exclusion is a mechanism that allows only one process at a time to access shared resources.

3 Need of Mutual Exclusion

Mutual exclusion is needed to:

- avoid data corruption,
 - maintain consistency,
 - prevent conflicts,
 - ensure proper synchronization.
-

4 Real-Life Analogy

ATM Machine Example

Only one person can use ATM machine at a time.

Others must wait until previous user finishes.

Similarly:

- only one process accesses critical section at a time.

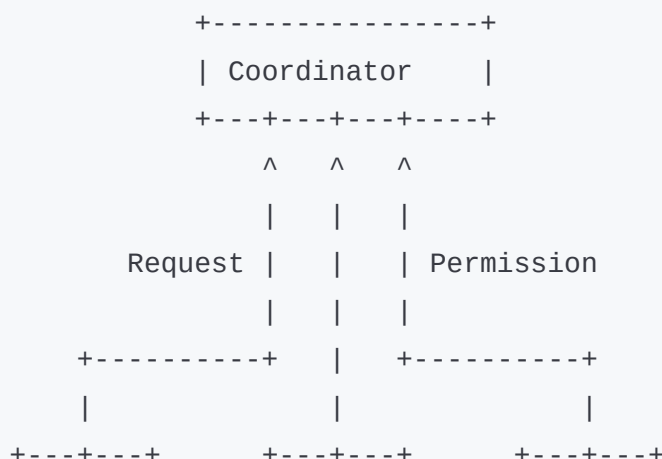
5 Permission-Based Centralized Mutual Exclusion Algorithm

Definition

In this algorithm:

- one process acts as Coordinator,
- all processes request permission from coordinator before entering critical section.

6 Diagram



P1	P2	P3
+-----+	+-----+	+-----+

7 Working of Centralized Algorithm

Step 1

Process sends request to coordinator.

Step 2

Coordinator checks resource availability.

Step 3

If resource is free:

- coordinator sends permission.

Otherwise:

- request waits in queue.

Step 4

Process enters critical section.

Step 5

After completion:

- process sends release message.
-

Step 6

Coordinator grants permission to next waiting process.

8 Advantages

- Simple implementation
 - Easy coordination
 - Fair scheduling
 - No simultaneous access
-

9 Disadvantages

- Single point of failure
- Coordinator bottleneck

- Poor scalability
- High dependency on coordinator

10 Applications

Used in:

- Distributed databases
- Banking systems
- Shared file systems
- Cloud services

11 Difference Between Centralized and Token Ring Algorithm

Centralized

Coordinator required

Simple

Coordinator failure issue

Bottleneck possible

Token Ring

Token required

Fair access

Token loss issue

Ring delay possible

12 Important Exam Keywords

- Critical Section
 - Coordinator
 - Permission Request
 - Shared Resource
 - Synchronization
 - Mutual Exclusion
 - Distributed Coordination
-

13 Memory Trick

“RPR”

- **R** → Request
- **P** → Permission
- **R** → Release

Remember:

Process follows Request → Permission → Release.

14 Conclusion

Mutual exclusion prevents simultaneous access to shared resources in distributed systems. The centralized permission-based algorithm uses a coordinator to safely control critical section access.

5/14/2026, 8:27:57 PM



7 NTP — [9M]

[9 Marks]

“What is NTP? With the help of a diagram, describe how NTP works.”

5/14/2026, 8:28:41 PM



Q. What is NTP? With the help of a diagram, describe how NTP works.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, different computers have different clock timings.

To perform correct operations:

- all systems should maintain approximately same time.

NTP helps synchronize clocks over the Internet or network.

2 Definition

NTP (Network Time Protocol)

NTP is a protocol used to synchronize the clocks of computers in a distributed system over a network.

It helps maintain accurate and consistent time among all connected systems.

3 Why NTP is Needed

NTP is needed because:

- computer clocks drift over time,
 - distributed systems require same time,
 - event ordering depends on proper timing,
 - logging and transactions require accurate timestamps.
-

4 Real-Life Analogy

School Bell Example

Imagine:

- all classrooms follow one central school bell.

This keeps every class synchronized.

Similarly:

- NTP synchronizes all computer clocks using a standard time server.
-

5 Components of NTP

1. NTP Server

Provides accurate standard time.

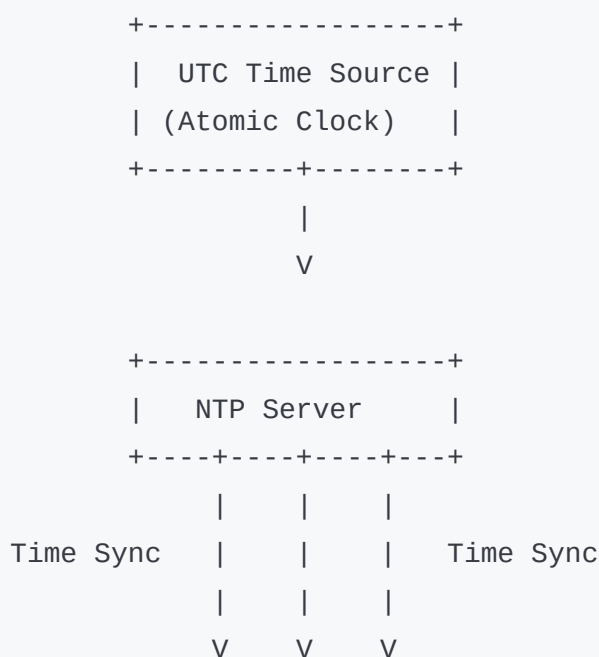
2. NTP Client

Requests time from server.

3. Network

Transfers synchronization messages.

6 NTP Architecture Diagram



```
+-----+ +-----+ +-----+
| C1 | | C2 | | C3 |
+-----+ +-----+ +-----+
```

7 Explanation of Diagram

UTC Time Source

- Accurate atomic clock or GPS clock.

NTP Server

- Receives standard time.
- Sends synchronized time to clients.

Clients (C1, C2, C3)

- Synchronize their clocks with server.

8 Working of NTP (Step-by-Step)

Step 1

NTP server gets accurate time from UTC source.

Step 2

Client sends time request to NTP server.

Step 3

Server replies with current timestamp.

Step 4

Client calculates:

- network delay,
 - round-trip delay,
 - time difference.
-

Step 5

Client adjusts local clock accordingly.

Step 6

Synchronization repeats periodically.

9 NTP Timestamp Exchange

NTP uses four timestamps:

Timestamp	Meaning
T1	Request sent time
T2	Request received time
T3	Reply sent time
T4	Reply received time

These help calculate:

- delay,
- clock offset.

10 Features of NTP

- High accuracy
- Internet-based synchronization
- Automatic synchronization
- Scalable
- Reliable

11 Advantages of NTP

- Maintains synchronized clocks

- Supports distributed systems
 - Accurate event ordering
 - Improves consistency
 - Easy centralized management
-

1 2 Disadvantages of NTP

- Network delay affects accuracy
 - Internet dependency
 - Security attacks possible
 - Configuration complexity
-

1 3 Applications of NTP

Used in:

- Banking systems
 - Cloud computing
 - Distributed databases
 - Airline reservation systems
 - Data centers
-

1 4 Real-World Industry Example

Google Data Centers

Google data centers across the world use time synchronization similar to NTP for:

- distributed logging,
- database consistency,
- event ordering.

15 Difference Between Berkeley Algorithm and NTP

Berkeley Algorithm	NTP
Internal synchronization	Internet-based synchronization
Uses average system time	Uses UTC standard time
Centralized local network	Global synchronization
No external clock required	Requires external UTC source

16 Important Exam Keywords

Write these keywords in exam:

- Network Time Protocol
- UTC
- Clock Synchronization
- Timestamp
- Clock Drift

- Time Server
 - Distributed Coordination
-

17 Memory Trick

“URTC”

- **U** → UTC Time
 - **R** → Request
 - **T** → Timestamp
 - **C** → Clock Adjustment
-

18 Conclusion

NTP is a widely used protocol for synchronizing clocks in distributed systems over a network. It maintains accurate and consistent timing using time servers and timestamp-based synchronization mechanisms.

5/14/2026, 8:28:43 PM



8 Lamport Logical Clock — [8M]
[8 Marks]

“What are the advantages of logical clock over physical clock? Consider Figure 1 that shows four processes (P1, P2, P3, P4)... Provide timestamps for each labeled event.”
Asked in:

May/Jun 2024
[8 Marks]

“Describe Lamport’s clock algorithm and give an example of how to order events in a distributed system. What are the algorithm’s limitations?”

5/14/2026, 8:29:09 PM



Q. What are the advantages of logical clock over physical clock? Describe Lamport Logical Clock algorithm and explain event ordering in distributed systems.

(8 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, many computers perform events independently.

Because physical clocks may show different times, it becomes difficult to identify:

- which event happened first,
- and correct event sequence.

Lamport introduced **Logical Clocks** to solve this problem.

2 Definition

Logical Clock

A Logical Clock is a software-based clock used to maintain the logical ordering of events in a distributed system.

It does not measure real time.

It only maintains:

- event sequence,
 - and causal relationship.
-

3 Why Logical Clock is Needed

Logical clocks are needed because:

- physical clocks may differ,
 - synchronization is difficult,
 - event ordering is important.
-

4 Advantages of Logical Clock over Physical Clock

Logical Clock	Physical Clock
Maintains event ordering	Maintains actual time
No clock synchronization required	Synchronization required
Avoids clock drift problem	Affected by clock drift
Simple implementation	Complex synchronization
Suitable for distributed systems	Difficult in distributed systems

5 Real-Life Analogy

Number Token Example

Imagine:

- customers take token numbers in bank.
- Token numbers determine order.

Actual wall clock time is not important.

Similarly:

- logical clock determines order of events.

6 Lamport Logical Clock Algorithm

Definition

Lamport Logical Clock Algorithm assigns timestamps to events to maintain proper event ordering in distributed systems.

7 Rules of Lamport Clock Algorithm

Rule 1 - Increment Before Every Event

Before executing any event:

- increment local clock by 1.
-

Rule 2 - Sending Message

When sending message:

- attach current timestamp with message.
-

Rule 3 - Receiving Message

When receiving message:

- set clock value as:

$$C_j = \max(C_j, t_m) + 1$$

Where:

- C_j = receiver clock
- t_m = received timestamp

8 Diagram of Lamport Logical Clock

P1: a(1) ----m1----> b(4)

P2: c(1) ----m2----> d(3)

P3: e(1) <---m1----- f(5)

9 Explanation of Diagram

Process P1

- Event a occurs → timestamp 1
- Sends message m1

Process P3

- Receives m1
- Updates logical clock using Lamport rule

10 Event Ordering in Lamport Clock

If:

$$a \rightarrow b$$

then:

- event a happened before event b.

This is called:

Happened-Before Relationship

1 1 Working Example

Assume:

- all clocks initially = 0
-

Step 1

P1 executes event:

Clock becomes 1.

Step 2

P1 sends message with timestamp 1.

Step 3

Receiver gets message.

Suppose receiver clock = 2.

Using Lamport rule:

$$\max(2, 1) + 1 = 3$$

Receiver clock becomes 3.

1 2 Features of Lamport Clock

- Logical ordering
 - No physical synchronization
 - Simple timestamp mechanism
 - Supports distributed coordination
-

1 3 Advantages

- Simple implementation
 - Avoids clock drift
 - Maintains event ordering
 - Efficient distributed coordination
-

14 Limitations of Lamport Clock

1. Cannot Detect Concurrent Events

Two independent events may still appear ordered.

2. Does Not Represent Real Time

Logical clocks do not show actual physical time.

3. Partial Ordering Only

Cannot fully determine all causal relationships.

15 Applications

Used in:

- Distributed databases
 - Distributed operating systems
 - Event synchronization
 - Distributed transaction systems
-

16 Industry Example

Amazon Distributed Databases

Amazon distributed services maintain event ordering using logical timestamp concepts for:

- transactions,
- synchronization,
- distributed coordination.

17 Difference Between Physical Clock and Logical Clock

Physical Clock	Logical Clock
Measures real time	Measures event order
Hardware clock	Software clock
Synchronization needed	No synchronization needed
Affected by drift	No drift problem

18 Important Exam Keywords

Write these keywords in exam:

- Logical Timestamp
- Event Ordering

- Happened-Before Relation
 - Clock Drift
 - Causal Ordering
 - Distributed Coordination
 - Lamport Clock
-

19 Memory Trick

“SIR”

- **S** → Send timestamp
- **I** → Increment clock
- **R** → Receive and update

Remember:

Lamport Clock = Increment → Send → Receive Update

20 Conclusion

Lamport Logical Clock helps maintain event ordering in distributed systems without requiring physical clock synchronization.

It uses logical timestamps to establish causal relationships between distributed events efficiently.

5/14/2026, 8:29:10 PM



UNIT 4 (Q3, Q4)

🔥 4 TIMES REPEATED

1 Checkpointing — [9M]

[9 Marks]

“What is checkpointing in a distributed system? Explain the working of Coordinated checkpointing recovery mechanism.”

5/14/2026, 8:29:50 PM



Q. What is checkpointing in a distributed system? Explain the working of Coordinated Checkpointing Recovery Mechanism.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, many computers work together.

Sometimes:

- system crashes,
- power failure occurs,
- or process stops unexpectedly.

If failure happens, all work may be lost.

Checkpointing helps save system state so recovery becomes easier.

2 Definition

Checkpointing

Checkpointing is the process of saving the current state of a process or distributed system periodically so that the system can restart from that saved state after failure.

3 Why Checkpointing is Needed

Checkpointing is needed because:

- distributed systems may fail,
 - restarting from beginning wastes time,
 - data loss should be avoided,
 - recovery should become faster.
-

4 Real-Life Analogy

Video Game Save Point Example

Imagine:

- while playing a game,

- progress is saved automatically.

If game crashes:

- player restarts from saved point,
- not from beginning.

Similarly:

- checkpoint stores process state for recovery.
-

5 Coordinated Checkpointing

Definition

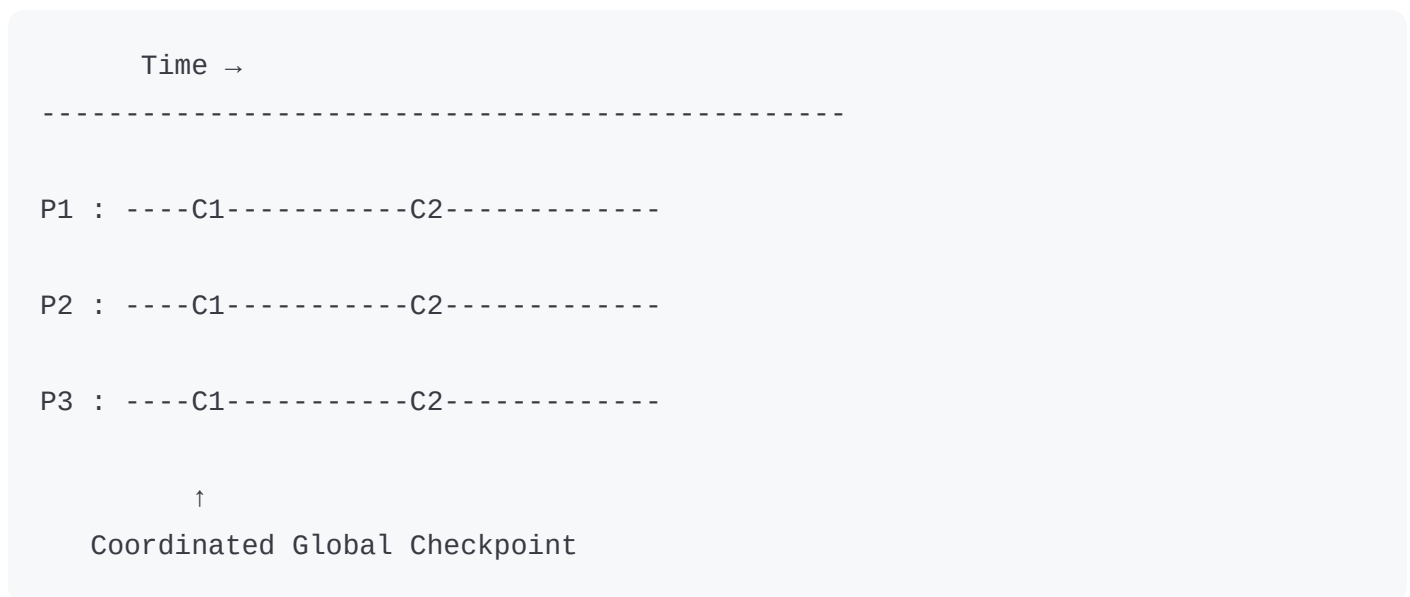
Coordinated checkpointing is a recovery mechanism in which all processes coordinate together and take checkpoints simultaneously to create a globally consistent state.

6 Need of Coordinated Checkpointing

It helps:

- avoid inconsistent checkpoints,
 - simplify recovery,
 - prevent domino effect,
 - maintain consistency.
-

7 Diagram of Coordinated Checkpointing



8 Explanation of Diagram

Processes P1, P2, P3

Distributed processes running simultaneously.

C1, C2

Checkpoint positions.

Global Checkpoint

All processes save state together.

9 Working of Coordinated Checkpointing (Step-by-Step)

Step 1

Coordinator initiates checkpoint request.

Step 2

All processes stop normal execution temporarily.

Step 3

Each process saves:

- memory state,
 - variables,
 - message status,
 - execution information.
-

Step 4

Processes confirm checkpoint completion.

Step 5

System resumes normal execution.

Step 6

If failure occurs:

- system rolls back to latest checkpoint.
-

10 Recovery Mechanism

When system crashes:

- processes reload saved checkpoints,
 - communication state is restored,
 - execution resumes from saved state.
-

1 1 Domino Effect

Definition

Domino effect occurs when rollback of one process forces rollback of many other processes repeatedly.

Coordinated checkpointing avoids domino effect because:

- all checkpoints are globally consistent.
-

12 Features of Coordinated Checkpointing

- Global consistency
 - Simple recovery
 - Fault tolerance
 - Coordinated rollback
 - Reliable recovery
-

13 Advantages

- Avoids domino effect
 - Easy recovery
 - Consistent checkpoints
 - Reliable fault recovery
 - Simplified management
-

14 Disadvantages

- Processes must pause
 - Coordination overhead
 - Increased communication
 - Scalability issues in large systems
-

15 Applications

Used in:

- Distributed databases
 - Cloud computing
 - Banking systems
 - Scientific computing
 - Distributed operating systems
-

16 Industry Example

Google Distributed Data Centers

Google distributed servers periodically save system state for:

- fault recovery,
- data consistency,
- distributed computation recovery.

Checkpointing helps restore operations after failures.

17 Difference Between Coordinated and Uncoordinated Checkpointing

Coordinated Checkpointing	Uncoordinated Checkpointing
Processes coordinate together	Processes checkpoint independently
Globally consistent	May become inconsistent
Avoids domino effect	Domino effect possible
Simple recovery	Complex recovery

18 Important Exam Keywords

Write these keywords in exam:

- Checkpoint
 - Recovery Mechanism
 - Fault Tolerance
 - Rollback Recovery
 - Global Consistency
 - Domino Effect
 - Coordinator Process
-

19 Memory Trick

“SCRR”

- **S** → Save state
- **C** → Coordination
- **R** → Recovery
- **R** → Rollback

Remember:

Checkpointing = Save → Coordinate → Recover

20 Conclusion

Checkpointing is an important recovery mechanism in distributed systems used to save process states periodically.

Coordinated checkpointing provides globally consistent checkpoints and helps recover efficiently after system failures.

5/14/2026, 8:29:52 PM



2 Replica Management / Replication — [9M]
[9 Marks]

“What are the key issues in Replica Management? Explain the following with respect to content replication and placement with suitable diagram:

- i) Permanent Replicas
- ii) Server-Initiated Replicas
- iii) Client-Initiated Replicas”

Asked in:

Nov/Dec 2023

May/Jun 2023

[9 Marks]

“Discuss the two important reasons for wanting to replicate data and how does replication relate to scalability.”

5/14/2026, 11:11:54 PM



Q. What are the key issues in Replica Management? Explain the following with respect to content replication and placement with suitable diagram:

i) Permanent Replicas

ii) Server-Initiated Replicas

iii) Client-Initiated Replicas

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, data is often copied and stored at multiple locations.

These copies are called **Replicas**.

Replication improves:

- availability,
- performance,
- fault tolerance,
- and scalability.

Managing these copies properly is called **Replica Management**.

2 Definition

Replica Management

Replica Management is the process of creating, placing, updating, and maintaining multiple copies of data in distributed systems.

3 Why Replication is Needed

Replication is needed to:

- improve data availability,
- reduce server load,

- increase performance,
 - provide fault tolerance.
-

4 Real-Life Analogy

Library Book Example

Imagine:

- only one copy of book exists in library.
- Many students must wait.

If multiple copies are available:

- students get book quickly.

Similarly:

- replication stores multiple copies of data for faster access.
-

5 Key Issues in Replica Management

1. Replica Placement

Where replicas should be stored.

2. Consistency Maintenance

All copies should contain updated data.

3. Scalability

System should support large number of users.

4. Fault Tolerance

Data should remain available even during failures.

5. Synchronization

Updates should propagate to all replicas.

6. Communication Overhead

Replication should minimize extra network traffic.

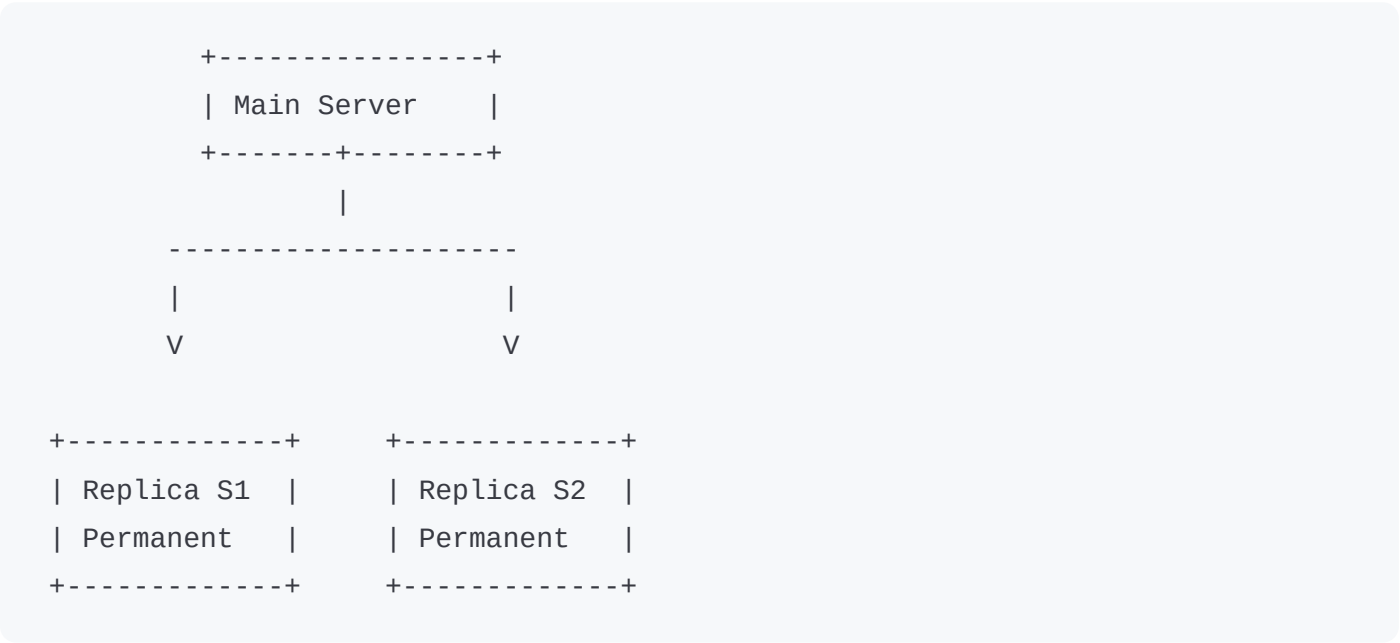
Types of Replicas

i) Permanent Replicas

Definition

Permanent replicas are fixed copies of data stored permanently at specific servers.

Diagram



Features

- Always available
- Stored permanently
- Located at fixed positions

Example

Mirror websites.

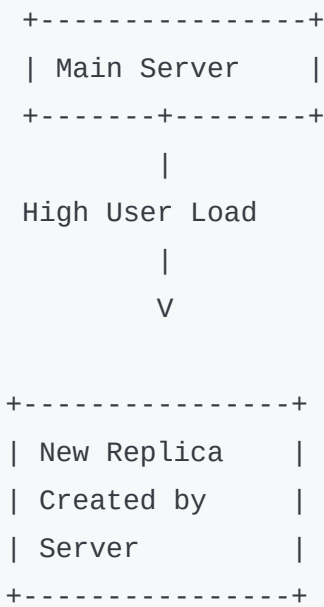
ii) Server-Initiated Replicas

Definition

Replicas created automatically by server depending on:

- workload,
- user demand,
- traffic.

Diagram



Features

- Dynamic replication
 - Server controls creation
 - Improves load balancing
-

Example

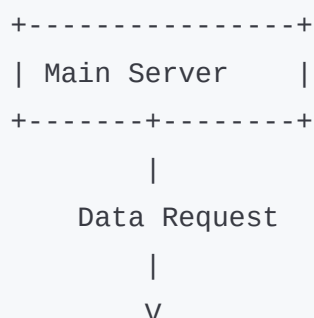
Cloud servers creating additional copies during heavy traffic.

iii) Client-Initiated Replicas

Definition

Replicas created at client side based on client requests.

Diagram



```
+-----+
| Client Cache |
| Local Replica|
+-----+
```

Features

- Client stores local copy
- Faster access
- Reduces network communication

Example

Browser caching.

7 Difference Between Replica Types

Permanent	Server-Initiated	Client-Initiated
Fixed replicas	Created by server	Created by client
Permanent storage	Dynamic	Local caching
Controlled centrally	Server controlled	Client controlled

8 Advantages of Replication

- Better availability
 - Faster data access
 - Improved fault tolerance
 - Better scalability
 - Reduced server load
-

9 Disadvantages of Replication

- Data consistency problem
 - Storage overhead
 - Synchronization complexity
 - Increased communication cost
-

10 Applications

Used in:

- Cloud computing
 - Distributed databases
 - CDN systems
 - Video streaming
 - Banking systems
-

11 Industry Example

Netflix CDN Replication

Netflix stores video copies at multiple servers worldwide.

This replication:

- improves speed,
 - reduces delay,
 - and handles millions of users efficiently.
-

12 Important Exam Keywords

Write these keywords in exam:

- Replica Placement
 - Scalability
 - Fault Tolerance
 - Consistency
 - Synchronization
 - Content Replication
 - Availability
-

13 Memory Trick

"PSC"

- **P** → Permanent
- **S** → Server-Initiated
- **C** → Client-Initiated

Remember:

Types of Replicas = PSC

14 Conclusion

Replica Management is important for maintaining multiple copies of data efficiently in distributed systems.

Different replica placement strategies improve scalability, availability, and system performance.

Q. Discuss the two important reasons for wanting to replicate data and how does replication relate to scalability.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

Distributed systems handle huge amounts of users and data.

If all users access one server:

- system becomes slow,
- overloaded,
- and unreliable.

Replication solves this problem by storing multiple copies of data.

2 Definition

Replication

Replication is the process of storing multiple copies of data at different locations in a distributed system.

3 Two Important Reasons for Replication

1. Reliability / Availability

Replication improves data availability.

If one server fails:

- another replica provides data.

Benefit

System continues working during failures.

2. Performance Improvement

Replication reduces access delay.

Users can access nearest replica.

Benefit

Faster response time and reduced server load.

4 Real-Life Analogy

Multiple ATM Example

Imagine:

- only one ATM exists in city.
- huge crowd creates delay.

If many ATMs exist:

- people get faster service.

Similarly:

- replication distributes load across many servers.
-

5 How Replication Relates to Scalability

Scalability

Scalability means system can handle increasing users and workload efficiently.

Relation with Replication

Replication improves scalability because:

1. Load Distribution

Requests are divided among replicas.

2. Reduced Bottleneck

Single server is not overloaded.

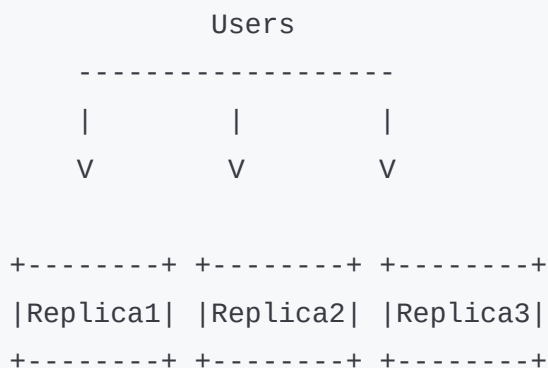
3. Faster User Access

Users access nearby replica.

4. Better Resource Utilization

Multiple servers share workload.

6 Diagram of Replication and Scalability



7 Explanation of Diagram

- Multiple replicas handle user requests.
- Load is distributed.

- System performance improves.
-

8 Advantages of Replication

- High availability
 - Better performance
 - Scalability improvement
 - Fault tolerance
 - Faster access
-

9 Disadvantages

- Data consistency issues
 - Synchronization overhead
 - Increased storage requirement
-

10 Applications

Used in:

- Cloud computing
- Distributed databases
- Video streaming
- E-commerce

- Social media
-

1 1 Industry Example

YouTube Content Replication

YouTube stores video copies globally.

This:

- reduces buffering,
 - improves speed,
 - and supports millions of users.
-

1 2 Important Exam Keywords

- Replication
 - Scalability
 - Availability
 - Load Balancing
 - Fault Tolerance
 - Distributed Storage
 - Replica Placement
-

13 Memory Trick

“RPS”

- **R** → Reliability
- **P** → Performance
- **S** → Scalability

Remember:

Replication improves Reliability, Performance, and Scalability.

14 Conclusion

Replication improves availability and performance by storing multiple copies of data. It helps distributed systems achieve scalability by distributing workload across multiple servers efficiently.

5/14/2026, 11:11:57 PM



[9 Marks]

“Why replication is important and how it relates with Scalability? How are replicas kept consistent? Explain the mechanism of Server-Initiated Replicas with suitable example.”

5/14/2026, 11:12:27 PM



Q. Why replication is important and how it relates with Scalability? How are replicas kept consistent? Explain the mechanism of Server-Initiated Replicas with suitable example.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, many users access the same data simultaneously.

If only one copy of data exists:

- server becomes overloaded,
- system becomes slow,
- and failures may stop services.

Replication solves this problem by storing multiple copies of data.

2 Definition

Replication

Replication is the process of storing multiple copies of data at different locations in a distributed system.

3 Why Replication is Important

Replication is important because:

1. Improves Availability

If one server fails:

- another replica provides data.
-

2. Improves Performance

Users access nearest replica.

This reduces delay.

3. Provides Fault Tolerance

System continues working during failures.

4. Reduces Server Load

Workload is distributed among replicas.

5. Faster Data Access

Nearby replicas improve response time.

4 Real-Life Analogy

Multiple Restaurant Branches Example

Imagine:

- only one restaurant branch exists.
- huge crowd causes delay.

If many branches exist:

- customers get faster service.

Similarly:

- multiple replicas improve distributed system performance.

5 Relation Between Replication and Scalability

Scalability

Scalability means:

system can handle increasing users and workload efficiently.

How Replication Improves Scalability

1. Load Distribution

User requests are divided among replicas.

2. Reduced Bottleneck

Single server does not become overloaded.

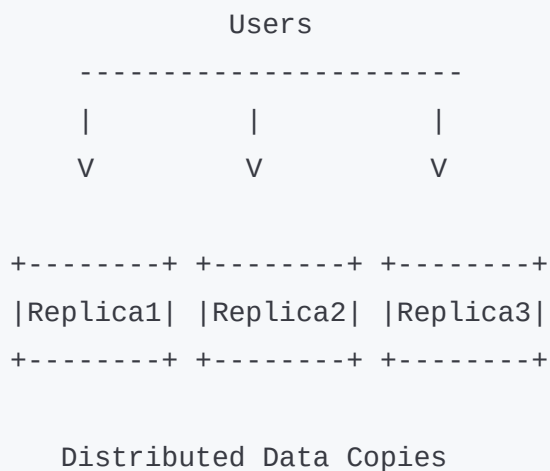
3. Parallel Processing

Multiple replicas serve users simultaneously.

4. Faster Access

Nearest replica reduces network delay.

6 Diagram of Replication and Scalability



7 How Replicas are Kept Consistent

Replica Consistency

All replicas should contain same updated data.

Consistency Mechanisms

1. Update Propagation

Whenever data changes:

- updates are sent to all replicas.
-

2. Synchronization

Replicas synchronize periodically.

3. Primary-Based Protocol

One primary server controls updates.

4. Write Propagation

Write operations are copied to all replicas.

Problems in Replica Consistency

- Delayed updates

- Conflicting data
 - Network failures
 - Synchronization overhead
-

9 Server-Initiated Replicas

Definition

In Server-Initiated Replication:

- server automatically creates replicas
 - depending on workload and demand.
-

10 Mechanism of Server-Initiated Replicas

Step 1

Server monitors user requests.

Step 2

If traffic becomes high:

- server creates additional replicas.

Step 3

Replica copies are placed near users.

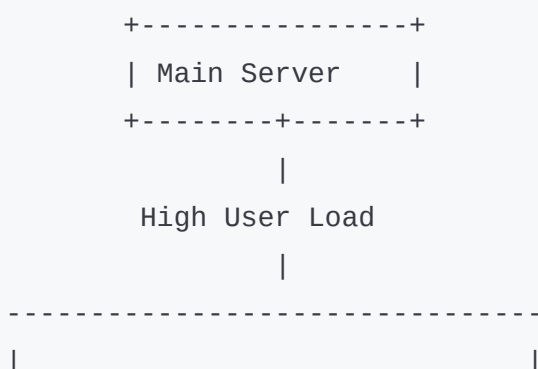
Step 4

User requests are distributed among replicas.

Step 5

Updates are synchronized among all replicas.

1 1 Diagram of Server-Initiated Replication



V

```
+-----+
| Replica S1 |
| Created by |
| Server     |
+-----+
```

V

```
+-----+
| Replica S2 |
| Created by |
| Server     |
+-----+
```

1 2 Suitable Example

Netflix Video Streaming

When millions of users watch same movie:

- Netflix automatically creates replicas on nearby servers.

This:

- reduces buffering,
- improves speed,
- and balances load.

1 3 Advantages of Server-Initiated Replicas

- Automatic scaling
- Better load balancing
- Faster access

- Improved availability
 - Better scalability
-

1 4 Disadvantages

- Increased storage cost
 - Synchronization overhead
 - Complex management
 - Consistency challenges
-

1 5 Applications

Used in:

- Cloud computing
 - CDN systems
 - Distributed databases
 - Video streaming
 - E-commerce platforms
-

1 6 Difference Between Server and Client Initiated Replicas

Server-Initiated

Replica created by server

Automatic scaling

Centralized control

Used for load balancing

Client-Initiated

Replica created by client

Local caching

Client-side control

Used for faster local access

17 Important Exam Keywords

Write these keywords in exam:

- Replication
- Scalability
- Consistency
- Synchronization
- Update Propagation
- Load Balancing
- Fault Tolerance
- Replica Placement

18 Memory Trick

"RCLS"

- **R** → Replication
- **C** → Consistency

- **L** → Load Balancing
- **S** → Scalability

Remember:

Replication improves Consistency and Scalability.

19 Conclusion

Replication improves availability, performance, and scalability in distributed systems. Server-initiated replicas dynamically create additional copies based on workload and maintain consistency through synchronization mechanisms.

5/14/2026, 11:12:30 PM



3 Causal Consistency — [9M]
[9 Marks]

“What are two primary reasons for replication? Explain the causal consistency model with suitable example using distributed shared database.”

Asked in:

May/Jun 2024

May/Jun 2023

[9 Marks]

“Define data-centric consistency model. Explain the causal consistency model with suitable example using distributed shared database.”

5/14/2026, 11:14:19 PM



Q. What are two primary reasons for replication? Explain the causal consistency model with suitable example using distributed shared database.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, data is stored at multiple locations using replication.

When many copies of same data exist:

- all copies should remain consistent.

Consistency models define rules for maintaining correct data updates.

One important consistency model is **Causal Consistency**.

2 Definition of Replication

Replication

Replication is the process of storing multiple copies of data at different servers in a distributed system.

3 Two Primary Reasons for Replication

1. Improved Availability

If one server fails:

- another replica provides data.

This increases system reliability.

2. Better Performance

Users access nearby replica.

This:

- reduces delay,
 - improves response time,
 - reduces server load.
-

4 Real-Life Analogy

Multiple ATM Example

If city has only one ATM:

- huge crowd forms.

With many ATMs:

- users get faster service.

Similarly:

- replication improves distributed system performance.
-

5 Data-Centric Consistency Model

Definition

A Data-Centric Consistency Model defines rules for maintaining consistency among multiple replicas of shared data in distributed systems.

6 Causal Consistency Model

Definition

Causal Consistency ensures that:

all causally related operations are seen by every process in the same order.

Independent operations may appear in different order.

7 Meaning of Causal Relationship

If one operation depends on another:

- they are causally related.

Example:

- reading a message,
- then replying to it.

Reply depends on original message.

8 Real-Life Analogy

WhatsApp Chat Example

Imagine:

- Person A sends:
"Exam tomorrow."
- Person B replies:
"Okay, I will study."

Everyone should first see:

1. original message,
2. then reply.

If reply appears before original message:

- conversation becomes confusing.

This is causal consistency.

9 Diagram of Causal Consistency

Process P1

Process P2

Write(X=10)

|

V

Read(X=10)

Write(Y=20)

All users must see:

X=10 before Y=20

10 Explanation of Diagram

Step 1

P1 writes:

$X = 10$

Step 2

P2 reads $X = 10$

Step 3

P2 writes:

$Y = 20$

Since Y depends on X:

- all processes must observe:
 $X=10$ before $Y=20$
-

11 Working of Causal Consistency

Step 1

Process performs write operation.

Step 2

Dependent process reads updated value.

Step 3

Dependent process performs another update.

Step 4

System ensures all users see updates in causal order.

1 2 Distributed Shared Database Example

Banking System Example

Suppose:

- balance updated from ₹1000 to ₹500,

- then transaction message generated.

All users must first see:

1. balance deduction,
2. then confirmation message.

Otherwise:

- inconsistency occurs.
-

13 Features of Causal Consistency

- Maintains causal order
 - Supports distributed replication
 - Allows concurrent operations
 - Better consistency management
-

14 Advantages

- Better consistency
 - Correct event ordering
 - Suitable for distributed databases
 - Improves user understanding
-

15 Disadvantages

-
- Complex implementation
 - Synchronization overhead
 - Additional communication cost
-

16 Applications

Used in:

- Distributed databases
 - Social media systems
 - Collaborative applications
 - Messaging systems
 - Cloud computing
-

17 Industry Example

WhatsApp Messaging System

WhatsApp maintains message order:

- original message first,
- replies later.

This follows causal consistency principles.

18 Difference Between Strong Consistency and Causal Consistency

Strong Consistency

All operations appear immediately

Very strict

High overhead

Causal Consistency

Only causally related operations ordered

More flexible

Better scalability

19 Important Exam Keywords

Write these keywords in exam:

- Replication
- Consistency Model
- Causal Relationship
- Event Ordering
- Distributed Database
- Shared Data
- Replica Consistency

20 Memory Trick

“ROW”

- **R** → Read

- **O** → Order
- **W** → Write

Remember:

Read-dependent writes must follow same order.

21 Conclusion

Replication improves availability and performance in distributed systems.

Causal consistency ensures that causally related operations are observed in the correct order across all replicas in distributed databases.

Q. Define Data-Centric Consistency Model. Explain the Causal Consistency Model with suitable example using distributed shared database.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

Distributed systems store data at many locations using replication.

To maintain correct data updates among replicas, consistency models are used.

Data-centric consistency models define how shared data remains consistent across distributed systems.

2 Definition

Data-Centric Consistency Model

A Data-Centric Consistency Model specifies rules for maintaining consistency of shared replicated data in distributed systems.

3 Need of Data-Centric Consistency

It is needed because:

- multiple copies of data exist,
 - updates happen simultaneously,
 - inconsistent data must be avoided.
-

4 Types of Data-Centric Consistency Models

- Strict Consistency
- Sequential Consistency

- Causal Consistency
 - FIFO Consistency
 - Weak Consistency
-

5 Causal Consistency Model

Definition

Causal Consistency ensures that:

all causally related write operations are seen by all processes in same order.

6 Real-Life Analogy

Facebook Comment Example

Suppose:

- one person posts:
"Party tonight!"
- another comments:
"I will come."

Everyone should first see:

- original post,
- then comment.

This is causal consistency.

7 Diagram

Process P1

Process P2

Write(A=1)

|

∨

Read(A=1)

Write(B=2)

All processes see:

A=1 before B=2

8 Working of Causal Consistency

Step 1

One process updates shared data.

Step 2

Another process reads updated value.

Step 3

Dependent write operation occurs.

Step 4

System maintains same causal order everywhere.

9 Distributed Shared Database Example

Online Shopping Example

Suppose:

- stock quantity updated first,
- then order confirmation generated.

All users must first see updated stock before confirmation.

10 Advantages

- Maintains logical order
- Better consistency

- Suitable for distributed applications
 - Supports replication
-

1 1 Disadvantages

- Complex synchronization
 - Communication overhead
 - Difficult implementation
-

1 2 Applications

- Distributed databases
 - Messaging systems
 - Collaborative editing
 - Social media systems
-

1 3 Industry Example

Facebook Social Media Updates

Facebook maintains ordered display of posts and comments using causal ordering concepts.

14 Important Exam Keywords

- Data-Centric Consistency
 - Replication
 - Causal Relationship
 - Event Ordering
 - Shared Database
 - Distributed Consistency
-

15 Memory Trick

“COW”

- **C** → Causal
- **O** → Ordered
- **W** → Writes

Remember:

Causally related writes must remain ordered.

16 Conclusion

Data-centric consistency models help maintain consistency among replicated shared data. Causal consistency ensures correct ordering of dependent operations in distributed shared databases.

5/14/2026, 11:14:22 PM



5 Push vs Pull Protocol — [9M]
[9 Marks]

“Discuss and compare Push versus Pull Protocols propagation design issue for content distribution.”

5/14/2026, 11:42:07 PM



Q. Discuss and compare Push versus Pull Protocols propagation design issue for content distribution.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, data and content must be distributed to many users and servers.

Whenever data changes:

- replicas must receive updated content.

Two important methods used for content propagation are:

- Push Protocol
- Pull Protocol

These protocols decide how updates are distributed among replicas.

2 Definition

Content Distribution

Content distribution is the process of transferring updated data from one server to multiple replicas or users in distributed systems.

3 Propagation Protocols

Propagation protocols define:

- how updates are transferred,
 - when replicas receive updates,
 - and how consistency is maintained.
-

4 Push Protocol

Definition

In Push Protocol:

server automatically sends updates to replicas whenever data changes.

Replicas do not request updates.

5 Pull Protocol

Definition

In Pull Protocol:

replicas periodically request updates from server.

Server sends updates only when requested.

6 Real-Life Analogy

WhatsApp Notification Example

Push Protocol

WhatsApp automatically sends notifications to your phone.

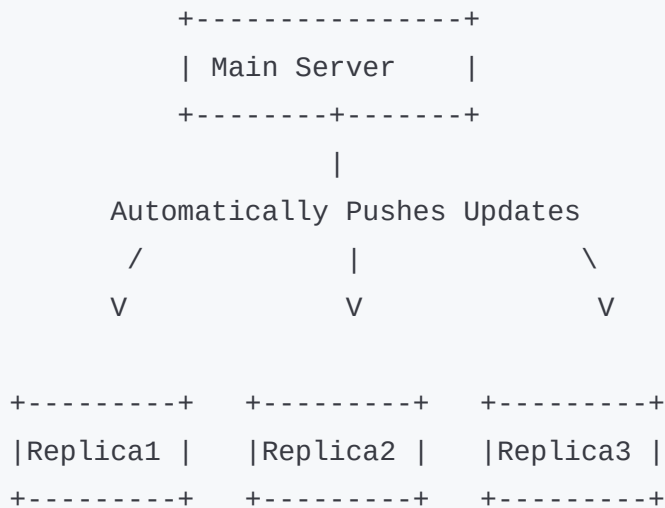
Pull Protocol

Refreshing Instagram feed manually to check new posts.

Relation with Distributed System

Real Life	Distributed System
WhatsApp notification	Push
Manual refresh	Pull

7 Diagram of Push Protocol



8 Working of Push Protocol

Step 1

Server data changes.

Step 2

Server immediately sends updates.

Step 3

All replicas receive latest content.

9 Advantages of Push Protocol

- Fast updates
- Better consistency
- Immediate synchronization
- Suitable for frequently changing data

10 Disadvantages of Push Protocol

- High communication overhead
- Unnecessary updates possible
- Increased server workload

1 1 Diagram of Pull Protocol



```
| Replica3 | -----/  
+-----+
```

1 2 Working of Pull Protocol

Step 1

Replica checks periodically for updates.

Step 2

Replica sends request to server.

Step 3

Server sends updated content if available.

1 3 Advantages of Pull Protocol

- Lower communication overhead
- Better bandwidth usage
- Less server load
- Suitable for rarely changing data

14 Disadvantages of Pull Protocol

- Delayed updates
 - Temporary inconsistency
 - Client polling overhead
-

15 Comparison Between Push and Pull Protocols

Push Protocol	Pull Protocol
Server sends updates automatically	Client requests updates
Immediate updates	Delayed updates
High communication overhead	Lower overhead
Better consistency	Temporary inconsistency possible
Suitable for dynamic content	Suitable for static content

16 Push vs Pull as Design Issue

Choosing between Push and Pull depends on:

- update frequency,
- bandwidth,
- consistency requirements,

- scalability,
 - server workload.
-

If data changes frequently:

Use Push Protocol.

If data changes rarely:

Use Pull Protocol.

17 Applications

Push Protocol Used In:

- Live notifications
- Stock market updates
- Messaging systems

Pull Protocol Used In:

- Web page refresh
- Browser caching
- Periodic synchronization

18 Industry Example

Netflix Content Delivery

Netflix uses content distribution techniques where:

- popular content may use push-like replication,
- clients also pull data while streaming videos.

19 Important Exam Keywords

Write these keywords in exam:

- Content Distribution
- Update Propagation
- Replica Synchronization
- Push Protocol
- Pull Protocol
- Scalability
- Consistency

20 Memory Trick

“PSI”

Push

- **P** → Proactive
- **S** → Server Sends
- **I** → Immediate Update

Pull

- **C** → Client Requests
- **D** → Delayed Update

21 Conclusion

Push and Pull protocols are important update propagation mechanisms used in distributed systems for content distribution.

Push provides immediate consistency, while Pull reduces communication overhead and improves scalability depending on application requirements.

5/14/2026, 11:42:09 PM



6 Two Phase Commit Protocol — [9M]
[9 Marks]

“What is the distributed commit problem? Discuss how this problem is solved using the two-phase commit protocol with suitable diagram.”

5/14/2026, 11:42:44 PM



Q. What is the distributed commit problem? Discuss how this problem is solved using the Two-Phase Commit Protocol with suitable diagram.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, one transaction may involve multiple computers or databases.

For example:

- money transfer,
- online booking,
- distributed database update.

All systems must either:

- complete transaction successfully,
- OR

- cancel transaction completely.

If some systems commit and others fail:

- data inconsistency occurs.

This problem is called the **Distributed Commit Problem**.

2 Definition

Distributed Commit Problem

The Distributed Commit Problem occurs when multiple distributed processes or databases must agree whether to:

- commit (save),
OR
 - abort (cancel)
a transaction together.
-

3 Why Distributed Commit Problem Occurs

It occurs because:

- systems are distributed,
- failures may happen,
- communication delays occur,
- some nodes may crash.

4 Need of Distributed Commit Protocol

Commit protocol is needed to:

- maintain consistency,
- avoid partial updates,
- ensure reliability,
- coordinate distributed transactions.

5 Real-Life Analogy

Group Project Submission Example

Imagine:

- 4 students must submit one project together.
- If one student fails to submit:
whole project may be rejected.

So:

- all members must agree before final submission.

Similarly:

- distributed systems require global agreement before commit.

6 Two-Phase Commit Protocol (2PC)

Definition

Two-Phase Commit Protocol is a distributed transaction protocol used to ensure that all participating processes either commit or abort a transaction together.

7 Components of 2PC

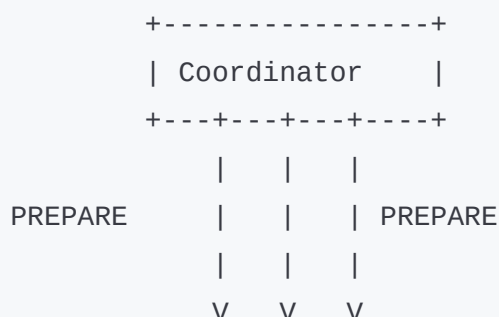
1. Coordinator

Controls transaction process.

2. Participants

Distributed processes/databases involved in transaction.

8 Diagram of Two-Phase Commit Protocol




```
+-----+ +-----+ +-----+
| P1 | | P2 | | P3 |
+-----+ +-----+ +-----+

YES / NO Responses
    ↑
    |
COMMIT / ABORT
```

9 Phases of Two-Phase Commit Protocol

Two-Phase Commit works in:

- Phase 1 → Voting Phase
- Phase 2 → Decision Phase

10 Phase 1 - Voting / Prepare Phase

Step 1

Coordinator sends PREPARE message to all participants.

Step 2

Participants check whether transaction can be completed.

Step 3

Participants reply:

- YES → ready to commit
OR
 - NO → cannot commit
-

1 1 Phase 2 - Commit / Abort Phase

Case 1: All YES

If all participants reply YES:

- coordinator sends COMMIT message.

All participants save transaction permanently.

Case 2: Any NO

If any participant replies NO:

- coordinator sends ABORT message.

All participants cancel transaction.

1 2 Working Example

Online Banking Example

Suppose:

- ₹500 transferred from Account A to Account B.

Transaction involves:

- Bank Server A
- Bank Server B

Using 2PC:

- both servers must agree before money transfer completes.

Otherwise:

- transaction aborts.
-

1 3 Advantages of Two-Phase Commit Protocol

- Maintains consistency
- Reliable distributed transaction handling
- Prevents partial updates

- Ensures atomicity
-

14 Disadvantages

- Slow due to waiting
 - Coordinator failure issue
 - Blocking problem possible
 - Communication overhead
-

15 Features of 2PC

- Atomic transaction execution
 - Global agreement
 - Fault handling
 - Transaction coordination
-

16 Applications

Used in:

- Distributed databases
- Banking systems
- Airline reservation systems
- Cloud transactions

- E-commerce systems
-

17 Industry Example

Amazon Distributed Transactions

Amazon distributed systems use transaction coordination mechanisms similar to 2PC for:

- order processing,
 - payment systems,
 - inventory updates.
-

18 Difference Between Commit and Abort

Commit

Transaction saved

Changes become permanent

Success state

Abort

Transaction canceled

Changes discarded

Failure state

19 Important Exam Keywords

Write these keywords in exam:

- Distributed Commit
 - Coordinator
 - Participants
 - PREPARE Phase
 - COMMIT
 - ABORT
 - Atomicity
 - Distributed Transaction
-

20 Memory Trick

“VCA”

- **V** → Voting Phase
- **C** → Commit
- **A** → Abort

Remember:

2PC = Vote → Commit/Abort

21 Conclusion

The distributed commit problem occurs when distributed processes must agree on transaction completion.

The Two-Phase Commit Protocol solves this problem using coordinated voting and commit/abort phases to maintain consistency in distributed systems.

5/14/2026, 11:42:46 PM



UNIT 5 (Q5, Q6)

🔥 4 TIMES REPEATED

1 DNS vs X.500 — [8M / 9M]

[8 Marks]

“What is a Directory Service? What is the difference between DNS and x500? Describe in detail the components of X.500 service architecture.”

Asked in:

Nov/Dec 2024

May/Jun 2025

May/Jun 2023

[9 Marks]

“What Is a Directory Service? What is the difference between DNS and x.500? Describe in detail the components of X.500 service architecture.”

5/14/2026, 11:49:33 PM



Q. What is a Directory Service? What is the difference between DNS and X.500? Describe in detail the components of X.500 service architecture.

(8/9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, users and applications need to locate:

- computers,
- services,
- printers,
- files,
- and users over network.

Directory Service helps store and search such information efficiently.

Two important directory systems are:

- DNS
- X.500

2 Definition of Directory Service

Directory Service

A Directory Service is a distributed database system that stores information about network resources and provides searching and lookup services.

3 Why Directory Service is Needed

Directory service is needed for:

- resource lookup,
- user management,

- service discovery,
 - network administration,
 - distributed communication.
-

4 Real-Life Analogy

Telephone Directory Example

Imagine:

- telephone directory stores names and phone numbers.

Similarly:

- directory service stores network resource information.
-

5 DNS (Domain Name System)

Definition

DNS is a naming service that converts domain names into IP addresses.

Example

`www.google.com → 142.x.x.x`

6 X.500

Definition

X.500 is a distributed directory service standard used for storing and managing detailed information about users, organizations, and network resources.

7 Difference Between DNS and X.500

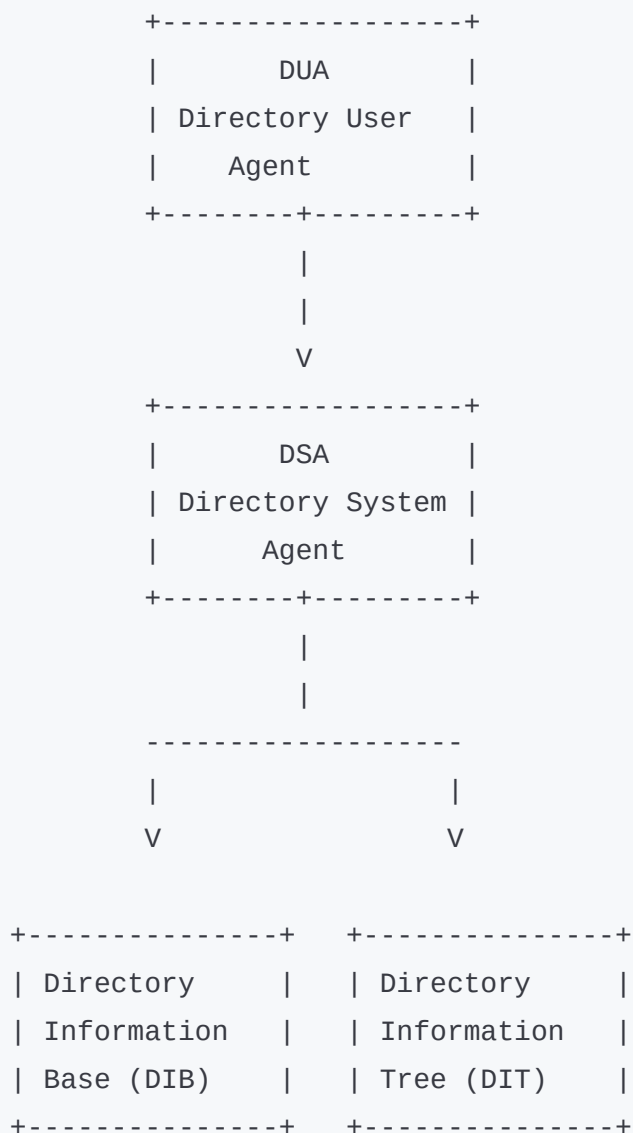
DNS	X.500
Resolves domain names	Provides directory services
Simple naming system	Detailed information storage
Hierarchical structure	Hierarchical directory tree
Stores IP mappings	Stores user/resource details
Lightweight	Complex and feature rich
Mainly Internet use	Enterprise directory systems

8 X.500 Service Architecture

X.500 architecture contains:

1. DUA
2. DSA
3. Directory Information Base (DIB)
4. Directory Information Tree (DIT)

9 Diagram of X.500 Architecture



10 Components of X.500 Architecture

1. DUA (Directory User Agent)

Definition

DUA is the client-side component that allows users/applications to access directory services.

Functions

- Sends requests
 - Searches directory
 - Retrieves information
-

2. DSA (Directory System Agent)

Definition

DSA is the server-side component that manages directory database and processes requests.

Functions

- Stores directory data
 - Handles queries
 - Maintains distributed directory
-

3. Directory Information Base (DIB)

Definition

DIB is the database that stores all directory information.

Stores

- User names
 - Addresses
 - Resources
 - Organizational data
-

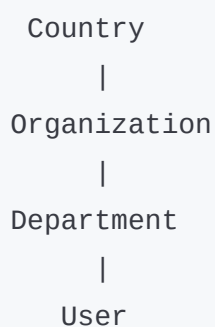
4. Directory Information Tree (DIT)

Definition

DIT is the hierarchical tree structure used to organize directory entries.

Example Structure

```
graph TD; Country --> Organization; Organization --> Department; Department --> User;
```



11 Working of X.500 Directory Service

Step 1

User sends request through DUA.

Step 2

DUA contacts DSA.

Step 3

DSA searches DIB/DIT.

Step 4

Requested information is returned to user.

12 Features of X.500

- Distributed directory management
 - Hierarchical organization
 - Resource lookup
 - Standardized directory structure
 - Scalability
-

13 Advantages of X.500

- Efficient resource management
 - Centralized information storage
 - Scalable directory system
 - Better organization
-

14 Disadvantages of X.500

- Complex implementation
 - High maintenance cost
 - Heavy protocol overhead
-

15 Applications

Used in:

- Enterprise networks

- User authentication
 - Resource management
 - Organizational directories
-

16 Industry Example

Microsoft Active Directory

Microsoft Active Directory works using concepts similar to X.500 directory services for:

- user management,
 - authentication,
 - network resource organization.
-

17 Important Exam Keywords

Write these keywords in exam:

- Directory Service
- DNS
- X.500
- DUA
- DSA
- DIB
- DIT
- Hierarchical Structure

18 Memory Trick

“UDIT”

- **U** → User Agent (DUA)
- **D** → DSA
- **I** → Information Base
- **T** → Tree Structure

Remember:

X.500 Architecture = UDIT

19 Conclusion

Directory services help distributed systems manage and locate network resources efficiently. DNS provides simple name resolution, while X.500 offers detailed hierarchical directory management using components like DUA, DSA, DIB, and DIT.

5/14/2026, 11:49:35 PM



2 Apache Web Server — [9M]
[9 Marks]

“What are web services? Describe with a suitable diagram the general organization of the Apache web server.”

5/14/2026, 11:50:19 PM



Q. What are Web Services? Describe with a suitable diagram the general organization of the Apache Web Server.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

In Distributed Systems, different applications running on different computers need to communicate and share services through the Internet.

Web services help applications exchange data and communicate easily.

Apache Web Server is one of the most widely used web servers for hosting websites and web services.

2 Definition of Web Services

Web Services

Web Services are software components that allow communication and data exchange between applications over a network using standard web protocols.

3 Why Web Services are Needed

Web services are needed for:

- application communication,
 - distributed computing,
 - data sharing,
 - interoperability,
 - remote access of services.
-

4 Real-Life Analogy

Food Delivery App Example

Imagine:

- customer places order,
- restaurant receives request,
- delivery system processes response.

Similarly:

- client sends request,

- web service processes it,
 - server returns response.
-

5 Features of Web Services

- Platform independent
 - Language independent
 - Distributed communication
 - Reusable services
 - Interoperability
-

6 Apache Web Server

Definition

Apache Web Server is an open-source web server software used to host websites, web applications, and web services over the Internet.

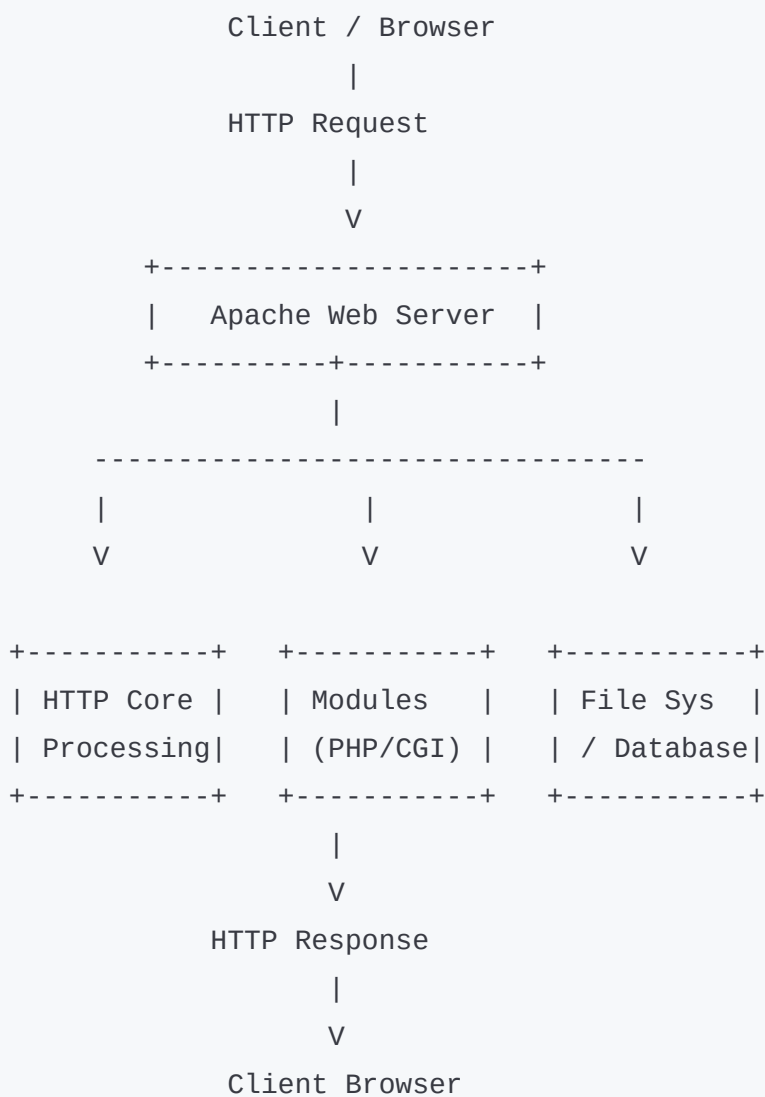
7 General Organization of Apache Web Server

Apache server works using:

- client requests,

- server processing,
- modules,
- and response generation.

8 Diagram of Apache Web Server Architecture



Explanation of Diagram

1. Client Browser

User sends HTTP request through browser.

Example:

- opening website,
 - requesting webpage.
-

2. Apache Web Server

Receives and processes client requests.

3. HTTP Core Processing

Handles:

- request parsing,
 - connection management,
 - protocol processing.
-

4. Modules

Apache supports modules such as:

- PHP,
- CGI,
- SSL,
- Python.

Modules extend server functionality.

5. File System / Database

Server accesses:

- HTML files,
 - images,
 - databases,
 - application data.
-

6. HTTP Response

Processed webpage/data is returned to client.

10 Working of Apache Web Server (Step-by-Step)

Step 1

Client sends HTTP request.

Step 2

Apache server receives request.

Step 3

Server processes request using modules.

Step 4

Requested files/data are fetched.

Step 5

Apache generates HTTP response.

Step 6

Client browser displays webpage.

11 Components of Apache Web Server

Component	Function
HTTP Core	Request handling
Modules	Additional functionality
Configuration Files	Server setup
Virtual Hosts	Multiple website hosting
Logging System	Error and activity logging

12 Advantages of Apache Web Server

- Open source
 - Highly scalable
 - Platform independent
 - Modular architecture
 - Secure and reliable
-

13 Disadvantages

- Complex configuration

- High memory usage under heavy load
 - Performance slower than lightweight servers
-

14 Applications

Used in:

- Website hosting
 - Cloud computing
 - Distributed web applications
 - E-commerce systems
 - REST APIs
-

15 Industry Example

Amazon Web Applications

Many distributed web applications and cloud systems use Apache-based web servers to manage:

- client requests,
 - APIs,
 - distributed services.
-

16 Difference Between Web Server and Web Service

Web Server

Hosts webpages

Handles HTTP requests

Example: Apache

Web Service

Provides application communication

Exchanges data between applications

Example: SOAP/REST API

17 Important Exam Keywords

Write these keywords in exam:

- Web Services
- Apache Web Server
- HTTP Request
- HTTP Response
- Modules
- Distributed Communication
- Interoperability
- Client-Server Architecture

18 Memory Trick

“RHMR”

- **R** → Request
- **H** → HTTP Processing
- **M** → Modules
- **R** → Response

Remember:

Apache Flow = Request → Handle → Module → Response

19 Conclusion

Web services enable communication between distributed applications over networks. Apache Web Server processes HTTP requests using modular architecture and efficiently delivers web pages and services to clients.

5/14/2026, 11:50:21 PM



3 Cooperative Caching — [8M / 9M]
[8 Marks]

"Explain in brief, the two places of client-side web caching?
Explain cooperative caching with suitable diagram."
Asked in:

Nov/Dec 2024
[9 Marks]

"Explain in brief, the two places of client-side web caching?
Explain cooperative caching with suitable diagram."

5/14/2026, 11:53:18 PM



Q. Explain in brief the two places of client-side web caching. Explain cooperative caching with suitable diagram.

(8/9 Marks - SPPU BE IT DS)

1 Simple Introduction

When users open websites repeatedly, downloading same data again and again wastes:

- time,
- bandwidth,
- and server resources.

Caching helps store previously accessed web content temporarily for faster access.

Cooperative caching improves this process by sharing cached data among multiple clients.

2 Definition of Web Caching

Web Caching

Web caching is the process of storing web pages, images, and web content temporarily so that future requests can be served faster.

3 Why Web Caching is Needed

Web caching is needed to:

- reduce loading time,
 - reduce network traffic,
 - improve performance,
 - reduce server load.
-

4 Real-Life Analogy

Shared Classroom Notes Example

Imagine:

- one student downloads notes,
- other students copy from him instead of downloading again.

Similarly:

- cached data can be shared among clients.
-

5 Two Places of Client-Side Web Caching

1. Browser Cache

Definition

Browser cache stores web pages and resources inside the user's browser.

Features

- Local storage
 - Faster webpage loading
 - Reduces repeated downloads
-

Example

Google Chrome cache.

2. Proxy Cache

Definition

Proxy cache stores web content on a proxy server shared by many users.

Features

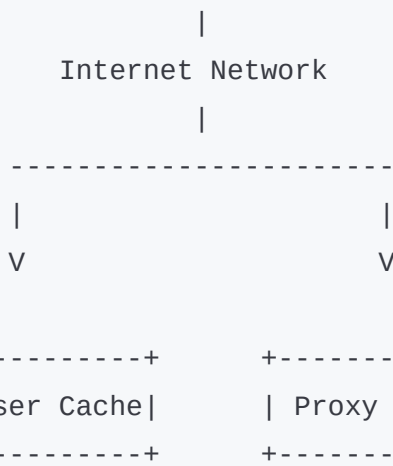
- Shared caching
 - Reduces Internet traffic
 - Improves network performance
-

Example

College network proxy server.

6 Diagram of Client-Side Web Caching

```
+-----+
| Web Server |
+-----+
```



7 Explanation of Diagram

Browser Cache

Stores content locally in browser.

Proxy Cache

Stores content centrally for multiple users.

8 Cooperative Caching

Definition

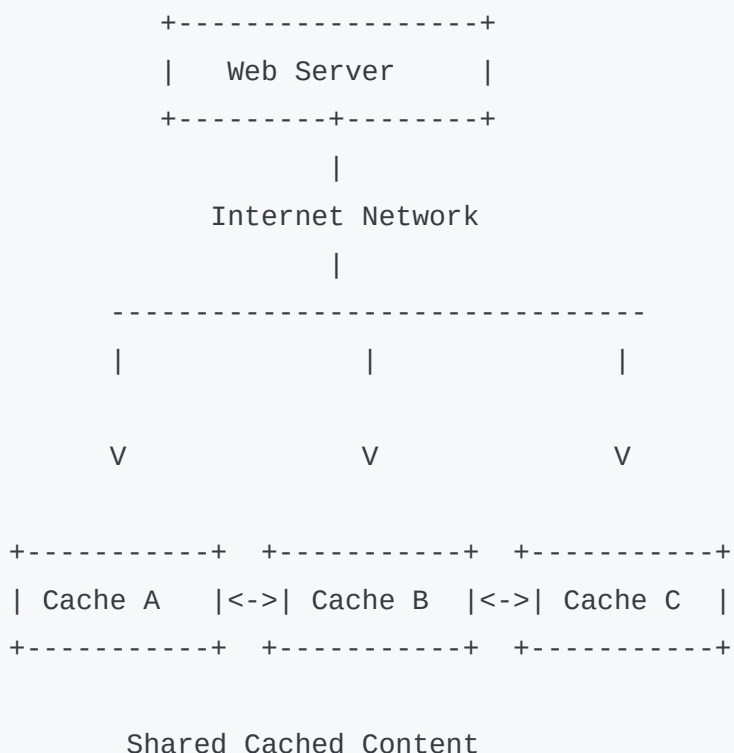
Cooperative caching is a caching technique where multiple caches cooperate and share cached web content with each other.

9 Why Cooperative Caching is Needed

It helps:

- reduce duplicate downloads,
- improve cache utilization,
- decrease server load,
- improve response time.

10 Diagram of Cooperative Caching



1 1 Explanation of Cooperative Caching Diagram

Cache A, B, C

Different caches cooperate together.

Shared Data

If one cache has requested content:

- other caches can use it.
-

Benefit

No need to contact main server repeatedly.

1 2 Working of Cooperative Caching

Step 1

Client requests webpage.

Step 2

Local cache checks for requested content.

Step 3

If not found locally:

- nearby cooperative caches are checked.
-

Step 4

If cache contains data:

- content is shared directly.
-

Step 5

If no cache contains data:

- request goes to main web server.
-

1 3 Advantages of Cooperative Caching

- Faster web access
 - Reduced server load
 - Lower bandwidth usage
 - Better cache utilization
 - Reduced network traffic
-

14 Disadvantages

- Cache consistency issues
 - Complex cache management
 - Extra coordination overhead
-

15 Applications

Used in:

- CDN systems
 - Web browsers
 - Distributed web systems
 - Proxy servers
 - Cloud computing
-

16 Industry Example

Google CDN and Browser Caching

Google services use caching techniques to:

- reduce webpage loading time,
- improve user experience,
- and minimize server traffic.

17 Difference Between Browser Cache and Proxy Cache

Browser Cache	Proxy Cache
Stored locally	Stored centrally
Single-user access	Shared by multiple users
Faster personal access	Reduces network traffic

18 Important Exam Keywords

Write these keywords in exam:

- Web Caching
- Browser Cache
- Proxy Cache
- Cooperative Caching
- Cache Sharing
- Cache Consistency

- Distributed Web System
-

19 Memory Trick

“BPC”

- **B** → Browser Cache
- **P** → Proxy Cache
- **C** → Cooperative Caching

Remember:

Client-side caching = BPC

20 Conclusion

Client-side web caching improves performance by storing frequently accessed web content locally or through proxy servers.

Cooperative caching further enhances efficiency by allowing multiple caches to share cached data in distributed systems.

5/14/2026, 11:53:19 PM



4 BitTorrent — [8M]

[8 Marks]

"Describe the design of a peer-to-peer file sharing application designed to support very large multimedia files. How does the BitTorrent protocol operate?"

Asked in:

May/Jun 2025

[8 Marks]

"Describe the design of a peer-to-peer download system designed to support very large multimedia files. How does the BitTorrent protocol operate?"

5/14/2026, 11:55:45 PM



Q. Describe the design of a peer-to-peer file sharing application designed to support very large multimedia files. How does the BitTorrent protocol operate?

(8 Marks - SPPU BE IT DS)

1 Simple Introduction

Large multimedia files like:

- movies,

- games,
- software,
- videos

are difficult to download from a single server because:

- server becomes overloaded,
- download becomes slow,
- bandwidth usage increases.

BitTorrent solves this problem using Peer-to-Peer (P2P) file sharing.

2 Definition of Peer-to-Peer (P2P) File Sharing

P2P File Sharing

P2P file sharing is a distributed system in which users directly share files with each other without depending completely on a central server.

3 Definition of BitTorrent

BitTorrent

BitTorrent is a peer-to-peer file sharing protocol used to efficiently distribute large multimedia files by dividing files into small pieces shared among multiple peers.

4 Why BitTorrent is Needed

BitTorrent is needed because:

- large files require high bandwidth,
- single server becomes bottleneck,
- downloads become slow,
- scalability problems occur.

BitTorrent improves:

- speed,
 - scalability,
 - bandwidth utilization.
-

5 Real-Life Analogy

Group Notes Sharing Example

Imagine:

- one student has full book.
- Instead of one student giving full book to everyone,
- many students share different pages simultaneously.

This makes sharing much faster.

Similarly:

- BitTorrent divides file into small pieces shared by many peers.

6 Design of Peer-to-Peer File Sharing Application

BitTorrent system contains:

- Tracker
- Seeder
- Peers
- Torrent File

7 Components of BitTorrent

1. Tracker

Definition

Tracker is a server that maintains information about peers sharing a file.

2. Seeder

Definition

Seeder is a peer having complete file.

It uploads file pieces to others.

3. Peer / Leecher

Definition

Peer downloads and uploads file pieces simultaneously.

4. Torrent File

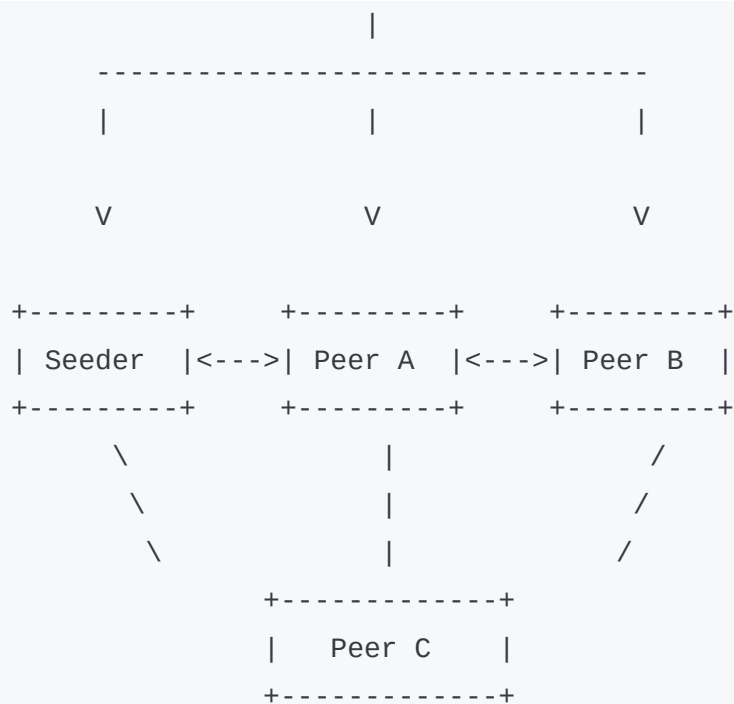
Definition

Small metadata file containing:

- file information,
 - tracker address,
 - piece details.
-

8 Diagram of BitTorrent Architecture

```
+-----+
| Tracker |
+-----+
```



9 Explanation of Diagram

Tracker

Maintains list of active peers.

Seeder

Has complete file and uploads pieces.

Peers

Download and upload file pieces simultaneously.

10 Working of BitTorrent Protocol

Step 1

User downloads torrent file.

Step 2

Torrent file contacts tracker.

Step 3

Tracker provides list of peers.

Step 4

File is divided into small pieces.

Step 5

Peer downloads different pieces from multiple peers simultaneously.

Step 6

Downloaded pieces are shared with other peers.

Step 7

When all pieces are collected:

- complete file is reconstructed.
-

11 Features of BitTorrent

- Peer-to-peer communication
 - Distributed downloading
 - Parallel file transfer
 - Efficient bandwidth usage
 - Scalability
-

12 Advantages of BitTorrent

- Faster downloads
 - Reduced server load
 - Efficient bandwidth utilization
 - Highly scalable
 - Fault tolerant
-

13 Disadvantages

- Security risks
 - Illegal content sharing possible
 - Requires active peers
 - Network congestion possible
-

14 Applications

Used in:

- Multimedia sharing
 - Software distribution
 - Game downloads
 - Large file transfer
-

15 Industry Example

Blizzard Entertainment Game Distribution

Some game distribution systems use peer-assisted downloading techniques similar to BitTorrent for large game updates.

16 Difference Between Client-Server and BitTorrent

Client-Server	BitTorrent
Single server provides file	Multiple peers share file
Server bottleneck possible	Distributed load
Slower for large files	Faster downloads
Less scalable	Highly scalable

17 Important Exam Keywords

Write these keywords in exam:

- Peer-to-Peer
- Seeder
- Peer
- Tracker
- Torrent File
- File Pieces
- Distributed Downloading

- Scalability
-

18 Memory Trick

“SPOT”

- **S** → Seeder
- **P** → Peer
- **O** → Obtain pieces
- **T** → Tracker

Remember:

BitTorrent works using Seeder + Peers + Tracker.

19 Conclusion

BitTorrent is a peer-to-peer file sharing protocol designed for efficient distribution of large multimedia files.

It improves scalability and download speed by dividing files into small pieces shared among multiple peers simultaneously.

5/14/2026, 11:55:47 PM



UNIT 6 (Q7, Q8)

🔥 5 TIMES REPEATED

1 Mach & CHORUS — [9M]

[9 Marks]

“Provide an overview of Mach and CHORUS microkernels. How are memory management techniques used to avoid physical copying of data in Mach and CHORUS?”

Asked in:

Nov/Dec 2024

Nov/Dec 2023

[9 Marks]

“Explain in brief following Microkernels:

i) CHORUS

ii) Mach”

Asked in:

May/Jun 2025

[9 Marks]

“Explain in brief following Microkernels:

i) Mach

ii) CHORUS”

Asked in:

May/Jun 2024

May/Jun 2023

5/14/2026, 11:56:54 PM



Q. Provide an overview of Mach and CHORUS microkernels. How are memory management techniques

used to avoid physical copying of data in Mach and CHORUS?

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

Operating systems contain a kernel that manages:

- memory,
- processes,
- communication,
- hardware resources.

Traditional kernels are large and complex.

Microkernel architecture keeps only essential services inside kernel and moves remaining services to user space.

Mach and CHORUS are famous microkernel-based distributed operating systems.

2 Definition of Microkernel

Microkernel

A Microkernel is a small kernel that provides only essential operating system services such as:

- process management,
- memory management,
- communication.

Other services run outside kernel as user processes.

3 Why Microkernel is Needed

Microkernel architecture is needed because:

- monolithic kernels are large,
- difficult to maintain,
- less secure,
- and difficult to scale.

Microkernels improve:

- modularity,
 - portability,
 - reliability,
 - and distributed computing support.
-

4 Real-Life Analogy

School Principal Example

Imagine:

- principal handles only important decisions.
- teachers handle other activities.

Similarly:

- microkernel handles only core services,
 - remaining services run separately.
-

5 Mach Microkernel

Definition

Mach is a microkernel-based distributed operating system developed at Carnegie Mellon University.

It supports:

- distributed computing,
 - multiprocessing,
 - interprocess communication.
-

6 Features of Mach

- Message-based communication
- Virtual memory management
- Multiprocessing support
- Portability
- Distributed processing

7 CHORUS Microkernel

Definition

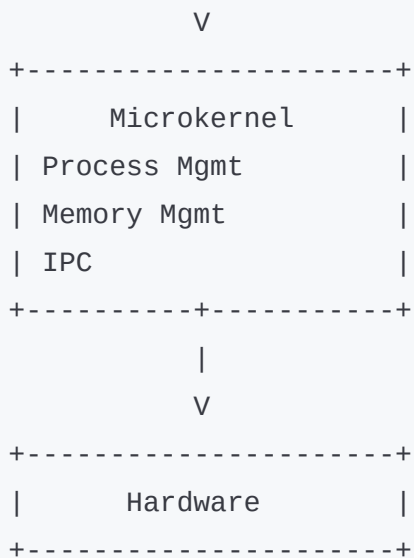
CHORUS is a distributed microkernel operating system designed for distributed and real-time applications.

8 Features of CHORUS

- Distributed object management
 - Modular architecture
 - Real-time support
 - High scalability
 - Distributed communication
-

9 Diagram of Microkernel Architecture

```
+-----+
| User Level Services |
| File System         |
| Device Drivers      |
| Network Services    |
+-----+-----+
      |
```

10 Explanation of Diagram

User-Level Services

Non-essential services run outside kernel.

Microkernel

Handles core operations only.

Hardware

Physical devices and resources.

11 Memory Management Techniques in Mach and CHORUS

Both Mach and CHORUS avoid physical copying of data using advanced memory management techniques.

12 Problem with Physical Copying

When processes exchange data:

- copying large data repeatedly wastes:
 - time,
 - memory,
 - CPU resources.
-

13 Technique Used - Virtual Memory Mapping

Instead of copying actual data:

- systems map same memory pages into another process address space.

This is called:

Copy-on-Write / Memory Mapping

14 Working of Memory Mapping

Step 1

Data stored in memory pages.

Step 2

Kernel maps same pages to receiver process.

Step 3

Actual physical copying is avoided.

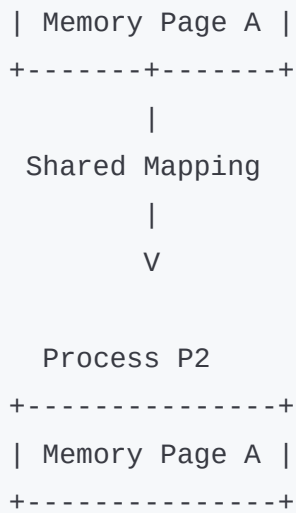
Step 4

Copy created only if process modifies data.

15 Diagram of Memory Mapping

Process P1

+-----+



16 Advantages of Memory Mapping

- Faster communication
- Reduced memory usage
- Improved performance
- Efficient IPC
- Reduced CPU overhead

17 Advantages of Microkernels

- Better modularity
- Improved reliability
- Easier maintenance
- Better portability
- Fault isolation

18 Disadvantages of Microkernels

- More communication overhead
- Slower than monolithic kernels
- Complex IPC mechanisms

19 Applications

Used in:

- Distributed systems
- Embedded systems
- Real-time systems
- Cloud computing

20 Industry Example

Apple macOS Kernel

Apple macOS uses Mach-based kernel concepts for:

- memory management,
- process handling,
- interprocess communication.

21 Difference Between Mach and CHORUS

Mach

CHORUS

Focus on multiprocessing

Focus on distributed systems

Message-based IPC

Distributed object management

Developed at Carnegie Mellon

Designed for distributed applications

22 Important Exam Keywords

Write these keywords in exam:

- Microkernel
 - Memory Mapping
 - Copy-on-Write
 - Virtual Memory
 - IPC
 - Distributed Operating System
 - Mach
 - CHORUS
-

23 Memory Trick

“MIC”

- **M** → Memory Mapping
- **I** → IPC
- **C** → Copy-on-Write

Remember:

Mach & CHORUS avoid copying using memory mapping.

24 Conclusion

Mach and CHORUS are important microkernel-based distributed operating systems. They improve modularity and distributed communication while using virtual memory mapping techniques to avoid costly physical copying of data.

5/14/2026, 11:56:56 PM



3 SOA (Service Oriented Architecture)

Exact Questions

Nov/Dec 2023 — [9 Marks]

“What is Service Oriented Architecture (SOA)? Explain the various SOA components. How does it differ from traditional software architecture?”

May/Jun 2024 — [9 Marks]

“Explain with an application (e.g., Travel Booking Service) various key components of Service Oriented Architecture.”

5/15/2026, 12:06:23 AM



Q. What is Service Oriented Architecture (SOA)? Explain the various SOA components. How does it differ from traditional software architecture?

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

Modern distributed applications are very large and complex.

Instead of building one huge software system, applications are divided into small reusable services.

These services communicate through network and work together.

This approach is called **Service Oriented Architecture (SOA)**.

2 Definition

Service Oriented Architecture (SOA)

SOA is a software architecture style in which applications are built using independent reusable services that communicate over a network.

3 Why SOA is Needed

SOA is needed because:

- large systems are difficult to manage,
 - applications need flexibility,
 - services should be reusable,
 - distributed communication is required.
-

4 Real-Life Analogy

Restaurant Example

Imagine:

- restaurant has separate services:
 - billing,
 - cooking,
 - delivery,
 - payment.

Each service works independently but together provides complete service.

Similarly:

- SOA divides software into reusable services.

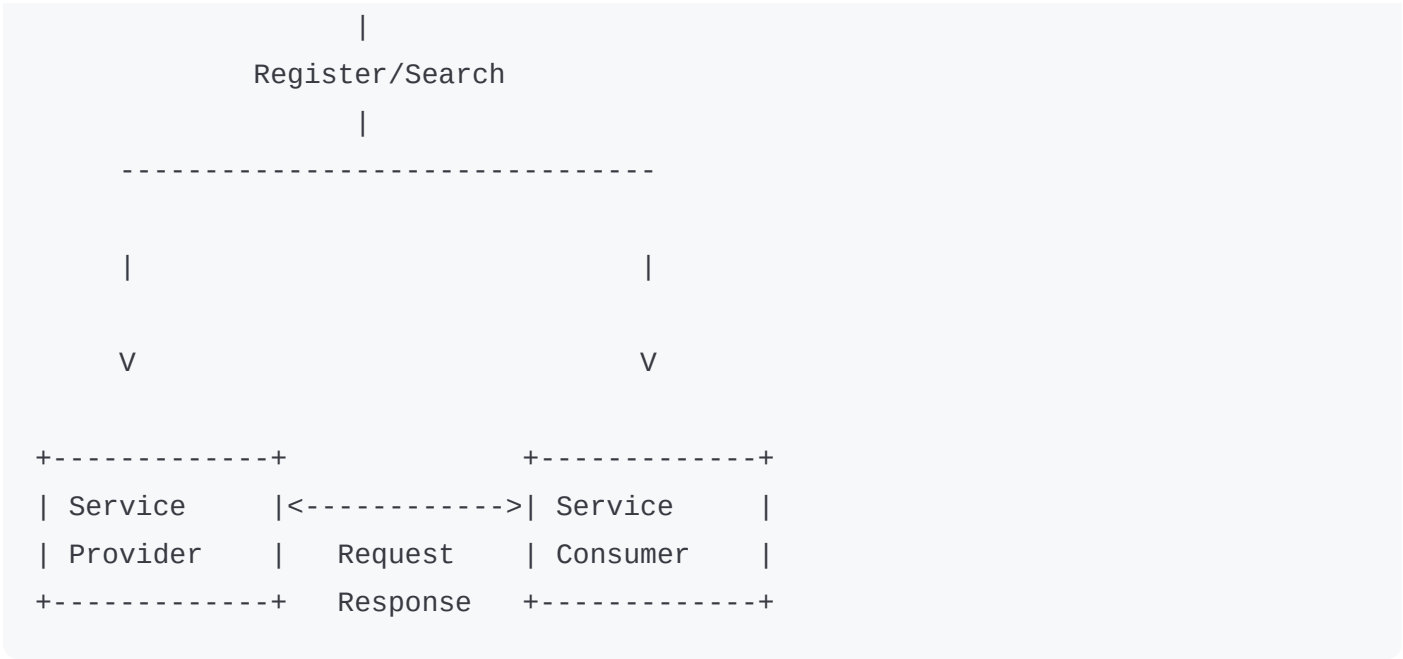
5 Key Components of SOA

Main SOA components are:

1. Service Provider
2. Service Consumer
3. Service Registry
4. Service Contract
5. Messaging System

6 Diagram of SOA Architecture

```
+-----+
| Service Registry |
+-----+
      ^
```



7 Explanation of SOA Components

1. Service Provider

Definition

Service provider creates and offers services.

Example

Payment service provider.

2. Service Consumer

Definition

Service consumer uses requested services.

Example

Online shopping application.

3. Service Registry

Definition

Directory where services are registered and discovered.

Function

Helps consumers find services.

4. Service Contract

Definition

Rules and interface describing:

- service operations,
 - input/output,
 - communication format.
-

5. Messaging System

Definition

Mechanism used for communication between services.

Example:

- SOAP,
 - REST,
 - HTTP.
-

Working of SOA

Step 1

Service provider registers service in registry.

Step 2

Consumer searches required service.

Step 3

Registry provides service details.

Step 4

Consumer communicates with provider.

Step 5

Provider returns response.

Advantages of SOA

- Reusable services
- Better scalability
- Platform independence
- Easy maintenance

- Loose coupling
 - Faster development
-

10 Disadvantages of SOA

- Complex management
 - Security challenges
 - Network dependency
 - Communication overhead
-

11 Difference Between SOA and Traditional Software Architecture

SOA	Traditional Architecture
Service-based	Monolithic application
Loosely coupled	Tightly coupled
Reusable services	Limited reuse
Distributed communication	Centralized structure
Easy scalability	Difficult scalability

12 Applications of SOA

Used in:

- Banking systems
 - E-commerce
 - Cloud computing
 - Travel booking systems
 - Enterprise applications
-

13 Industry Example

Amazon Microservices

Amazon uses service-oriented concepts where:

- payment,
- delivery,
- inventory,
- authentication

work as separate services.

14 Important Exam Keywords

Write these keywords in exam:

- Service Provider
- Service Consumer

- Service Registry
 - Loose Coupling
 - Reusability
 - Distributed Services
 - Messaging Protocol
-

15 Memory Trick

“PCRS”

- **P** → Provider
- **C** → Consumer
- **R** → Registry
- **S** → Service Contract
- **M** → Messaging

Remember:

SOA Components = PCRS

16 Conclusion

SOA is a distributed software architecture based on reusable independent services communicating over networks.

It improves scalability, flexibility, and maintainability compared to traditional monolithic architectures.

Q. Explain with an application (e.g., Travel Booking Service) various key components of Service Oriented Architecture (SOA).

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

Large distributed applications such as travel booking systems require many services working together.

SOA helps divide application into separate reusable services.

2 Definition

Service Oriented Architecture (SOA)

SOA is an architecture where software applications are developed using independent reusable services communicating through a network.

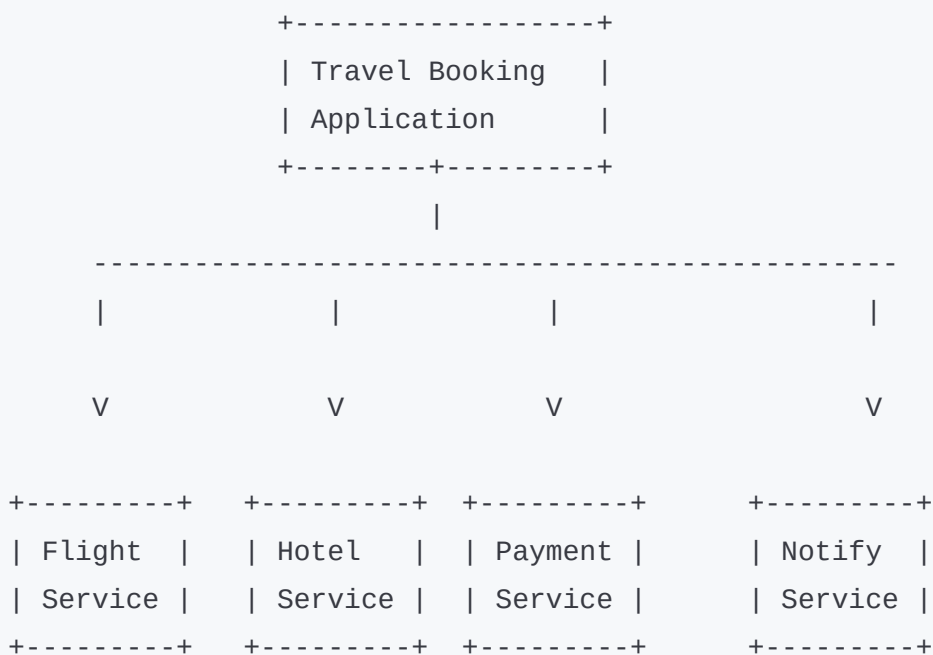
3 Travel Booking Service Example

A travel booking application contains separate services like:

- Flight booking
- Hotel booking
- Payment service
- User authentication
- Notification service

Each service works independently.

4 SOA Architecture Diagram for Travel Booking



5 Key Components of SOA with Travel Booking Example

1. Service Provider

Provides services like:

- flight booking,
 - hotel booking,
 - payment processing.
-

2. Service Consumer

Travel booking application acts as consumer.

3. Service Registry

Stores available services.

Application searches services from registry.

4. Service Contract

Defines:

- booking methods,
 - payment APIs,
 - communication rules.
-

5. Messaging System

Services communicate using:

- REST,
 - SOAP,
 - HTTP.
-

Working of SOA in Travel Booking

Step 1

User searches travel details.

Step 2

Application contacts flight and hotel services.

Step 3

Payment service processes transaction.

Step 4

Notification service sends confirmation.

7 Advantages of SOA

- Reusability
 - Scalability
 - Flexibility
 - Faster integration
 - Better maintenance
-

8 Disadvantages

- Complex communication
- Security issues
- Service dependency
- Network overhead

9 Applications

- Travel booking systems
 - Banking systems
 - E-commerce
 - Cloud applications
-

10 Industry Example

MakeMyTrip Online Booking Services

Travel platforms use SOA-like architecture where:

- booking,
- payments,
- notifications,
- hotel services

work independently.

1 1 Important Exam Keywords

- SOA
- Service Provider
- Service Consumer

- Service Registry
 - Loose Coupling
 - Distributed Services
 - REST API
-

12 Memory Trick

“FHPN”

Travel SOA services:

- **F** → Flight
 - **H** → Hotel
 - **P** → Payment
 - **N** → Notification
-

13 Conclusion

SOA divides distributed applications into reusable independent services.

In travel booking systems, services such as flight booking, hotel booking, and payment processing work together efficiently using SOA principles.

5/15/2026, 12:06:25 AM



4 Prometheus

Exact Questions

Nov/Dec 2023 — [9 Marks]

"Explain in brief, the key features of Prometheus including data model, query language, or alerting rules."

May/Jun 2024 — [9 Marks]

"Compare the following tools for Distributed System Monitoring:
Prometheus, Zabbix, Nagios."

5/15/2026, 12:07:47 AM



Q. Explain in brief the key features of Prometheus including data model, query language, and alerting rules.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

Distributed systems contain many:

- servers,
- applications,
- databases,
- cloud services.

Monitoring these systems is very important to detect:

- failures,
- performance issues,
- overload conditions.

Prometheus is a popular monitoring tool used for distributed systems.

2 Definition

Prometheus

Prometheus is an open-source monitoring and alerting tool used for collecting, storing, querying, and monitoring time-series data in distributed systems.

3 Why Prometheus is Needed

Prometheus is needed because:

- distributed systems are complex,
 - continuous monitoring is required,
 - failures must be detected quickly,
 - performance analysis is important.
-

4 Real-Life Analogy

Hospital Monitoring Example

Imagine:

- hospital monitors patient heartbeat continuously.
- If heartbeat becomes abnormal:
alarm is generated.

Similarly:

- Prometheus continuously monitors system metrics and generates alerts.

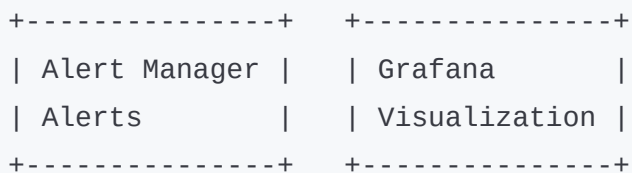
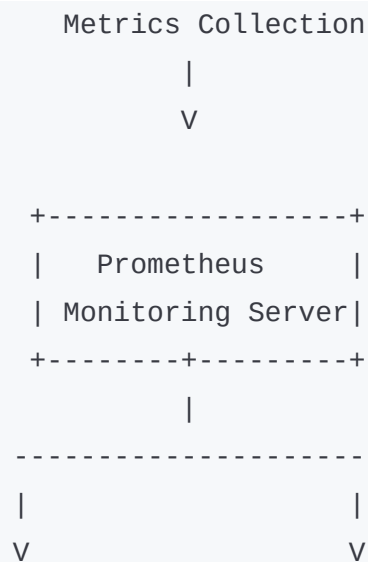
5 Key Features of Prometheus

Main features are:

1. Time-Series Data Model
2. Powerful Query Language (PromQL)
3. Alerting Rules
4. Pull-Based Monitoring
5. Visualization Support

6 Prometheus Architecture Diagram

```
+-----+
| Applications /   |
| Servers / Nodes |
+-----+-----+
      |
```



7 Explanation of Components

1. Metrics Collection

Prometheus collects:

- CPU usage,
- memory usage,
- requests,
- server status.

2. Prometheus Server

Stores and manages monitoring data.

3. Alert Manager

Sends alerts during failures or abnormal conditions.

4. Grafana

Used for dashboards and graphical visualization.

Time-Series Data Model

Definition

Prometheus stores monitoring data as:

time-series data.

Each metric contains:

- metric name,
- timestamp,
- value.

Example

```
CPU_Usage = 70%  
Time = 10:30 AM
```

9 Features of Time-Series Data

- Stores historical data
- Tracks changes over time
- Supports performance analysis

10 Query Language - PromQL

Definition

PromQL (Prometheus Query Language) is used to query and analyze monitoring data.

Example Query

```
cpu_usage > 80%
```

This finds servers with CPU usage greater than 80%.

11 Features of PromQL

- Flexible queries
- Filtering support
- Aggregation operations
- Real-time monitoring

12 Alerting Rules

Definition

Alerting rules define conditions for generating alerts when abnormal system behavior occurs.

Example

```
IF CPU_Usage > 90%  
THEN Generate Alert
```

13 Working of Alerting

Step 1

Prometheus continuously monitors metrics.

Step 2

Alert rules are evaluated.

Step 3

If condition becomes true:

- alert is generated.
-

Step 4

Alert Manager sends notification.

14 Advantages of Prometheus

- Real-time monitoring
- Powerful querying
- Easy alert generation
- Scalable

- Open-source
-

15 Disadvantages

- Limited long-term storage
 - Complex configuration
 - Pull model limitations
-

16 Applications

Used in:

- Cloud computing
 - Kubernetes monitoring
 - Distributed systems
 - Microservices
 - DevOps monitoring
-

17 Industry Example

Google Cloud Monitoring

Google cloud-native systems use monitoring concepts similar to Prometheus for:

- server monitoring,

- distributed system analysis,
 - performance tracking.
-

18 Important Exam Keywords

Write these keywords in exam:

- Prometheus
 - Time-Series Data
 - PromQL
 - Alerting Rules
 - Monitoring
 - Metrics Collection
 - Distributed System Monitoring
-

19 Memory Trick

“MQA”

- **M** → Metrics
- **Q** → Query Language
- **A** → Alerting

Remember:

Prometheus = Metrics + Queries + Alerts

20 Conclusion

Prometheus is a powerful distributed system monitoring tool that collects and analyzes time-series data.

Its features like PromQL, alerting rules, and scalable monitoring make it suitable for modern distributed environments.

Q. Compare the following tools for Distributed System Monitoring: Prometheus, Zabbix, Nagios.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

Distributed systems require continuous monitoring to detect:

- failures,
- performance issues,
- network problems.

Prometheus, Zabbix, and Nagios are popular monitoring tools used in distributed environments.

2 Monitoring Tools Overview

1. Prometheus

Open-source monitoring tool focused on:

- time-series metrics,
 - cloud-native systems,
 - alerting.
-

2. Zabbix

Enterprise monitoring tool used for:

- servers,
 - networks,
 - infrastructure monitoring.
-

3. Nagios

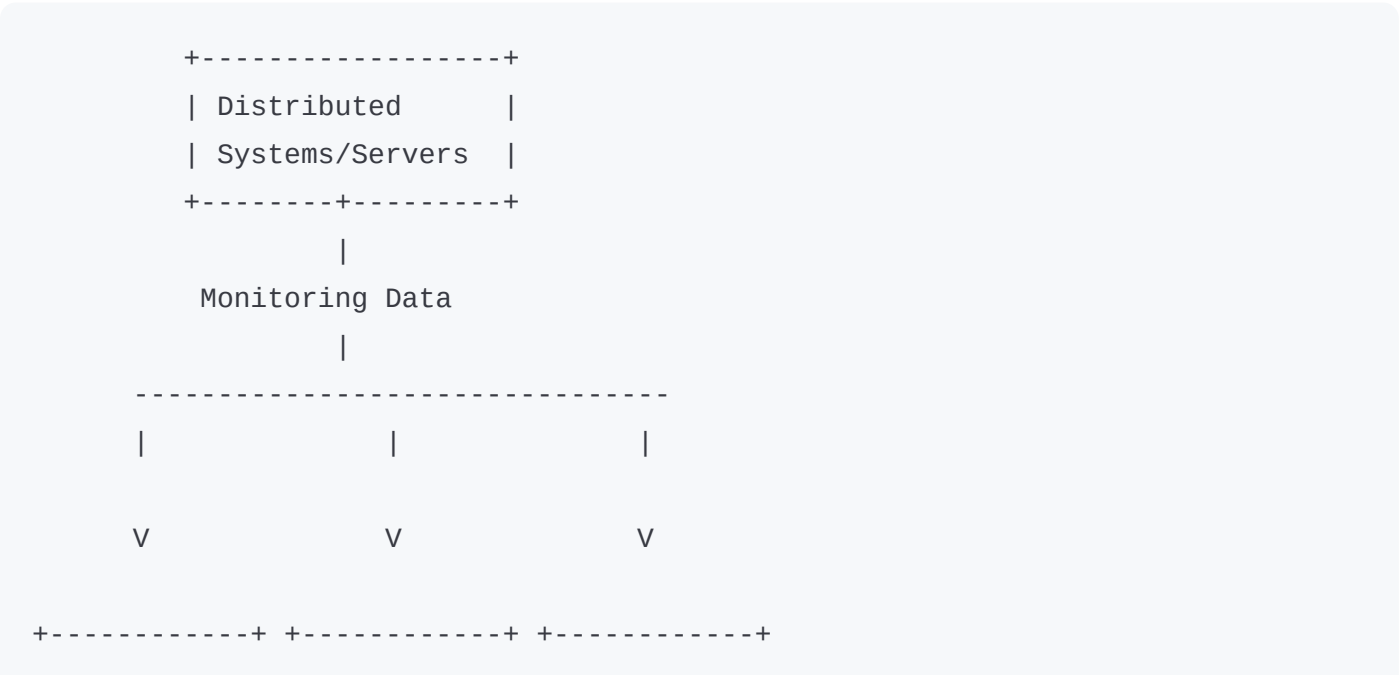
Monitoring tool mainly focused on:

- system availability,
 - network monitoring,
 - alert management.
-

3 Comparison Table

Feature	Prometheus	Zabbix	Nagios
Monitoring Type	Time-series monitoring	Infrastructure monitoring	Network/System monitoring
Data Collection	Pull model	Agent-based	Plugin-based
Query Language	PromQL	Basic queries	Limited query support
Visualization	Grafana integration	Built-in dashboards	Basic interface
Alerting	Advanced alerting	Strong alerting	Basic alerting
Scalability	Highly scalable	Moderate scalability	Less scalable
Cloud Native Support	Excellent	Moderate	Limited
Ease of Setup	Moderate	Easy	Moderate

4 Diagram of Monitoring Architecture



5 Advantages of Prometheus

- Excellent for cloud systems
- Powerful query language
- Real-time monitoring

6 Advantages of Zabbix

- Easy management
- Strong dashboards
- Good infrastructure monitoring

7 Advantages of Nagios

- Stable monitoring
- Good plugin ecosystem
- Network monitoring support

8 Applications

Tool	Applications
Prometheus	Kubernetes, cloud systems
Zabbix	Enterprise infrastructure
Nagios	Server/network monitoring

9 Industry Example

Netflix Distributed Monitoring

Large distributed platforms use monitoring tools similar to:

- Prometheus,
- Zabbix,
- Nagios

to monitor:

- servers,
- services,
- distributed applications.

10 Important Exam Keywords

- Monitoring

- Prometheus
 - Zabbix
 - Nagios
 - Alerting
 - Metrics Collection
 - Distributed Systems
-

11 Memory Trick

“PZN”

- **P** → Prometheus
- **Z** → Zabbix
- **N** → Nagios

Remember:

Popular Monitoring Tools = PZN

12 Conclusion

Prometheus, Zabbix, and Nagios are important distributed system monitoring tools. Each tool provides monitoring, alerting, and performance analysis features suitable for different distributed environments.

5/15/2026, 12:07:49 AM



5 Wearable Devices + PVM + JINI
Exact Questions
Nov/Dec 2024 — [9 Marks]

“Explain the following in brief:
i) Wearable devices
ii) PVM
iii) JINI”

5/15/2026, 12:08:35 AM



Q. Explain the following in brief:

i) Wearable Devices

ii) PVM

iii) JINI

(9 Marks - SPPU BE IT DS)

i) Wearable Devices

1 Simple Introduction

Technology devices are becoming smaller and smarter.

Some devices can be worn directly on the body and connected to networks for monitoring and communication.

These are called wearable devices.

2 Definition

Wearable Devices

Wearable devices are smart electronic devices worn on the human body that can collect, process, and exchange data through networks.

3 Features of Wearable Devices

- Small and portable
 - Wireless communication
 - Sensor-based monitoring
 - Real-time data collection
 - Smart connectivity
-

4 Examples of Wearable Devices

- Smart watches
- Fitness bands

- Smart glasses
 - Health monitoring devices
-

5 Real-Life Analogy

Smart Watch Example

A smartwatch:

- tracks heartbeat,
- shows notifications,
- connects to mobile phone.

Similarly:

- wearable devices collect and share data continuously.
-

6 Diagram of Wearable Device System

```
+-----+
| Wearable Device|
| (Smart Watch) |
+-----+-----+
      |
    Wireless Network
      |
      V
+-----+
```

7 Applications

Used in:

- Healthcare
- Fitness tracking
- Military systems
- Smart homes
- IoT applications

8 Advantages

- Continuous monitoring
- Portability
- Real-time tracking
- Improved healthcare

9 Disadvantages

- Privacy concerns
- Limited battery life

- Security risks
-

10 Industry Example

Apple Smart Watch

Apple smart watches monitor:

- heartbeat,
 - fitness,
 - notifications,
 - health data.
-

1 1 Important Keywords

- Smart Devices
 - Sensors
 - Wireless Communication
 - Real-Time Monitoring
 - IoT
-

ii) PVM (Parallel Virtual Machine)

1 Simple Introduction

Distributed systems often combine multiple computers to solve large problems together.

PVM allows many computers to work like one virtual parallel computer.

2 Definition

PVM

PVM (Parallel Virtual Machine) is a software system that connects multiple computers into one virtual parallel computing environment.

3 Purpose of PVM

PVM is used for:

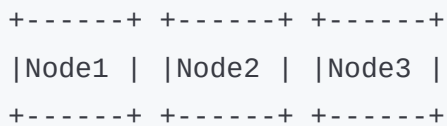
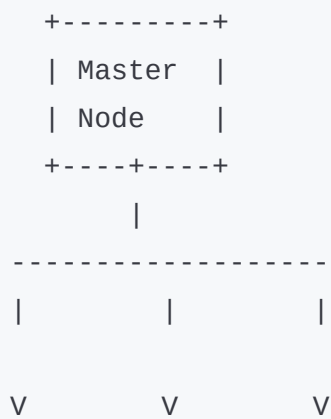
- parallel processing,
 - distributed computing,
 - high-performance computation.
-

4 Features of PVM

- Parallel task execution
- Message passing

- Heterogeneous system support
- Distributed resource sharing

5 Diagram of PVM



6 Working of PVM

Step 1

Multiple computers are connected.

Step 2

Tasks are divided among nodes.

Step 3

Nodes communicate using message passing.

Step 4

Results are combined.

7 Advantages

- Faster computation
 - Resource sharing
 - Scalability
 - Cost-effective parallel computing
-

8 Disadvantages

- Communication overhead
- Complex programming

- Network dependency
-

9 Applications

Used in:

- Scientific computing
 - Weather forecasting
 - Research simulations
-

10 Industry Example

NASA Scientific Computation

Large scientific simulations use parallel distributed computing concepts similar to PVM.

11 Important Keywords

- Parallel Computing
- Virtual Machine
- Distributed Processing
- Message Passing
- Scalability

iii) JINI

1 Simple Introduction

Distributed systems contain many devices and services connected dynamically.

JINI helps devices discover and communicate with services automatically over network.

2 Definition

JINI

JINI is a Java-based distributed networking technology used for service discovery, distributed communication, and dynamic service management.

3 Purpose of JINI

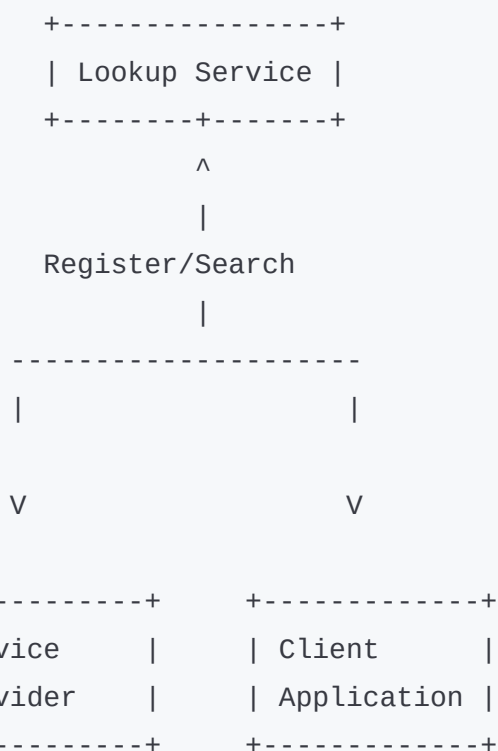
JINI is used for:

- automatic service discovery,
 - distributed communication,
 - dynamic network services.
-

4 Components of JINI

- Service Provider
- Client
- Lookup Service

5 Diagram of JINI Architecture



6 Working of JINI

Step 1

Service provider registers service.

Step 2

Client searches required service.

Step 3

Lookup service provides service information.

Step 4

Client communicates with service provider.

Features of JINI

- Dynamic service discovery
 - Java-based architecture
 - Distributed communication
 - Plug-and-play networking
-

8 Advantages

- Easy service discovery
 - Flexible distributed system
 - Automatic configuration
-

9 Disadvantages

- Java dependency
 - Complex management
 - Security concerns
-

10 Applications

Used in:

- Distributed systems
 - Smart devices
 - Network services
 - IoT systems
-

1 1 Industry Example

Oracle Java-Based Distributed Services

JINI concepts are used in Java-based distributed networking environments.

1 2 Important Keywords

- Service Discovery
 - Lookup Service
 - Distributed Networking
 - Java Technology
 - Dynamic Services
-

1 3 Memory Trick

“WPJ”

- **W** → Wearable Devices
- **P** → PVM
- **J** → JINI

Remember:

Unit 6 short notes = WPJ

14 Conclusion

Wearable devices enable smart monitoring and communication, PVM supports parallel distributed computing, and JINI provides dynamic service discovery in distributed systems.

5/15/2026, 12:08:38 AM



6 Nagios + Zabbix Comparison
May/Jun 2024 — [9 Marks]

"Compare the following tools for Distributed System Monitoring:
Prometheus, Zabbix, Nagios."

5/15/2026, 12:09:24 AM



Q. Compare the following tools for Distributed System Monitoring: Prometheus, Zabbix, Nagios.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

Distributed systems contain:

- servers,
- databases,
- cloud services,
- applications,
- networks.

Monitoring tools are used to:

- detect failures,
- track performance,

- generate alerts,
- and maintain system health.

Three important monitoring tools are:

- Prometheus
 - Zabbix
 - Nagios
-

2 Definition of Monitoring Tool

Distributed System Monitoring Tool

A monitoring tool is software used to observe, analyze, and manage the performance and health of distributed systems.

3 Why Monitoring Tools are Needed

Monitoring tools help to:

- detect failures quickly,
 - monitor server health,
 - improve performance,
 - reduce downtime,
 - generate alerts.
-

4 Real-Life Analogy

Hospital Monitoring Example

Imagine:

- doctors monitor patient heartbeat,
- oxygen level,
- blood pressure.

Similarly:

- monitoring tools continuously monitor system performance and health.
-

5 Overview of Tools

1. Prometheus

Open-source monitoring tool mainly used for:

- cloud-native systems,
 - time-series monitoring,
 - Kubernetes environments.
-

2. Zabbix

Enterprise monitoring tool used for:

- infrastructure monitoring,
- network monitoring,
- server monitoring.

3. Nagios

Monitoring tool mainly focused on:

- system availability,
- server monitoring,
- alert management.

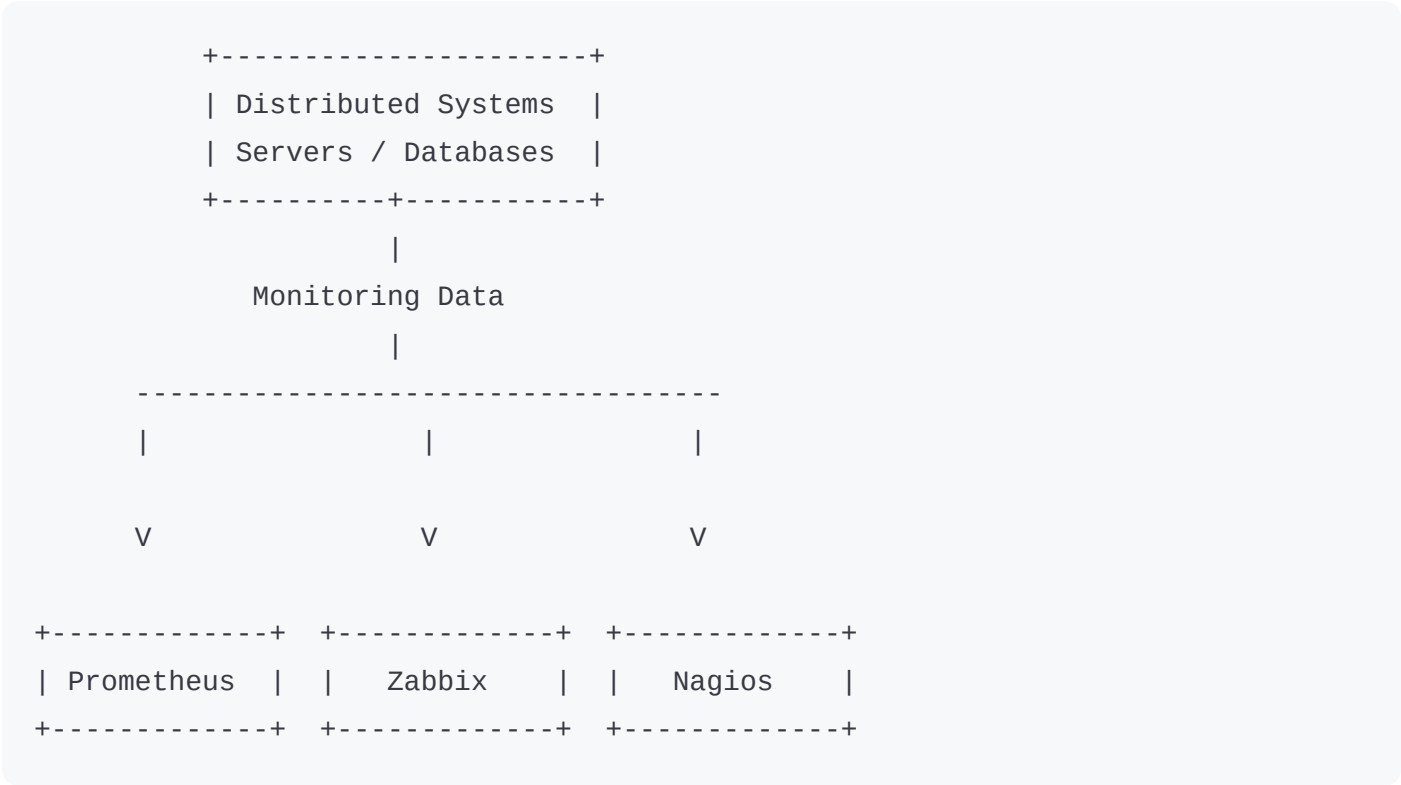
6

Comparison Table

Feature	Prometheus	Zabbix	Nagios
Monitoring Type	Time-series monitoring	Infrastructure monitoring	System & network monitoring
Data Collection	Pull-based	Agent-based	Plugin-based
Query Language	PromQL	Basic queries	Limited query support
Visualization	Grafana integration	Built-in dashboards	Basic interface
Alerting	Advanced alert rules	Strong alerting	Basic alerting
Scalability	Highly scalable	Moderate scalability	Less scalable
Cloud Support	Excellent	Moderate	Limited

Feature	Prometheus	Zabbix	Nagios
Kubernetes Support	Excellent	Moderate	Limited
Ease of Setup	Moderate	Easy	Moderate
Performance Monitoring	Excellent	Good	Good

7 Diagram of Monitoring Architecture



8 Explanation of Diagram

Distributed Systems

Generate monitoring data.

Prometheus

Collects time-series metrics.

Zabbix

Monitors infrastructure and servers.

Nagios

Checks network and service availability.

9 Advantages of Prometheus

- Excellent cloud-native monitoring
 - Powerful query language
 - Real-time monitoring
 - Highly scalable
-

10 Advantages of Zabbix

- Easy infrastructure monitoring
 - Built-in dashboards
 - Good alerting system
-

1 1 Advantages of Nagios

- Stable monitoring
 - Strong plugin ecosystem
 - Reliable network monitoring
-

1 2 Disadvantages

Tool	Disadvantages
Prometheus	Limited long-term storage
Zabbix	Heavy resource usage
Nagios	Complex configuration

1 3 Applications

Tool	Applications
Prometheus	Kubernetes, cloud monitoring
Zabbix	Enterprise monitoring
Nagios	Network/server monitoring

14 Industry Example

Netflix Distributed Monitoring

Large distributed platforms use monitoring tools similar to:

- Prometheus,
- Zabbix,
- Nagios

to monitor:

- servers,
- applications,
- cloud infrastructure.

15 Difference Summary

Prometheus	Zabbix	Nagios
Best for cloud-native systems	Best for enterprise infrastructure	Best for network monitoring

Prometheus

Advanced metrics analysis

Zabbix

Strong dashboard support

Nagios

Plugin-based monitoring

16 Important Exam Keywords

Write these keywords in exam:

- Monitoring
 - Prometheus
 - Zabbix
 - Nagios
 - Alerting
 - Metrics Collection
 - Infrastructure Monitoring
 - Distributed Systems
-

17 Memory Trick

“PZN”

- **P** → Prometheus
- **Z** → Zabbix
- **N** → Nagios

Remember:

18 Conclusion

Prometheus, Zabbix, and Nagios are important monitoring tools used in distributed systems. Each tool provides monitoring, alerting, and performance analysis features suitable for different system environments and monitoring requirements.

5/15/2026, 12:09:26 AM



7 Wearable Computing Problems
May/Jun 2024 — [9 Marks]

"How wearable devices work in distributed systems?
Discuss the problems involved with wearable computing."

5/15/2026, 12:10:35 AM



Q. How wearable devices work in distributed systems? Discuss the problems involved with wearable computing.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

Technology is becoming smarter and smaller.

Devices like:

- smart watches,
- fitness bands,
- smart glasses

can connect to networks and exchange data continuously.

These devices are called **Wearable Devices** and are important parts of distributed systems.

2 Definition of Wearable Computing

Wearable Computing

Wearable computing is a technology in which smart electronic devices worn on the body collect, process, and exchange data through distributed networks.

3 Wearable Devices in Distributed Systems

Wearable devices work as distributed nodes that:

- collect data,
 - communicate through networks,
 - send information to servers/cloud systems,
 - and receive responses.
-

4 Real-Life Analogy

Smart Fitness Watch Example

A smart watch:

- measures heartbeat,
- sends data to mobile phone/cloud,
- receives notifications.

Similarly:

- wearable devices continuously communicate with distributed systems.
-

5 Working of Wearable Devices in Distributed Systems

Step 1

Sensors collect data from user.

Examples:

- heartbeat,
 - temperature,
 - movement.
-

Step 2

Data is processed locally.

Step 3

Device sends data through:

- Wi-Fi,
 - Bluetooth,
 - Internet.
-

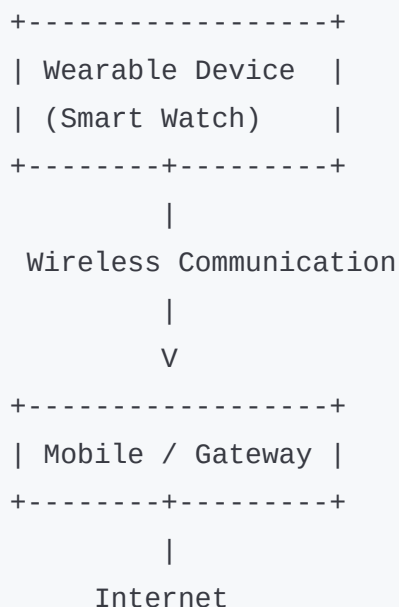
Step 4

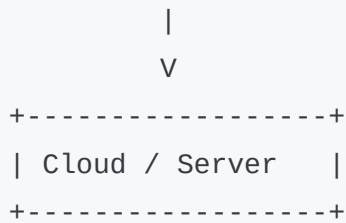
Cloud/distributed server stores and analyzes data.

Step 5

Processed information is returned to user.

6 Diagram of Wearable Computing System





7 Explanation of Diagram

Wearable Device

Collects user data.

Mobile/Gateway

Transfers data to Internet/cloud.

Cloud Server

Processes and stores distributed data.

8 Features of Wearable Devices

- Real-time monitoring
- Mobility

- Wireless communication
 - Sensor integration
 - Continuous connectivity
-

9 Applications of Wearable Computing

Used in:

- Healthcare
 - Fitness tracking
 - Military systems
 - Smart homes
 - Industrial monitoring
-

10 Problems Involved with Wearable Computing

1. Limited Battery Life

Wearable devices are small, so battery capacity is limited.

2. Security Problems

Personal data may be hacked or leaked.

3. Privacy Issues

Devices continuously collect sensitive user data.

4. Limited Processing Power

Wearable devices have less CPU and memory.

5. Connectivity Problems

Network interruptions affect communication.

6. Small Display Size

Limited screen size reduces usability.

7. Data Synchronization Issues

Maintaining consistency with cloud/server becomes difficult.

8. Heat and Comfort Issues

Continuous usage may cause heating or discomfort.

1 1 Advantages of Wearable Computing

- Continuous monitoring
 - Real-time tracking
 - Mobility
 - Improved healthcare
 - Smart automation
-

1 2 Disadvantages

- Expensive devices
- Privacy risks
- Limited battery
- Security vulnerabilities

13 Industry Example

Apple Smart Watch

Apple smart watches monitor:

- health,
- heart rate,
- activity tracking,
- notifications

using distributed cloud-based services.

14 Difference Between Mobile Computing and Wearable Computing

Mobile Computing	Wearable Computing
Uses smartphones/laptops	Uses body-worn devices
Larger devices	Small portable devices
Higher processing power	Limited processing power

15 Important Exam Keywords

Write these keywords in exam:

- Wearable Computing
 - Sensors
 - Wireless Communication
 - Distributed Systems
 - Real-Time Monitoring
 - Privacy
 - Security
 - Cloud Connectivity
-

16 Memory Trick

“BSPC”

Problems in wearable computing:

- **B** → Battery issue
 - **S** → Security issue
 - **P** → Privacy issue
 - **C** → Connectivity issue
-

17 Conclusion

Wearable devices work as distributed nodes that collect and exchange real-time data through networks.

Although wearable computing provides mobility and smart monitoring, it faces challenges such as battery limitations, security risks, and connectivity problems.

5/15/2026, 12:10:38 AM



7 Wearable Computing Problems
May/Jun 2024 — [9 Marks]

"How wearable devices work in distributed systems?
Discuss the problems involved with wearable computing."

5/15/2026, 12:12:01 AM



Q. How wearable devices work in distributed systems? Discuss the problems involved with wearable computing.

(9 Marks - SPPU BE IT DS)

1 Simple Introduction

Modern distributed systems include smart wearable devices like:

- smart watches,
- fitness bands,
- smart glasses,
- health monitoring devices.

These devices continuously collect and exchange data through networks.

Wearable computing combines:

- sensors,

- wireless communication,
 - cloud computing,
 - and distributed processing.
-

2 Definition of Wearable Computing

Wearable Computing

Wearable computing is a technology in which electronic devices worn on the human body collect, process, and communicate data through distributed systems.

3 Wearable Devices in Distributed Systems

Wearable devices act as distributed nodes that:

- collect user data,
 - communicate with servers/cloud,
 - and provide real-time services.
-

4 Real-Life Analogy

Smart Watch Example

A smartwatch:

- tracks heartbeat,
- counts steps,
- sends data to mobile/cloud,
- receives notifications.

Similarly:

- wearable devices continuously communicate in distributed systems.

5 Working of Wearable Devices in Distributed Systems

Step 1 - Data Collection

Sensors collect:

- heartbeat,
- temperature,
- motion,
- location data.

Step 2 - Local Processing

Device performs basic processing.

Step 3 - Wireless Communication

Data is transmitted using:

- Bluetooth,
- Wi-Fi,
- Internet.

Step 4 - Cloud Processing

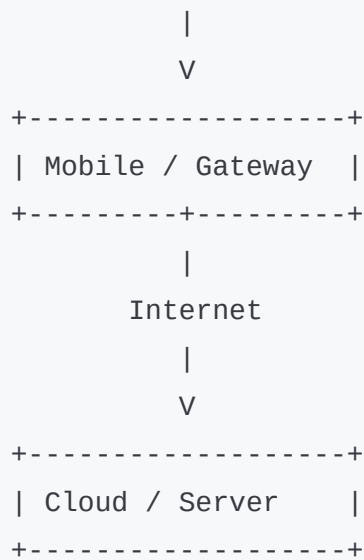
Distributed cloud servers analyze and store data.

Step 5 - Response Generation

Processed information is returned to user device.

6 Diagram of Wearable Computing System

```
+-----+
| Wearable Device |
| (Smart Watch)   |
+-----+
      |
    Wireless Network
```

7 Explanation of Diagram

Wearable Device

Collects sensor data.

Mobile/Gateway

Transfers data to distributed servers.

Cloud Server

Stores and processes data.

8 Features of Wearable Computing

- Real-time monitoring
 - Wireless communication
 - Mobility
 - Sensor integration
 - Continuous connectivity
-

9 Applications of Wearable Computing

Used in:

- Healthcare
 - Fitness tracking
 - Military systems
 - Smart homes
 - Industrial monitoring
-

10 Problems Involved with Wearable Computing

1. Limited Battery Life

Wearable devices are small, so battery capacity is limited.

Continuous monitoring consumes power quickly.

2. Security Issues

Sensitive user data may be hacked or stolen.

3. Privacy Problems

Devices continuously collect personal information such as:

- health data,
 - location,
 - activity tracking.
-

4. Limited Processing Power

Wearable devices have:

- small CPU,
 - limited memory,
 - limited storage.
-

5. Connectivity Problems

Network interruption affects communication with cloud systems.

6. Small Display and Interface

Limited screen size reduces usability.

7. Data Synchronization Issues

Maintaining consistency between device and cloud becomes difficult.

8. Heat and Comfort Problems

Long-time usage may create:

- heat,
 - discomfort,
 - skin irritation.
-

Advantages of Wearable Computing

- Continuous health monitoring
 - Mobility
 - Real-time alerts
 - Smart automation
 - Better healthcare support
-

1 2 Disadvantages

- Privacy risks
 - Battery limitations
 - High cost
 - Security vulnerabilities
-

1 3 Industry Example

Apple Smart Watch Ecosystem

Apple smart watches use distributed cloud services for:

- fitness monitoring,
 - heartbeat tracking,
 - notification systems,
 - health analytics.
-

14 Difference Between Mobile Computing and Wearable Computing

Mobile Computing

Smartphone/laptop based

Larger devices

More processing power

Wearable Computing

Body-worn devices

Compact devices

Limited processing power

15 Important Exam Keywords

Write these keywords in exam:

- Wearable Computing
- Sensors
- Wireless Communication
- Distributed Systems
- Cloud Connectivity
- Real-Time Monitoring
- Privacy
- Security

16 Memory Trick

"BSPC"

Problems of wearable computing:

- **B** → Battery issue
 - **S** → Security issue
 - **P** → Privacy issue
 - **C** → Connectivity issue
-

17 Conclusion

Wearable devices work as distributed nodes that collect and exchange real-time data through networks and cloud systems.

Although wearable computing improves smart monitoring and mobility, it faces challenges such as battery limitations, security risks, privacy issues, and connectivity problems.

5/15/2026, 12:12:02 AM